OPTIMIZATION FOR DEEP LEARNING

Personal Notes

ZACH LIU

ABSTRACT

Notes on optimization for deep learning, from foundations to the curvature methods behind SOFO. Part I collects the toolkit: optimality conditions, linear algebra, function regularity (convexity, smoothness, the Polyak–Łojasiewicz inequality), Lagrange and KKT conditions, and automatic differentiation, including the Jacobian-vector and GGN-vector products that second-order methods are built from. Part II treats every method as the minimiser of a local quadratic model and covers gradient descent, line search and momentum. Part III develops stochastic-gradient theory (descent minus noise, the noise floor, minibatching), learning-rate schedules and large-batch training, and the adaptive methods from gradient clipping to Adam and AdamW. Part IV covers Newton's method, trust regions and damping, the generalised Gauss–Newton matrix, the Fisher information and natural gradient, their low-rank structure and the measured loss landscape of neural networks, then the ways to approximate the curvature at scale: Hessian-free methods, Kronecker-factored approximations (KFAC, Shampoo, SOAP) and diagonal scaling. Part V recasts optimizers as steepest descent under a choice of norm, which recovers sign descent, Shampoo and Muon, and natural gradient. The companion SOFO notes build on Parts I and IV: SOFO sketches the GGN in a random subspace using forward-mode products, and SOFO-NPG applies the same sketch to the Fisher of a softmax policy, which equals its GGN.

DISCLOSURE

These notes were written collaboratively with Claude (Anthropic). I sketched the high-level structure, and Claude and I edited each paragraph and sentence iteratively. I take responsibility for any errors.

CONTENTS

I Foundations 4

1 Problem Setup and Optimality 4
- 1.1 Gradient and Hessian . . . 4
- 1.2 Taylor Expansion . . . 4
- 1.3 Necessary and Sufficient Conditions . . . 5

2 Linear Algebra Toolkit 6
- 2.1 Symmetric Eigendecomposition and the Rayleigh Quotient . . . 6
- 2.2 Positive Semi-Definite Matrices . . . 6
- 2.3 Singular Value Decomposition . . . 7
- 2.4 Moore–Penrose Pseudoinverse . . . 7
- 2.5 Range, Null Space, and Projections . . . 8

3 Function Regularity 9
- 3.1 Convexity . . . 9
- 3.2 Lipschitz Smoothness and the Descent Lemma . . . 10
- 3.3 Strong Convexity . . . 10
- 3.4 The Polyak–Łojasiewicz Inequality . . . 11
- 3.5 Condition Numbers . . . 12
- 3.6 Summary . . . 12

4 Lagrangian and KKT Conditions 12
- 4.1 Equality Constraints and Lagrange Multipliers . . . 12
- 4.2 Inequality Constraints and KKT . . . 13

5 Automatic Differentiation 14
5.1 Jacobians and Jacobian–Vector Products . . . 14
5.2 Forward and Reverse Mode on Chains . . . 15
5.3 General Computation Graphs . . . 16
5.4 Complexity Comparison . . . 17
5.5 Composing Modes: Hessian–Vector and GGN–Vector Products . . . 17

II First-Order Methods 18
6 Local Models and Convergence Templates 18
6.1 The Local Quadratic Model . . . 18
6.2 Rate Taxonomy . . . 18
6.3 Linear Convergence under a PL-type Bound . . . 18
6.4 Sublinear Convergence under Convexity . . . 19
6.5 Stationarity in the Non-Convex Case . . . 19
7 Gradient Descent 19
7.1 Per-Step Descent . . . 20
7.2 Convergence in Four Regimes . . . 20
7.3 Line Search . . . 21
8 Momentum and Acceleration 21
8.1 Heavy Ball . . . 21
8.2 Nesterov Accelerated Gradient . . . 22
8.3 Momentum in the Stochastic Setting . . . 22

III Stochastic Optimization 23
9 Why Stochastic Methods 23
9.1 Expected and Empirical Risk . . . 23
9.2 Stochastic versus Batch . . . 23
9.3 Work Complexity for Large-Scale Learning . . . 24
10 Convergence of Stochastic Gradient 24
10.1 The Generalised SG Algorithm and Assumptions . . . 24
10.2 Two Fundamental Lemmas . . . 25
10.3 Objectives Satisfying PL . . . 25
10.4 General Nonconvex Objectives . . . 27
10.5 Minibatching Trade-offs . . . 27
10.6 Noise Reduction . . . 27
11 Learning-Rate Schedules and Large-Batch Training 28
11.1 Schedules . . . 28
11.2 The Linear Scaling Rule . . . 29
11.3 Warmup . . . 29
11.4 Critical Batch Size . . . 29
12 Adaptive Diagonal Methods 30
12.1 Gradient Clipping . . . 30
12.2 Preconditioned SGD . . . 31
12.3 Adagrad . . . 31
12.4 RMSProp and AdaDelta . . . 31
12.5 Adam . . . 31
12.6 AdamW: Decoupled Weight Decay . . . 33

IV Second-Order Methods and Curvature 34
13 Newton's Method 34
13.1 The Newton Decrement . . . 34
13.2 Local Quadratic Convergence . . . 34
13.3 Affine Invariance . . . 35
13.4 Failure Modes and Damped Newton . . . 36
14 Trust Regions and Damping 36
14.1 Trust Region versus Line Search . . . 36
14.2 Tikhonov Damping and the Moré–Sorensen Characterisation . . . 36
14.3 Levenberg–Marquardt Adaptation . . . 38

14.4 Structural Damping . . . 38
14.5 Relative Damping and Links to First-Order Methods . . . 38
15 Generalized Gauss-Newton 38
15.1 Levenberg–Marquardt Damping . . . 40
16 Fisher Information and Natural Gradient 40
16.1 The Fisher Matrix . . . 40
16.2 Fisher for Neural Networks . . . 41
16.3 Exponential Family Losses . . . 42
16.4 Natural Gradient Descent . . . 42
16.5 Interpretations of Natural Gradient . . . 43
16.6 The Empirical Fisher and Why It Fails . . . 44
16.7 Convergence: The Pre-Asymptotic Advantage . . . 45
16.8 Approximating the Fisher . . . 45
17 Low-Rank Structure 45
17.1 Conditioning and Convergence . . . 46
18 Loss Landscape of Neural Networks 46
18.1 Bulk and Outliers . . . 46
18.2 Sharpness and the Edge of Stability . . . 47
18.3 PL in Over-Parameterised Networks . . . 48
19 Hessian-Free Methods 49
19.1 Conjugate Gradient . . . 49
19.2 Inexact Newton with Conjugate Gradient . . . 50
19.3 Hessian-Vector Products . . . 51
19.4 Subsampled Hessians and Nonconvexity . . . 51
20 Kronecker-Factored Methods 51
20.1 Kronecker Product Toolkit . . . 51
20.2 KFAC . . . 52
20.3 Shampoo . . . 53
20.4 SOAP . . . 53
21 Diagonal Scaling as Curvature 53

V Steepest Descent Under Norms 55
22 Steepest Descent as Norm Choice 55
22.1 The Framework . . . 55
22.2 Induced Operator Norms . . . 56
22.3 Majorization–Minimization as the Design Pattern . . . 56
23 Instances: From Gradient Descent to Muon 57
23.1 Table of Norms and Optimizers . . . 57
23.2 Sign Descent and Adam . . . 58
23.3 Spectral Descent: Shampoo and Muon . . . 58
23.4 Natural Gradient as the Fisher Norm . . . 59
23.5 Muon in Practice . . . 59

Part I.
Foundations

1 PROBLEM SETUP AND OPTIMALITY

We consider the general optimization problem of

$$\min_{\boldsymbol{\theta}\in\mathcal{C}} f(\boldsymbol{\theta}), \qquad \mathcal{C}\subseteq\mathbb{R}^P, \tag{1}$$

where $f:\mathbb{R}^P\to\mathbb{R}$ is the *objective* (or loss, cost), $\boldsymbol{\theta}$ collects the P *parameters*, and the *feasible set* $\mathcal{C}$ is constrained by equality and inequality constraints. Maximising a reward R is minimising $f=-R$, so every problem is a minimisation. For now, we consider the unconstrained problem of $\mathcal{C}=\mathbb{R}^P$ until §4. We assume throughout that f is twice continuously differentiable.

Definition 1 (Minima)**.** A point $\boldsymbol{\theta}^*\in\mathcal{C}$ is a

- *global minimum* if $f(\boldsymbol{\theta}^*)\le f(\boldsymbol{\theta})$ for all $\boldsymbol{\theta}\in\mathcal{C}$;
- *local minimum* if there is $\delta>0$ with $f(\boldsymbol{\theta}^*)\le f(\boldsymbol{\theta})$ for all $\boldsymbol{\theta}\in\mathcal{C}$ with $\|\boldsymbol{\theta}-\boldsymbol{\theta}^*\|<\delta$;
- *strict* (global or local) minimum if the inequality is strict for every $\boldsymbol{\theta}\neq\boldsymbol{\theta}^*$ in the respective set.

We write $f^*:=\min_{\boldsymbol{\theta}\in\mathcal{C}} f(\boldsymbol{\theta})$ for the global minimum value when it exists.

Finding a global minimum of a general non-convex f is computationally intractable [1], so the methods of this document target local minima, or more modestly *stationary points*. Two geometric properties can restore global guarantees: convexity (§3), under which every local minimum is global, and the PL inequality (Definition 15), under which every stationary point is global. An algorithm that reaches a stationary point from any starting point is called *globally convergent*, although this says nothing about reaching a global minimum.

1.1 Gradient and Hessian

Definition 2 (Gradient)**.** The *gradient* of f at $\boldsymbol{\theta}$ — the vector of first partial derivatives:

$$\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \;=\; \begin{bmatrix} \dfrac{\partial f}{\partial\theta_1} \\ \vdots \\ \dfrac{\partial f}{\partial\theta_P} \end{bmatrix} \;\in\; \mathbb{R}^P.$$

We write ∇f for $\nabla_{\boldsymbol{\theta}} f$ when the variable is clear. A direction $\mathbf{d}$ is a *descent direction* at $\boldsymbol{\theta}$ if $\mathbf{d}^\top\nabla f(\boldsymbol{\theta})<0$.

Definition 3 (Hessian)**.** The *Hessian* of f at $\boldsymbol{\theta}$ — the matrix of second partial derivatives:

$$\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}) \;:=\; \nabla^2_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \;=\; \begin{bmatrix} \dfrac{\partial^2 f}{\partial\theta_1^2} & \cdots & \dfrac{\partial^2 f}{\partial\theta_1\partial\theta_P} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial^2 f}{\partial\theta_P\partial\theta_1} & \cdots & \dfrac{\partial^2 f}{\partial\theta_P^2} \end{bmatrix} \;\in\; \mathbb{R}^{P\times P}.$$

Symmetry of $\mathbf{H}_{\boldsymbol{\theta}}$ (Schwarz's theorem) means the eigendecomposition and all the linear-algebra machinery of §2 apply to it.

1.2 Taylor Expansion

Every method in this document starts from a local polynomial model of f around the current iterate: the Taylor expansion truncated at first order (gradient methods, Part II) or at second order (Newton-type methods, Part IV).[1]

1 For $P=1$ the expansion reads $f(x)=f(x^*)+(x-x^*)f'(x^*)+\tfrac{1}{2}(x-x^*)^2f''(x^*)+o(|x-x^*|^2)$; the multivariate version replaces f' by the gradient and f'' by the Hessian, with the quadratic term measuring the curvature of f along $\mathbf{d}$ through the Rayleigh quotient of Definition 5.

Theorem 1 (Second-order Taylor expansion). *For $f \in C^2(\mathbb{R}^P)$ and any $\boldsymbol{\theta}, \mathbf{d} \in \mathbb{R}^P$,*

$$f(\boldsymbol{\theta}+\mathbf{d}) = f(\boldsymbol{\theta}) + \int_0^1 \nabla f(\boldsymbol{\theta}+s\mathbf{d})^\top \mathbf{d}\, ds, \tag{2}$$

$$\nabla f(\boldsymbol{\theta}+\mathbf{d}) = \nabla f(\boldsymbol{\theta}) + \int_0^1 \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}+s\mathbf{d})\, \mathbf{d}\, ds. \tag{3}$$

Substituting (3) (up to s) into (2) and swapping integrals, $\int_0^1 \int_0^s du\, ds = \int_0^1 (1-u)\, du$:

$$f(\boldsymbol{\theta}+\mathbf{d}) = f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^\top \mathbf{d} + \int_0^1 (1-s)\, \mathbf{d}^\top \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}+s\mathbf{d})\, \mathbf{d}\, ds. \tag{4}$$

Freezing $\mathbf{H}_{\boldsymbol{\theta}}$ at $s=0$ ($\int_0^1 (1-s)\, ds = \frac{1}{2}$; error $o(\|\mathbf{d}\|^2)$ by continuity):

$$f(\boldsymbol{\theta}+\mathbf{d}) = f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^\top \mathbf{d} + \tfrac{1}{2}\, \mathbf{d}^\top \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta})\, \mathbf{d} + o(\|\mathbf{d}\|^2). \tag{5}$$

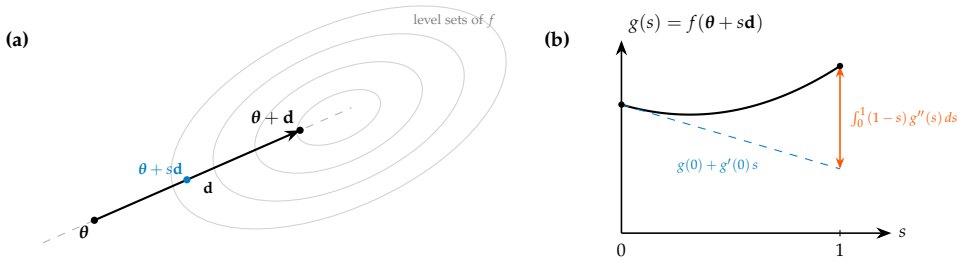


Figure 1: The expansion is a statement along the straight segment $\boldsymbol{\theta}+s\mathbf{d}$, $s \in [0,1]$. **(a)** The integrals in Theorem 1 evaluate ∇f and $\mathbf{H}_{\boldsymbol{\theta}}$ at points along this segment. **(b)** Restricted to the segment, f is the scalar $g(s) = f(\boldsymbol{\theta}+s\mathbf{d})$ with $g'(s) = \nabla f^\top \mathbf{d}$ and $g''(s) = \mathbf{d}^\top \mathbf{H}_{\boldsymbol{\theta}} \mathbf{d}$ evaluated at $\boldsymbol{\theta}+s\mathbf{d}$. The dashed line is the first-order model; the orange gap at $s=1$ is the remainder of (4), a $(1-s)$-weighted average of the curvature along the segment, which the quadratic term of (5) approximates by its value at $s=0$.

1.3 Necessary and Sufficient Conditions

The Taylor expansion characterizes optimality with conditions on the gradient and Hessian at the candidate point.

Definition 4 (Stationary points and feasible directions). $\boldsymbol{\theta}^*$ is *stationary* if $\nabla f(\boldsymbol{\theta}^*) = \mathbf{0}$. $\mathbf{d}$ is a *feasible direction* at $\boldsymbol{\theta} \in \mathcal{C}$ if some feasible curve leaves $\boldsymbol{\theta}$ in that direction: $\boldsymbol{\theta}(t) \in \mathcal{C}$ for all small $t \geq 0$, with $\boldsymbol{\theta}(0) = \boldsymbol{\theta}$ and $\boldsymbol{\theta}'(0) = \mathbf{d}$. In the unconstrained case $\mathcal{C} = \mathbb{R}^P$, every direction is feasible (take the straight line $\boldsymbol{\theta}+t\mathbf{d}$).

Theorem 2 (First-order necessary condition). *If $\boldsymbol{\theta}^*$ is a local minimum of f over $\mathcal{C}$, then*

$$\nabla f(\boldsymbol{\theta}^*)^\top \mathbf{d} \;\geq\; 0 \qquad \textit{for every feasible direction } \mathbf{d}.$$

Unconstrained, every $\mathbf{d}$ is feasible and this forces $\nabla f(\boldsymbol{\theta}^*) = \mathbf{0}$: local minima are stationary points.

Proof. Move along a feasible curve $\boldsymbol{\theta}(t)$ with $\boldsymbol{\theta}'(0) = \mathbf{d}$, so $\boldsymbol{\theta}(t) - \boldsymbol{\theta}^* = t\mathbf{d} + O(t^2)$, and expand with (5):

$$f(\boldsymbol{\theta}(t)) - f(\boldsymbol{\theta}^*) \;=\; t\, \nabla f(\boldsymbol{\theta}^*)^\top \mathbf{d} + O(t^2).$$

If $\nabla f(\boldsymbol{\theta}^*)^\top \mathbf{d} < 0$ (a descent direction, Definition 2) the right-hand side is negative for small t, contradicting local minimality. Unconstrained, $\mathbf{d}$ and $-\mathbf{d}$ are both feasible, so $\nabla f(\boldsymbol{\theta}^*)^\top \mathbf{d} = 0$ for all $\mathbf{d}$, i.e. $\nabla f(\boldsymbol{\theta}^*) = \mathbf{0}$. □

The condition is necessary for minima but not sufficient: maxima and saddle points are stationary too, hence we need the second-order term.

Theorem 3 (Second-order conditions). *Let $\nabla f(\boldsymbol{\theta}^*) = \mathbf{0}$ and write $\mathbf{H}_* := \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}^*)$.*

(i) Necessary: *if $\boldsymbol{\theta}^*$ is a local minimum, then $\mathbf{H}_* \succeq 0$.*
(ii) Sufficient: *if $\mathbf{H}_* \succ 0$, then $\boldsymbol{\theta}^*$ is a strict local minimum.*

Proof. At a stationary point the linear term of (5) vanishes:

$$f(\boldsymbol{\theta}^*+\mathbf{d}) - f(\boldsymbol{\theta}^*) \;=\; \tfrac{1}{2}\, \mathbf{d}^\top \mathbf{H}_*\, \mathbf{d} + o(\|\mathbf{d}\|^2).$$

(i) If $\mathbf{d}^\top \mathbf{H}_* \mathbf{d} < 0$ for some $\mathbf{d}$, then along $t\mathbf{d}$ the right-hand side is $\frac{t^2}{2}\, \mathbf{d}^\top \mathbf{H}_* \mathbf{d} + o(t^2) < 0$ for small t, contradicting local minimality. *(ii)* If $\mathbf{H}_* \succ 0$, the Rayleigh bound (8) gives $\mathbf{d}^\top \mathbf{H}_* \mathbf{d} \geq \lambda_{\min}(\mathbf{H}_*)\, \|\mathbf{d}\|^2$ with $\lambda_{\min} > 0$, which dominates the $o(\|\mathbf{d}\|^2)$ remainder, so the difference is positive for every small $\mathbf{d} \neq \mathbf{0}$. □

A stationary point with semidefinite $\mathbf{H}_* \succeq 0$ but singular may be a minimum, a maximum, or neither. Table 1 classifies stationary points by the eigenvalue test of Definition 6.

$\mathbf{H}_*$ at a stationary point	Eigenvalues	Nature of $\boldsymbol{\theta}^*$
Positive definite $\succ 0$	all $\lambda_i > 0$	strict local minimum
Negative definite $\prec 0$	all $\lambda_i < 0$	strict local maximum
Indefinite	both signs	saddle point
Semidefinite, singular	some $\lambda_i = 0$, rest one sign	inconclusive

Table 1: Classification of a stationary point by its Hessian (Theorem 3, and the same applied to $-f$).

2 LINEAR ALGEBRA TOOLKIT

2.1 Symmetric Eigendecomposition and the Rayleigh Quotient

For real symmetric $\mathbf{A} = \mathbf{A}^\top \in \mathbb{R}^{n\times n}$, the spectral theorem gives the orthogonal eigendecomposition:

$$\mathbf{A} \;=\; \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top, \qquad \mathbf{U}^\top\mathbf{U} = \mathbf{U}\mathbf{U}^\top = \mathbf{I}_n, \qquad \boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1 \geq \cdots \geq \lambda_n), \tag{6}$$

with real eigenvalues λ_i and orthonormal eigenvectors as columns of $\mathbf{U}$.

Definition 5 (Rayleigh quotient)**.** For symmetric $\mathbf{A}$ and $\mathbf{v} \neq \mathbf{0}$:

$$R_{\mathbf{A}}(\mathbf{v}) \;:=\; \frac{\mathbf{v}^\top\mathbf{A}\,\mathbf{v}}{\mathbf{v}^\top\mathbf{v}}.$$

Theorem 4 (Rayleigh–Courant–Fischer extremes)**.** *For symmetric* $\mathbf{A}$ *with eigendecomposition* (6)*:*

$$\lambda_n \;=\; \min_{\mathbf{v}\neq\mathbf{0}} R_{\mathbf{A}}(\mathbf{v}), \qquad \lambda_1 \;=\; \max_{\mathbf{v}\neq\mathbf{0}} R_{\mathbf{A}}(\mathbf{v}), \tag{7}$$

where the minimum is attained at the eigenvector $\mathbf{v} = \mathbf{u}_n$ *and the maximum at* $\mathbf{v} = \mathbf{u}_1$*. Equivalently,*

$$\lambda_n \|\mathbf{v}\|^2 \;\leq\; \mathbf{v}^\top\mathbf{A}\,\mathbf{v} \;\leq\; \lambda_1 \|\mathbf{v}\|^2, \quad \forall \mathbf{v} \neq \mathbf{0}. \tag{8}$$

Proof. Write $\mathbf{w} = \mathbf{U}^\top\mathbf{v}$, so $\|\mathbf{w}\|^2 = \mathbf{w}^\top\mathbf{w} = \mathbf{v}^\top\mathbf{U}\mathbf{U}^\top\mathbf{v} = \mathbf{v}^\top\mathbf{v} = \|\mathbf{v}\|^2$ (orthogonality of $\mathbf{U}$). Then $\mathbf{v}^\top\mathbf{A}\,\mathbf{v} = \mathbf{v}^\top\mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top\mathbf{v} = (\mathbf{U}^\top\mathbf{v})^\top\boldsymbol{\Lambda}(\mathbf{U}^\top\mathbf{v}) = \mathbf{w}^\top\boldsymbol{\Lambda}\,\mathbf{w} = \sum_i \lambda_i w_i^2$. Since $\lambda_n \sum w_i^2 \leq \sum \lambda_i w_i^2 \leq \lambda_1 \sum w_i^2$ and $\sum w_i^2 = \|\mathbf{w}\|^2 = \|\mathbf{v}\|^2$, we get (8). The upper bound is attained with $\mathbf{w} = \mathbf{e}_1$ [2], which gives $\sum_i \lambda_i w_i^2 = \lambda_1$ and $\|\mathbf{w}\|^2 = 1$. Inverting $\mathbf{w} = \mathbf{U}^\top\mathbf{v}$ gives $\mathbf{v} = \mathbf{U}\mathbf{e}_1 = \mathbf{u}_1$, the first column of $\mathbf{U}$. Hence $R_{\mathbf{A}}(\mathbf{u}_1) = \lambda_1$. Symmetrically, $\mathbf{w} = \mathbf{e}_n$ gives $\mathbf{v} = \mathbf{u}_n$ at the lower bound. □

The variant $\lambda_{\min}^+(\mathbf{A}) := \min\{\lambda_i : \lambda_i > 0\}$ — the smallest *strictly positive* eigenvalue — is the relevant quantity when $\mathbf{A}$ is PSD but rank-deficient; restricting (7) to $\mathbf{v} \in \operatorname{Range}(\mathbf{A})$ recovers $\lambda_{\min}^+$.

Lemma 5 (Cyclic eigenvalue property)**.** *For matrices* $\mathbf{A} \in \mathbb{R}^{m\times n}$ *and* $\mathbf{B} \in \mathbb{R}^{n\times m}$*, the products* $\mathbf{AB} \in \mathbb{R}^{m\times m}$ *and* $\mathbf{BA} \in \mathbb{R}^{n\times n}$ *have the same nonzero eigenvalues (with multiplicity).*

Proof. If $\mathbf{ABx} = \lambda\mathbf{x}$ with $\lambda \neq 0$, then $\mathbf{Bx} \neq \mathbf{0}$ and $\mathbf{BA}(\mathbf{Bx}) = \mathbf{B}(\lambda\mathbf{x}) = \lambda(\mathbf{Bx})$, so λ is also an eigenvalue of $\mathbf{BA}$. The argument is symmetric. □

2.2 Positive Semi-Definite Matrices

Definition 6 (PSD partial order)**.** A symmetric matrix $\mathbf{A}$ is *positive semi-definite* ($\mathbf{A} \succeq 0$) if $\mathbf{v}^\top\mathbf{A}\,\mathbf{v} \geq 0$ for all $\mathbf{v}$, equivalently if all eigenvalues are non-negative. It is *positive definite* ($\mathbf{A} \succ 0$) if all eigenvalues are strictly positive. For symmetric $\mathbf{A}, \mathbf{B}$,

$$\mathbf{A} \preceq \mathbf{B} \iff \mathbf{B} - \mathbf{A} \succeq 0 \iff \mathbf{v}^\top\mathbf{A}\,\mathbf{v} \leq \mathbf{v}^\top\mathbf{B}\,\mathbf{v} \;\; \forall \mathbf{v}.$$

The PSD partial order is the cleanest way to express the central function-regularity assumption $\mu\mathbf{I} \preceq \nabla^2_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \preceq L\mathbf{I}$ (§3 below).

Definition 7 (PSD square root)**.** For symmetric $\mathbf{A} \succeq 0$ with eigendecomposition $\mathbf{A} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$, the *principal square root* is

$$\mathbf{A}^{1/2} \;:=\; \mathbf{U}\boldsymbol{\Lambda}^{1/2}\mathbf{U}^\top, \qquad (\boldsymbol{\Lambda}^{1/2})_{ii} = \sqrt{\lambda_i}.$$

It is the unique PSD matrix satisfying $\mathbf{A}^{1/2}\mathbf{A}^{1/2} = \mathbf{A}$.

2 $\mathbf{e}_i$ is the standard basis vector (1 in position i, zeros elsewhere)

The square root underlies the identity

$$\mathbf{v}^\top \mathbf{A}\,\mathbf{v} \;=\; \|\mathbf{A}^{1/2}\mathbf{v}\|^2 \;=:\; \|\mathbf{v}\|_{\mathbf{A}}^2, \tag{9}$$

which defines the $\mathbf{A}$-*norm* (a genuine norm when $\mathbf{A} \succ 0$, a semi-norm when $\mathbf{A}$ is merely PSD).

Definition 8 (Operator norm)**.** For any $\mathbf{A} \in \mathbb{R}^{m\times n}$,

$$\|\mathbf{A}\|_{\mathrm{op}} \;\coloneqq\; \max_{\mathbf{v}\neq\mathbf{0}} \frac{\|\mathbf{A}\mathbf{v}\|}{\|\mathbf{v}\|} \;=\; \sigma_1(\mathbf{A}),$$

the largest singular value, where $\sigma_i(\mathbf{A}) \coloneqq \sqrt{\lambda_i(\mathbf{A}^\top\mathbf{A})}$.

For symmetric $\mathbf{A}$, consider the SVD $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. The eigendecomposition is $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ with real λ_i. Since $\mathbf{A}^\top\mathbf{A} = \mathbf{A}^2 = \mathbf{Q}\mathbf{\Lambda}^2\mathbf{Q}^\top$, the singular values are the absolute eigenvalues: $\sigma_i(\mathbf{A}) = \sqrt{\lambda_i^2(\mathbf{A})} = |\lambda_i(\mathbf{A})|$, so $\|\mathbf{A}\|_{\mathrm{op}} = \max_i |\lambda_i(\mathbf{A})|$.

For PSD $\mathbf{A} \succeq 0$, all $\lambda_i \geq 0$, so $|\lambda_i| = \lambda_i$ and the SVD coincide with the eigendecomposition ($\mathbf{U} = \mathbf{V} = \mathbf{Q}$, $\mathbf{\Sigma} = \mathbf{\Lambda}$); $\|\mathbf{A}\|_{\mathrm{op}} = \lambda_{\max}(\mathbf{A})$.

2.3 Singular Value Decomposition

For non-square or non-symmetric $\mathbf{A} \in \mathbb{R}^{m\times n}$, the analogue of the eigendecomposition is the *singular value decomposition*:

$$\mathbf{A} \;=\; \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top, \qquad \mathbf{U} \in \mathbb{R}^{m\times m},\;\; \mathbf{V} \in \mathbb{R}^{n\times n},\;\; \mathbf{\Sigma} \in \mathbb{R}^{m\times n}, \tag{10}$$

with $\mathbf{U}, \mathbf{V}$ orthogonal and $\mathbf{\Sigma}$ "diagonal" with non-negative entries $\sigma_1 \geq \cdots \geq \sigma_{\min(m,n)} \geq 0$, the *singular values* of $\mathbf{A}$. The *thin SVD* retains only columns of $\mathbf{U}, \mathbf{V}$ corresponding to nonzero singular values:

$$\mathbf{A} \;=\; \mathbf{P}\,\mathbf{D}\,\mathbf{Q}^\top, \qquad \mathbf{P} \in \mathbb{R}^{m\times s},\;\; \mathbf{Q} \in \mathbb{R}^{n\times s},\;\; \mathbf{D} \in \mathbb{R}^{s\times s}, \tag{11}$$

where $s = \mathrm{rank}(\mathbf{A})$, $\mathbf{P}^\top\mathbf{P} = \mathbf{Q}^\top\mathbf{Q} = \mathbf{I}_s$, and $\mathbf{D}$ is invertible diagonal.

2.4 Moore–Penrose Pseudoinverse

Definition 9 (Moore–Penrose pseudoinverse)**.** For any $\mathbf{A} \in \mathbb{R}^{m\times n}$, the *Moore–Penrose pseudoinverse* $\mathbf{A}^+ \in \mathbb{R}^{n\times m}$ is the unique matrix satisfying the four *Penrose conditions*:

$$\begin{aligned} &\text{(P1)}\quad \mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}, && \text{(P2)}\quad \mathbf{A}^+\mathbf{A}\mathbf{A}^+ = \mathbf{A}^+,\\ &\text{(P3)}\quad (\mathbf{A}\mathbf{A}^+)^\top = \mathbf{A}\mathbf{A}^+, && \text{(P4)}\quad (\mathbf{A}^+\mathbf{A})^\top = \mathbf{A}^+\mathbf{A}. \end{aligned}$$

Existence follows from the thin SVD (11): $\mathbf{A}^+ = \mathbf{Q}\mathbf{D}^{-1}\mathbf{P}^\top$ satisfies (P1)–(P4).

Lemma 6 (Key pseudoinverse identities)**.**

1. $\mathbf{A}\mathbf{A}^+$ *is the orthogonal projection onto* $\mathrm{Range}(\mathbf{A})$.
2. $\mathbf{A}^+\mathbf{A}$ *is the orthogonal projection onto* $\mathrm{Range}(\mathbf{A}^\top)$.
3. $(\mathbf{A}^\top\mathbf{A})^+\mathbf{A}^\top = \mathbf{A}^+ = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^+$.
4. *For full row rank* $\mathbf{B}$*:* $\mathbf{B}^\top(\mathbf{B}\mathbf{B}^\top)^{-1}\mathbf{B} = \mathbf{\Pi}_{\mathrm{Range}(\mathbf{B}^\top)}$*, and* $\mathbf{B}(\mathbf{B}^\top\mathbf{B})^+\mathbf{B}^\top = \mathbf{I}_m$.

Lemma 7 (Some pseudoinverse results)**.**

1. *Symmetric* ($\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$) $\Longrightarrow \mathbf{A}^+ = \mathbf{U}\mathbf{\Lambda}^+\mathbf{U}^\top$*, where* $(\mathbf{\Lambda}^+)_{ii} = 1/\lambda_i$ *if* $\lambda_i \neq 0$*, else* 0.
2. *Full row rank* ($\mathrm{rank}(\mathbf{A}) = m \leq n$) $\Longrightarrow \mathbf{A}^+ = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1}$*, and* $\mathbf{A}\mathbf{A}^+ = \mathbf{I}_m$.
3. *Full column rank* ($\mathrm{rank}(\mathbf{A}) = n \leq m$) $\Longrightarrow \mathbf{A}^+ = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top$*, and* $\mathbf{A}^+\mathbf{A} = \mathbf{I}_n$.
4. *Invertible* $\mathbf{A} \Longrightarrow \mathbf{A}^+ = \mathbf{A}^{-1}$.

Lemma 8 (Range and null space of $\mathbf{A}^+$)**.** *For any* $\mathbf{A}$*,*

$$\mathrm{Range}(\mathbf{A}^+) \;=\; \mathrm{Range}(\mathbf{A}^\top), \qquad \mathrm{Null}(\mathbf{A}^+) \;=\; \mathrm{Null}(\mathbf{A}^\top). \tag{12}$$

$\mathbf{A}^+$ *acts as the inverse of* $\mathbf{A}$ *restricted to the map* $\mathrm{Range}(\mathbf{A}^\top) \to \mathrm{Range}(\mathbf{A})$*, and is zero on the orthogonal complement.*

2.5 Range, Null Space, and Projections

Theorem 9 (Fundamental subspace decomposition). *For any* $\mathbf{A} \in \mathbb{R}^{m\times n}$,

$$\mathbb{R}^n \;=\; \mathrm{Range}(\mathbf{A}^\top) \oplus^{\perp} \mathrm{Null}(\mathbf{A}), \qquad \mathbb{R}^m \;=\; \mathrm{Range}(\mathbf{A}) \oplus^{\perp} \mathrm{Null}(\mathbf{A}^\top), \tag{13}$$

where $\oplus^{\perp}$ *denotes orthogonal direct sum.*

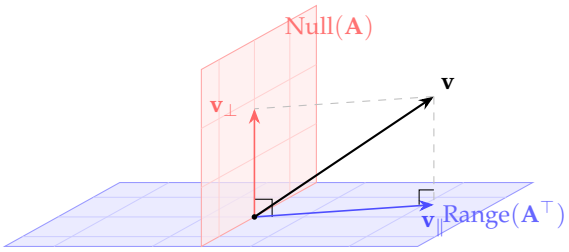


Figure 2: Fundamental subspace decomposition of $\mathbb{R}^n$: any $\mathbf{v}$ splits uniquely into $\mathbf{v}_\parallel \in \mathrm{Range}(\mathbf{A}^\top)$ (blue horizontal plane) and $\mathbf{v}_\perp \in \mathrm{Null}(\mathbf{A})$ (red vertical plane), with $\mathbf{v} = \mathbf{v}_\parallel + \mathbf{v}_\perp$. The two subspaces intersect orthogonally and together span $\mathbb{R}^n$.

Definition 10 (Projection). A matrix $\mathbf{P} \in \mathbb{R}^{n\times n}$ is a *projection* if $\mathbf{P}^2 = \mathbf{P}$. It is an *orthogonal projection* if additionally $\mathbf{P}^\top = \mathbf{P}$ (symmetric), in which case its eigenvalues are 0 or 1 and $\|\mathbf{P}\|_{\mathrm{op}} \le 1$. Otherwise it is an *oblique projection*.

Lemma 10 (Orthogonal projection formula). *For* $\mathbf{M} \in \mathbb{R}^{n\times r}$, *define* $\mathbf{\Pi} := \mathbf{M}(\mathbf{M}^\top\mathbf{M})^{+}\mathbf{M}^\top$. *Then* $\mathbf{\Pi}$ *is the orthogonal projection onto* $\mathrm{Range}(\mathbf{M})$. *That is, for every* $\mathbf{v} \in \mathbb{R}^n$,

$$\mathbf{\Pi}\mathbf{v} \;=\; \arg\min_{\mathbf{u}\,\in\,\mathrm{Range}(\mathbf{M})} \|\mathbf{u}-\mathbf{v}\|^2. \tag{14}$$

Equivalently, $\mathbf{\Pi} = \mathbf{M}\mathbf{M}^{+}$ *(Lemma 6). When* $\mathbf{M}$ *has full column rank, then* $\mathbf{M}^\top\mathbf{M}$ *is invertible so* $(\mathbf{M}^\top\mathbf{M})^{+} = (\mathbf{M}^\top\mathbf{M})^{-1}$.

Proof. Write $\mathbf{u} = \mathbf{M}\mathbf{h}$ and set the gradient to zero:

$$\nabla_{\mathbf{h}} \|\mathbf{M}\mathbf{h}-\mathbf{v}\|^2 = 2\mathbf{M}^\top(\mathbf{M}\mathbf{h}-\mathbf{v}) = \mathbf{0} \quad\Longrightarrow\quad \mathbf{h} = (\mathbf{M}^\top\mathbf{M})^{+}\mathbf{M}^\top\mathbf{v},$$

so the minimiser is $\mathbf{\Pi}\mathbf{v} = \mathbf{M}\mathbf{h} = \mathbf{M}(\mathbf{M}^\top\mathbf{M})^{+}\mathbf{M}^\top\mathbf{v}$. Symmetry ($\mathbf{\Pi}^\top = \mathbf{\Pi}$) and idempotency ($\mathbf{\Pi}^2 = \mathbf{\Pi}$) follow from the pseudoinverse identities $(\mathbf{M}\mathbf{M}^{+})^\top = \mathbf{M}\mathbf{M}^{+}$ and $\mathbf{M}\mathbf{M}^{+}\mathbf{M} = \mathbf{M}$ (Definition 9). □

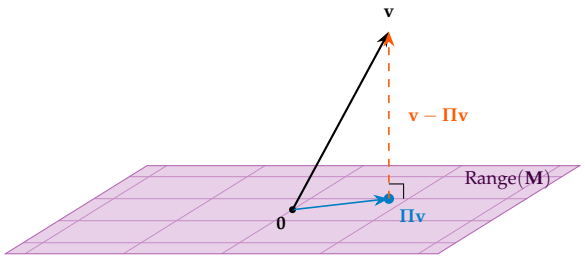


Figure 3: Orthogonal projection: $\mathbf{\Pi}\mathbf{v}$ is the closest point in $\mathrm{Range}(\mathbf{M})$ to $\mathbf{v}$. The residual $\mathbf{v} - \mathbf{\Pi}\mathbf{v}$ is perpendicular to the subspace (right-angle marker).

Oblique projections appear when one measures distance in a non-Euclidean metric induced by some $\mathbf{B} \succ 0$.

Lemma 11 ($\mathbf{B}$-orthogonal projection). *Let* $\mathbf{M} \in \mathbb{R}^{n\times r}$ *have full column rank and* $\mathbf{B} \in \mathbb{R}^{n\times n}$ *with* $\mathbf{B} \succ 0$. *Define*

$$\mathbf{P} \;:=\; \mathbf{M}(\mathbf{M}^\top\mathbf{B}\mathbf{M})^{-1}\mathbf{M}^\top\mathbf{B}. \tag{15}$$

Then $\mathbf{P}$ *is a projection (*$\mathbf{P}^2 = \mathbf{P}$*) with* $\mathrm{Range}(\mathbf{P}) = \mathrm{Range}(\mathbf{M})$, *and for any* $\mathbf{v} \in \mathbb{R}^n$:

$$\mathbf{P}\mathbf{v} \;=\; \arg\min_{\mathbf{u}\in\mathrm{Range}(\mathbf{M})} \|\mathbf{u}-\mathbf{v}\|_{\mathbf{B}}^2, \tag{16}$$

with $\mathbf{B}$*-orthogonal residual:* $\mathbf{M}^\top\mathbf{B}\,(\mathbf{P}\mathbf{v}-\mathbf{v}) = \mathbf{0}$. *In general* $\mathbf{P} \neq \mathbf{P}^\top$, *but* $\mathbf{P}$ *satisfies the* $\mathbf{B}$-self-adjoint *relation* $\mathbf{P}^\top\mathbf{B} = \mathbf{B}\,\mathbf{P}$ *(equivalent to ordinary symmetry of* $\mathbf{B}^{1/2}\mathbf{P}\mathbf{B}^{-1/2}$*).*

Proof. Idempotency: $\mathbf{P}^2 = \mathbf{M}(\mathbf{M}^\top\mathbf{B}\mathbf{M})^{-1}\mathbf{M}^\top\mathbf{B}\mathbf{M}(\mathbf{M}^\top\mathbf{B}\mathbf{M})^{-1}\mathbf{M}^\top\mathbf{B} = \mathbf{P}$, the inner factor cancelling. For (16), set $\mathbf{u} = \mathbf{M}\mathbf{h}$; then

$$\nabla_{\mathbf{h}}\|\mathbf{M}\mathbf{h} - \mathbf{v}\|_{\mathbf{B}}^2 = 2\mathbf{M}^\top\mathbf{B}(\mathbf{M}\mathbf{h} - \mathbf{v}) = \mathbf{0} \quad \Longleftrightarrow \quad \mathbf{h} = (\mathbf{M}^\top\mathbf{B}\mathbf{M})^{-1}\mathbf{M}^\top\mathbf{B}\,\mathbf{v},$$

giving $\mathbf{u} = \mathbf{P}\mathbf{v}$. Residual orthogonality $\mathbf{M}^\top\mathbf{B}(\mathbf{P}\mathbf{v} - \mathbf{v}) = \mathbf{M}^\top\mathbf{B}\mathbf{M}\mathbf{h} - \mathbf{M}^\top\mathbf{B}\mathbf{v} = \mathbf{0}$ by definition of $\mathbf{h}$. $\mathbf{B}$-self-adjointness: $\mathbf{P}^\top\mathbf{B} = \mathbf{B}\mathbf{M}(\mathbf{M}^\top\mathbf{B}\mathbf{M})^{-1}\mathbf{M}^\top\mathbf{B} = \mathbf{B}\mathbf{P}$ by inspection. □

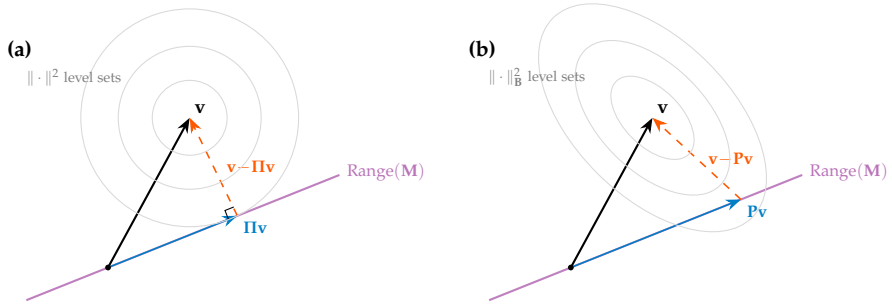


Figure 4: Orthogonal vs. $\mathbf{B}$-orthogonal *(oblique)* projection of $\mathbf{v}$ onto Range$(\mathbf{M})$. **(a)** Euclidean projection $\mathbf{\Pi v}$: minimises $\|\cdot\|^2$ distance (circular level sets); the residual is perpendicular to Range$(\mathbf{M})$. **(b)** $\mathbf{B}$-orthogonal projection $\mathbf{P}\mathbf{v}$ (15): minimises $\|\cdot\|_{\mathbf{B}}^2$ (elliptical level sets); the residual is $\mathbf{B}$-orthogonal but *not* Euclidean-perpendicular, so the projection point shifts along the subspace.

Lemma 12 (Trace of a projection)**.** *For any projection* $\mathbf{P}$ *(orthogonal or oblique),* $\operatorname{tr}(\mathbf{P}) = \operatorname{rank}(\mathbf{P})$.

Proof. Eigenvalues of any projection are 0 or 1 (since $\lambda^2 = \lambda$ from $\mathbf{P}^2 = \mathbf{P}$), and trace equals the sum of eigenvalues. The number of 1's is $\dim[\text{Range}(\mathbf{P})] = \operatorname{rank}(\mathbf{P})$. □

3 FUNCTION REGULARITY

We now collect the function-class assumptions used throughout the convergence analyses, grouped by the structural property they capture.

3.1 Convexity

Definition 11 (Convex set)**.** A set $\mathcal{C} \subseteq \mathbb{R}^P$ is *convex* if $t\,\boldsymbol{\theta} + (1-t)\,\boldsymbol{\theta}' \in \mathcal{C}$ for all $\boldsymbol{\theta}, \boldsymbol{\theta}' \in \mathcal{C}$ and $t \in [0,1]$.

Definition 12 (Convexity)**.** A function f is *convex* if for all $\boldsymbol{\theta}, \boldsymbol{\theta}'$ and $t \in [0,1]$:

$$f\big(t\,\boldsymbol{\theta} + (1-t)\,\boldsymbol{\theta}'\big) \;\le\; t\,f(\boldsymbol{\theta}) + (1-t)\,f(\boldsymbol{\theta}').$$

The Jensen form above requires no differentiability. When $f \in C^1$ or C^2,[3] several forms are equivalent:

Proposition 13 (Equivalent characterisations of convexity)**.** *For* $f \in C^1(\mathbb{R}^P)$*, the following are equivalent:*

(C1) Jensen: $f(t\boldsymbol{\theta} + (1-t)\boldsymbol{\theta}') \le t\,f(\boldsymbol{\theta}) + (1-t)\,f(\boldsymbol{\theta}')$ *for all* $t \in [0,1]$.
(C2) First-order: $f(\boldsymbol{\theta}') \ge f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^\top(\boldsymbol{\theta}' - \boldsymbol{\theta})$ *for all* $\boldsymbol{\theta}, \boldsymbol{\theta}'$.
(C3) Monotone gradient:[4] $\langle \nabla f(\boldsymbol{\theta}) - \nabla f(\boldsymbol{\theta}'),\, \boldsymbol{\theta} - \boldsymbol{\theta}' \rangle \ge 0$ *for all* $\boldsymbol{\theta}, \boldsymbol{\theta}'$.

For $f \in C^2$*, all three are further equivalent to:*

(C4) PSD Hessian: $\nabla^2_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \succeq 0$ *for all* $\boldsymbol{\theta}$.

Each implication comes from restricting f to a line, $g(t) = f(\boldsymbol{\theta} + t(\boldsymbol{\theta}' - \boldsymbol{\theta}))$, and using the Taylor forms of Theorem 1 [2, §2.1]. The one used most below is (C2): a convex function lies above every tangent plane.

3 C^k denotes the class of functions whose first k derivatives exist and are continuous. C^1: gradient ∇f exists and is continuous. C^2: Hessian $\nabla^2 f$ also exists and is continuous.

4 $\langle \mathbf{a}, \mathbf{b} \rangle := \mathbf{a}^\top\mathbf{b} = \sum_i a_i b_i$ denotes the standard Euclidean inner product.

3.2 Lipschitz Smoothness and the Descent Lemma

Definition 13 (L-smoothness)**.** A function f is *L-smooth* (has L-Lipschitz gradient) if

$$\|\nabla f(\boldsymbol{\theta}) - \nabla f(\boldsymbol{\theta}')\| \;\le\; L\,\|\boldsymbol{\theta} - \boldsymbol{\theta}'\| \qquad \forall\, \boldsymbol{\theta}, \boldsymbol{\theta}'.$$

Proposition 14 (Characterisations of L-smoothness)**.** *For $f \in C^2(\mathbb{R}^P)$, the first two conditions are equivalent, and either implies the third:*

(L1) Lipschitz gradient: $\|\nabla f(\boldsymbol{\theta}) - \nabla f(\boldsymbol{\theta}')\| \le L\,\|\boldsymbol{\theta} - \boldsymbol{\theta}'\|$ *for all* $\boldsymbol{\theta}, \boldsymbol{\theta}'$.
(L2) Hessian bound: $-L\,\mathbf{I} \preceq \nabla^2_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \preceq L\,\mathbf{I}$ *for all* $\boldsymbol{\theta}$, *equivalently* $\|\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta})\|_{\mathrm{op}} \le L$ *everywhere.*
(L3) Quadratic upper bound (descent lemma):

$$f(\boldsymbol{\theta}') \le f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^\top(\boldsymbol{\theta}' - \boldsymbol{\theta}) + \tfrac{L}{2}\|\boldsymbol{\theta}' - \boldsymbol{\theta}\|^2 \qquad \forall\, \boldsymbol{\theta}, \boldsymbol{\theta}'. \tag{17}$$

The converse (L3) ⇒ (L1) holds when f is also convex [2, Thm. 2.1.5], but not in general: any concave f satisfies (L3) with $L = 0$.

Proof. Write $\mathbf{d} = \boldsymbol{\theta}' - \boldsymbol{\theta}$ and $\mathbf{H} = \mathbf{H}_{\boldsymbol{\theta}}$.

(L2) ⇒ (L1). By (3), $\nabla f(\boldsymbol{\theta}') - \nabla f(\boldsymbol{\theta}) = \int_0^1 \mathbf{H}(\boldsymbol{\theta} + s\mathbf{d})\,\mathbf{d}\,ds$, and $\|\mathbf{H}\|_{\mathrm{op}} \le L$ bounds the norm of the integrand by $L\|\mathbf{d}\|$.

(L1) ⇒ (L2). Take $\mathbf{d} = t\mathbf{v}$ with $\|\mathbf{v}\| = 1$; then (3) gives $\nabla f(\boldsymbol{\theta} + t\mathbf{v}) - \nabla f(\boldsymbol{\theta}) = t\,\mathbf{H}(\boldsymbol{\theta})\mathbf{v} + o(t)$, so (L1) forces $\|\mathbf{H}(\boldsymbol{\theta})\mathbf{v}\| \le L$ for every unit $\mathbf{v}$.

(L1) ⇒ (L3). By (2), subtracting $\nabla f(\boldsymbol{\theta})^\top\mathbf{d}$ from both sides:

$$\begin{aligned} f(\boldsymbol{\theta}') - f(\boldsymbol{\theta}) - \nabla f(\boldsymbol{\theta})^\top\mathbf{d} &= \int_0^1 \big(\nabla f(\boldsymbol{\theta} + s\mathbf{d}) - \nabla f(\boldsymbol{\theta})\big)^\top \mathbf{d}\,ds \\ &\le \int_0^1 \|\nabla f(\boldsymbol{\theta} + s\mathbf{d}) - \nabla f(\boldsymbol{\theta})\|\,\|\mathbf{d}\|\,ds \quad \text{(Cauchy–Schwarz)} \\ &\le \int_0^1 L\,s\,\|\mathbf{d}\|^2\,ds \;=\; \tfrac{L}{2}\,\|\mathbf{d}\|^2. \qquad \text{((L1) with } \|s\mathbf{d}\| = s\|\mathbf{d}\|) \end{aligned}$$

□

3.3 Strong Convexity

Definition 14 (μ-strong convexity)**.** A function f is *μ-strongly convex* (with $\mu > 0$) if

$$f(\boldsymbol{\theta}') \;\ge\; f(\boldsymbol{\theta}) + \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta})^\top(\boldsymbol{\theta}' - \boldsymbol{\theta}) + \frac{\mu}{2}\|\boldsymbol{\theta}' - \boldsymbol{\theta}\|^2 \qquad \forall\, \boldsymbol{\theta}, \boldsymbol{\theta}'.$$

Proposition 15 (Equivalent characterisations of μ-strong convexity)**.** *For $f \in C^1(\mathbb{R}^P)$ and $\mu > 0$, the following are equivalent:*

(SC1) Gradient form: $f(\boldsymbol{\theta}') \ge f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^\top(\boldsymbol{\theta}' - \boldsymbol{\theta}) + \frac{\mu}{2}\|\boldsymbol{\theta}' - \boldsymbol{\theta}\|^2$.
(SC2) Strong Jensen: $f(t\boldsymbol{\theta} + (1-t)\boldsymbol{\theta}') \le t\,f(\boldsymbol{\theta}) + (1-t)\,f(\boldsymbol{\theta}') - \frac{\mu}{2}\,t(1-t)\|\boldsymbol{\theta} - \boldsymbol{\theta}'\|^2$.
(SC3) Strong monotonicity: $\langle \nabla f(\boldsymbol{\theta}) - \nabla f(\boldsymbol{\theta}'), \boldsymbol{\theta} - \boldsymbol{\theta}'\rangle \ge \mu\|\boldsymbol{\theta} - \boldsymbol{\theta}'\|^2$.
(SC4) Shift to convex: $\boldsymbol{\theta} \mapsto f(\boldsymbol{\theta}) - \frac{\mu}{2}\|\boldsymbol{\theta}\|^2$ *is convex.*

For $f \in C^2$, all four are further equivalent to:

(SC5) Hessian bound: $\nabla^2_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \succeq \mu\,\mathbf{I}$ *for all* $\boldsymbol{\theta}$.

Combined with L-smoothness, (SC5) gives $\mu\mathbf{I} \preceq \nabla^2_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \preceq L\mathbf{I}$ everywhere.

All five follow from Proposition 13 applied to the shifted function $f - \frac{\mu}{2}\|\boldsymbol{\theta}\|^2$ of (SC4), whose gradient and Hessian are $\nabla f - \mu\boldsymbol{\theta}$ and $\nabla^2 f - \mu\mathbf{I}$.

Corollary 16 (Consequences of μ-strong convexity)**.** *If f is μ-strongly convex, the minimiser $\boldsymbol{\theta}^*$ is unique, and for all $\boldsymbol{\theta}$:*

$$f(\boldsymbol{\theta}) - f^* \;\ge\; \tfrac{\mu}{2}\,\|\boldsymbol{\theta} - \boldsymbol{\theta}^*\|^2 \qquad \textit{(quadratic growth)}, \tag{18}$$

$$\|\nabla f(\boldsymbol{\theta})\|^2 \;\ge\; 2\mu\,(f(\boldsymbol{\theta}) - f^*) \qquad \textit{(PL inequality Def 15, same constant)}. \tag{19}$$

The converse of (19) fails: strong convexity implies PL, but PL does not imply convexity.[5]

5 Karimi et al. [3] give the counterexample $f(\theta) = \theta^2 + 3\sin^2(\theta)$, which satisfies PL with $\mu = 1/32$ but has non-positive curvature in places.

Proof. Uniqueness. Any two minimisers $\boldsymbol{\theta}_1^*, \boldsymbol{\theta}_2^*$ would each have f-value f^* but $f(\boldsymbol{\theta}_1^*) \geq f(\boldsymbol{\theta}_2^*) + \frac{\mu}{2}\|\boldsymbol{\theta}_1^* - \boldsymbol{\theta}_2^*\|^2$ forces $\boldsymbol{\theta}_1^* = \boldsymbol{\theta}_2^*$.

Quadratic growth. Apply (SC1) at $(\boldsymbol{\theta}^*, \boldsymbol{\theta})$ with $\nabla f(\boldsymbol{\theta}^*) = \mathbf{0}$: $f(\boldsymbol{\theta}) \geq f^* + \frac{\mu}{2}\|\boldsymbol{\theta} - \boldsymbol{\theta}^*\|^2$.

PL. The right-hand side of (SC1) is a quadratic in $\boldsymbol{\theta}'$; setting its gradient to zero,

$$\nabla f(\boldsymbol{\theta}) + \mu(\boldsymbol{\theta}' - \boldsymbol{\theta}) = \mathbf{0} \quad \Longrightarrow \quad \boldsymbol{\theta}'_{\min} = \boldsymbol{\theta} - \tfrac{1}{\mu}\nabla f(\boldsymbol{\theta}),$$

so the right-hand side is bounded below by its value at $\boldsymbol{\theta}'_{\min}$, for every $\boldsymbol{\theta}'$:

$$\begin{aligned} f(\boldsymbol{\theta}') \;&\geq\; f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^\top \underbrace{(\boldsymbol{\theta}'_{\min} - \boldsymbol{\theta})}_{-\frac{1}{\mu}\nabla f} + \frac{\mu}{2}\underbrace{\|\boldsymbol{\theta}'_{\min} - \boldsymbol{\theta}\|^2}_{\frac{1}{\mu^2}\|\nabla f\|^2} \\ &=\; f(\boldsymbol{\theta}) - \frac{1}{\mu}\|\nabla f(\boldsymbol{\theta})\|^2 + \frac{1}{2\mu}\|\nabla f(\boldsymbol{\theta})\|^2 \;=\; f(\boldsymbol{\theta}) - \frac{\|\nabla f(\boldsymbol{\theta})\|^2}{2\mu}. \end{aligned}$$

Taking $\boldsymbol{\theta}' = \boldsymbol{\theta}^*$ gives $f^* \geq f(\boldsymbol{\theta}) - \|\nabla f(\boldsymbol{\theta})\|^2/(2\mu)$, which is (19). □

3.4 The Polyak–Łojasiewicz Inequality

The PL inequality is the central regularity assumption under which gradient methods converge linearly (without convexity). Strong convexity constrains the curvature everywhere; PL only constrains the slope: wherever f is above its minimum, the gradient cannot be small. This is plausible for over-parameterised networks, where strong convexity is not.

Definition 15 (Polyak–Łojasiewicz inequality)**.** A function f satisfies the *PL inequality* with constant $\mu > 0$ if for all $\boldsymbol{\theta}$:

$$f(\boldsymbol{\theta}) - f^* \;\leq\; \frac{\|\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta})\|^2}{2\mu}, \qquad \text{equivalently} \qquad \|\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta})\|^2 \;\geq\; 2\mu\,(f(\boldsymbol{\theta}) - f^*). \tag{20}$$

The defining feature of PL — and the reason it suffices for global-convergence analysis without convexity — is that every stationary point is automatically a *global* minimum. Saddles and local maxima are excluded.

Lemma 17 (PL implies invexity: every stationary point is global)**.** *If f satisfies PL with constant $\mu > 0$ and $\nabla f(\boldsymbol{\theta}) = \mathbf{0}$, then $f(\boldsymbol{\theta}) = f^*$.*

Proof. From (20), $f(\boldsymbol{\theta}) - f^* \leq \|\nabla f(\boldsymbol{\theta})\|^2/(2\mu) = 0$, and $f(\boldsymbol{\theta}) \geq f^*$ always. □

PL AND THE HESSIAN. PL is not a bound on the Hessian [6]. At a minimiser $\boldsymbol{\theta}^*$, however, it does constrain the eigenvalues λ_i of $\mathbf{H}_* = \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}^*)$. Step along an eigenvector, $\boldsymbol{\theta} = \boldsymbol{\theta}^* + t\,\mathbf{u}_i$, and expand to second order with $\nabla f(\boldsymbol{\theta}^*) = \mathbf{0}$:

$$\begin{aligned} f(\boldsymbol{\theta}) - f^* \approx \tfrac{1}{2}\lambda_i t^2, \qquad \|\nabla f(\boldsymbol{\theta})\|^2 \approx \|\mathbf{H}_*\, t\,\mathbf{u}_i\|^2 = \lambda_i^2 t^2 \quad \text{(Taylor (5))} \\ \text{PL} \;\Longrightarrow\; \lambda_i^2 t^2 \geq \mu\lambda_i t^2 \;\Longrightarrow\; \lambda_i = 0 \;\text{ or }\; \lambda_i \geq \mu. \end{aligned}$$

PL therefore allows exactly flat directions, such as those along a valley of minimisers, but forbids small nonzero curvature $0 < \lambda_i < \mu$, which would let the gradient vanish faster than the gap. Strong convexity forbids the flat directions as well. This is the last row of Figure 5, and it is why PL suits neural networks: the generalised Gauss–Newton matrix is severely rank-deficient (§17), so the curvature at a minimum has many zero eigenvalues.

A CHAIN OF CONDITIONS. Karimi et al. [3] show that for L-smooth f, the following chain holds (with strict implications in general but *all equivalent* when f is convex):

$$\text{strong conv.} \;\Rightarrow\; \text{essential SC} \;\Rightarrow\; \text{weak SC} \;\Rightarrow\; \text{RSI} \;\Rightarrow\; \text{EB} \;\equiv\; \text{PL} \;\Rightarrow\; \text{QG}.$$

With $\boldsymbol{\theta}_p^*$ the minimiser nearest to $\boldsymbol{\theta}$, the error bound (EB) says the gradient grows with the distance to the minimisers, $\|\nabla f(\boldsymbol{\theta})\| \geq \mu\,\|\boldsymbol{\theta} - \boldsymbol{\theta}_p^*\|$, and quadratic growth (QG) says the function does, $f(\boldsymbol{\theta}) - f^* \geq \frac{\mu}{2}\|\boldsymbol{\theta} - \boldsymbol{\theta}_p^*\|^2$, the lower bound (18) without convexity. The other conditions are defined in [3]. PL is the weakest condition in this chain under which gradient descent still converges linearly to a global minimum. [7]

6 $f(\theta) = \theta^2 + 3\sin^2\theta$ satisfies PL but has negative curvature in places [3]

7 QG alone constrains f but not its gradient, so it is not enough

3.5 Condition Numbers

Definition 16 (Condition number)**.** The *condition number* of f at $\boldsymbol{\theta}$ is

$$\kappa(\boldsymbol{\theta}) \;:=\; \frac{\lambda_{\max}(\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}))}{\lambda_{\min}(\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}))}.$$

For L-smooth and μ-strongly convex f, the Hessian bound $\mu\mathbf{I} \preceq \mathbf{H}_{\boldsymbol{\theta}} \preceq L\mathbf{I}$ ((SC5), (L2)) gives $\kappa(\boldsymbol{\theta}) \le L/\mu$. Large κ means curvature varies enormously across directions, causing gradient descent to zig-zag along the dominant eigendirection of $\mathbf{H}_{\boldsymbol{\theta}}$.

3.6 Summary

Figure 5 summarises how each regularity condition constrains the Hessian eigenvalues.

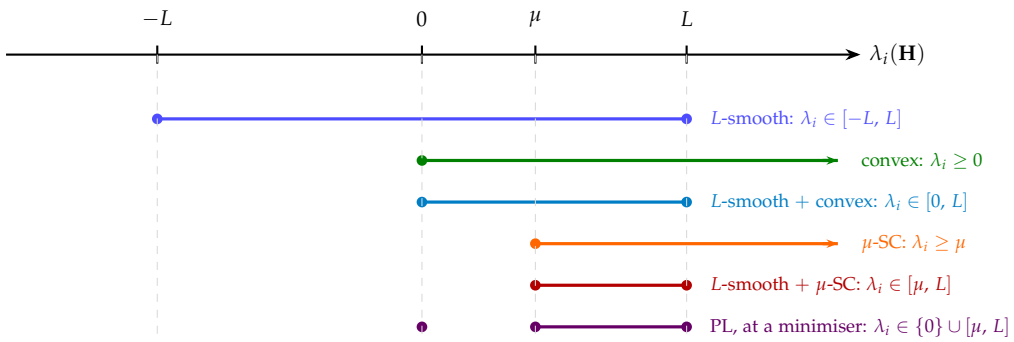


Figure 5: Eigenvalue ranges of the Hessian $\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta})$ under each regularity condition (Propositions 14, 13, 15). The first five hold at every $\boldsymbol{\theta}$. PL constrains the eigenvalues only at a minimiser (§3.4), where it allows exact zeros but no curvature in $(0, \mu)$.

4 LAGRANGIAN AND KKT CONDITIONS

We now allow constraints in (1):[8]

$$\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \quad \text{s.t.} \quad h_i(\boldsymbol{\theta}) = 0,\; i = 1, \dots, m, \qquad g_j(\boldsymbol{\theta}) \le 0,\; j = 1, \dots, q,$$

so the feasible set is $\mathcal{C} = \{\boldsymbol{\theta} : \mathbf{h}(\boldsymbol{\theta}) = \mathbf{0},\; \mathbf{g}(\boldsymbol{\theta}) \le \mathbf{0}\}$. Deep learning meets constraints in three places, and each needs only the conditions of this section: the trust-region radius, whose multiplier becomes the damping parameter of §14; the KL-constrained policy step of TRPO (§9 of the RL notes); and projected steps onto a constraint set. The first-order condition of Theorem 2, $\nabla f(\boldsymbol{\theta}^*)^\top \mathbf{d} \ge 0$ for every feasible $\mathbf{d}$, still holds; the work is to describe the feasible directions.[9]

4.1 Equality Constraints and Lagrange Multipliers

With equality constraints only, the feasible set is the surface $\{\boldsymbol{\theta} : \mathbf{h}(\boldsymbol{\theta}) = \mathbf{0}\}$. A direction $\mathbf{d}$ at $\boldsymbol{\theta}^*$ on the surface is feasible if it keeps every h_i at zero to first order,

$$h_i(\boldsymbol{\theta}^* + t\mathbf{d}) \;=\; \underbrace{h_i(\boldsymbol{\theta}^*)}_{=0} + t\,\nabla h_i(\boldsymbol{\theta}^*)^\top \mathbf{d} + O(t^2),$$

so the feasible directions form the *tangent space*

$$\mathcal{T}(\boldsymbol{\theta}^*) \;:=\; \{\mathbf{d} : \nabla h_i(\boldsymbol{\theta}^*)^\top \mathbf{d} = 0,\; i = 1, \dots, m\},$$

the orthogonal complement of $\operatorname{span}\{\nabla h_1(\boldsymbol{\theta}^*), \dots, \nabla h_m(\boldsymbol{\theta}^*)\}$. It is two-sided: $\mathbf{d} \in \mathcal{T}$ implies $-\mathbf{d} \in \mathcal{T}$.

Theorem 18 (Lagrange condition)**.** *If $\boldsymbol{\theta}^*$ is a local minimum of f on $\{\mathbf{h} = \mathbf{0}\}$ and the $\nabla h_i(\boldsymbol{\theta}^*)$ are linearly independent, then there exist $\lambda_1, \dots, \lambda_m \in \mathbb{R}$, unrestricted in sign, with*

$$\nabla f(\boldsymbol{\theta}^*) + \sum_{i=1}^{m} \lambda_i \nabla h_i(\boldsymbol{\theta}^*) \;=\; \mathbf{0}.$$

8 This section condenses the constrained-optimization chapter of the 3M1 Markdown notes [4]

9 In this section λ_i and μ_j denote Lagrange multipliers, following standard usage. Elsewhere in the document λ_i are eigenvalues and μ is the strong-convexity or PL constant.

Proof. Theorem 2 gives $\nabla f(\boldsymbol{\theta}^*)^\top \mathbf{d} \geq 0$ for all $\mathbf{d} \in \mathcal{T}$; applied to $-\mathbf{d}$ as well, $\nabla f(\boldsymbol{\theta}^*)^\top \mathbf{d} = 0$. Hence $\nabla f(\boldsymbol{\theta}^*) \in \mathcal{T}^\perp = \operatorname{span}\{\nabla h_i(\boldsymbol{\theta}^*)\}$, which is the claim with $\nabla f(\boldsymbol{\theta}^*) = -\sum_i \lambda_i \nabla h_i(\boldsymbol{\theta}^*)$. Two-sidedness of $\mathcal{T}$ is what leaves each λ_i free in sign. □

Geometrically, $\boldsymbol{\theta}^*$ is where a level set of f touches the constraint surface: at any other feasible point, $-\nabla f$ has a component along $\mathcal{T}$, and moving along the surface in that direction still lowers f (Figure 6).

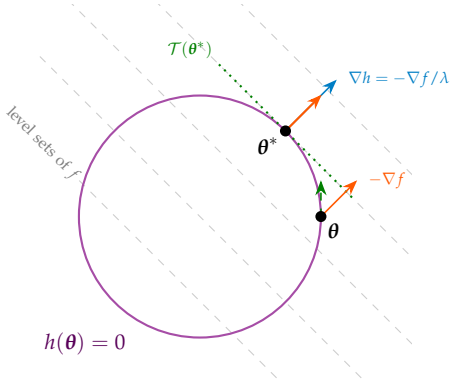


Figure 6: One equality constraint in the plane: $h(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|^2 - 1$ (circle) and $f(\boldsymbol{\theta}) = -\theta_1 - \theta_2$ (dashed level sets). At $\boldsymbol{\theta}^* = (1,1)/\sqrt{2}$ a level set is tangent to the circle, so $-\nabla f$ (orange) is parallel to ∇h (blue) and orthogonal to the tangent space $\mathcal{T}(\boldsymbol{\theta}^*)$ (dotted), with $\lambda = 1/\sqrt{2}$. At the non-optimal point $\boldsymbol{\theta} = (1,0)$, $-\nabla f$ has a component along the circle (green, dashed), a feasible descent direction.

Definition 17 (Lagrangian)**.**

$$\Lambda(\boldsymbol{\theta}, \boldsymbol{\lambda}) \;:=\; f(\boldsymbol{\theta}) + \sum_{i=1}^{m} \lambda_i \, h_i(\boldsymbol{\theta}) \;=\; f(\boldsymbol{\theta}) + \boldsymbol{\lambda}^\top \mathbf{h}(\boldsymbol{\theta}).$$

On the surface $\mathbf{h} = \mathbf{0}$ the Lagrangian equals f; the multiplier terms extend f off the surface so that its stationary points are the constrained optima. Treating $\boldsymbol{\theta}$ and $\boldsymbol{\lambda}$ as independent variables,

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} \Lambda &= \nabla f(\boldsymbol{\theta}) + \sum_{i=1}^{m} \lambda_i \nabla h_i(\boldsymbol{\theta}) = \mathbf{0} && (P \text{ equations: Theorem 18}) \\ \nabla_{\boldsymbol{\lambda}} \Lambda &= \mathbf{h}(\boldsymbol{\theta}) = \mathbf{0} && (m \text{ equations: feasibility}) \end{aligned}$$

gives $P + m$ equations in the $P + m$ unknowns $(\boldsymbol{\theta}, \boldsymbol{\lambda})$. The constrained minimum is a *stationary point* of Λ, not a minimum of it: at any infeasible $\boldsymbol{\theta}$, $\Lambda \to -\infty$ as $\lambda_i \to \pm\infty$, so Λ is a saddle in $(\boldsymbol{\theta}, \boldsymbol{\lambda})$.

4.2 Inequality Constraints and KKT

An inequality constraint $g_j \leq 0$ is *active* at $\boldsymbol{\theta}^*$ if $g_j(\boldsymbol{\theta}^*) = 0$ and *inactive* if $g_j(\boldsymbol{\theta}^*) < 0$. An inactive constraint plays no role locally, so its multiplier is zero. An active one admits feasible directions on one side only, $\nabla g_j(\boldsymbol{\theta}^*)^\top \mathbf{d} \leq 0$, so its multiplier is signed: $\nabla f(\boldsymbol{\theta}^*) = -\mu_j \nabla g_j(\boldsymbol{\theta}^*)$ with $\mu_j \geq 0$, meaning f can only decrease by leaving $\mathcal{C}$. Both cases read $\mu_j \, g_j(\boldsymbol{\theta}^*) = 0$ (Figure 7).

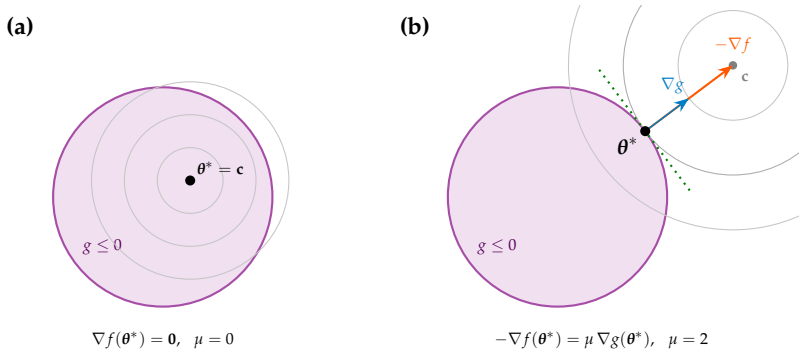


Figure 7: One inequality constraint $g(\boldsymbol{\theta}) = \frac{1}{2}(\|\boldsymbol{\theta}\|^2 - 1) \le 0$ (shaded disk) with $f(\boldsymbol{\theta}) = \|\boldsymbol{\theta} - \mathbf{c}\|^2$ (grey level sets). **(a)** Inactive: the unconstrained minimiser $\mathbf{c}$ is feasible, so $\boldsymbol{\theta}^* = \mathbf{c}$, $\nabla f(\boldsymbol{\theta}^*) = \mathbf{0}$ and $\mu = 0$. **(b)** Active: $\mathbf{c} = (1.6, 1.2)$ lies outside, the level set of radius 1 touches the disk at $\boldsymbol{\theta}^* = (0.8, 0.6)$, and $-\nabla f(\boldsymbol{\theta}^*) = 2(\mathbf{c} - \boldsymbol{\theta}^*)$ points out of the feasible set along $\nabla g(\boldsymbol{\theta}^*) = \boldsymbol{\theta}^*$ with $\mu = 2 \ge 0$ (arrows at half scale). Every feasible direction lies on the inner side of the dotted tangent line, so none decreases f. Panel (b) is the trust-region subproblem of §14 with $\mathbf{B} = 2\mathbf{I}$.

Theorem 19 (Karush–Kuhn–Tucker conditions)**.** *If $\boldsymbol{\theta}^*$ is a local minimum of f on $\mathcal{C}$ and the gradients of the active constraints at $\boldsymbol{\theta}^*$ are linearly independent, then there exist $\boldsymbol{\lambda} \in \mathbb{R}^m$ and $\boldsymbol{\mu} \in \mathbb{R}^q$ with*

$$\begin{aligned} \nabla f(\boldsymbol{\theta}^*) + \sum_{i=1}^{m} \lambda_i \nabla h_i(\boldsymbol{\theta}^*) + \sum_{j=1}^{q} \mu_j \nabla g_j(\boldsymbol{\theta}^*) &= \mathbf{0} && \text{(stationarity of } \Lambda = f + \boldsymbol{\lambda}^\top \mathbf{h} + \boldsymbol{\mu}^\top \mathbf{g}\text{)} \\ \mathbf{h}(\boldsymbol{\theta}^*) = \mathbf{0}, \qquad \mathbf{g}(\boldsymbol{\theta}^*) &\le \mathbf{0} && \text{(primal feasibility)} \\ \boldsymbol{\mu} &\ge \mathbf{0} && \text{(dual feasibility)} \\ \mu_j \, g_j(\boldsymbol{\theta}^*) &= 0 \quad \forall j && \text{(complementary slackness)} \end{aligned}$$

If moreover f and every g_j are convex and every h_i is affine, so that $\mathcal{C}$ is a convex set (Definition 11), the KKT conditions are also sufficient for a global minimum [1, §8.5.2].

The trust-region subproblem of §14 is the case $m = 0$, $q = 1$, $g(\mathbf{p}) = \frac{1}{2}(\|\mathbf{p}\|^2 - \Delta^2)$: its multiplier is the damping γ, and complementary slackness $\gamma \, g(\mathbf{p}) = 0$ reduces to $\gamma \, (\Delta - \|\mathbf{p}\|) = 0$.

5 AUTOMATIC DIFFERENTIATION

Every method in this document consumes derivatives of a program: a gradient, a Jacobian–vector product, or a curvature–vector product. Automatic differentiation (AD) computes these exactly by decomposing the program into elementary operations and evaluating the chain rule numerically [5, 6]. Unlike finite differences it incurs no approximation error beyond floating point; unlike symbolic differentiation it avoids expression swell. AD has two modes, *forward* and *reverse*, which differ in the order in which the chain of local Jacobians is multiplied.

5.1 Jacobians and Jacobian–Vector Products

Definition 18 (Jacobian)**.** For a differentiable map $\boldsymbol{\phi} : \mathbb{R}^P \to \mathbb{R}^d$, the *Jacobian* in numerator layout (outputs index rows, inputs index columns) is

$$\mathbf{J}(\boldsymbol{\theta}) \coloneqq \frac{\partial \boldsymbol{\phi}}{\partial \boldsymbol{\theta}^\top} = \begin{bmatrix} \dfrac{\partial \phi_1}{\partial \theta_1} & \cdots & \dfrac{\partial \phi_1}{\partial \theta_P} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial \phi_d}{\partial \theta_1} & \cdots & \dfrac{\partial \phi_d}{\partial \theta_P} \end{bmatrix} = \begin{bmatrix} — \, \nabla_{\boldsymbol{\theta}}^\top \phi_1 \, — \\ \vdots \\ — \, \nabla_{\boldsymbol{\theta}}^\top \phi_d \, — \end{bmatrix} \in \mathbb{R}^{d \times P}. \tag{21}$$

$\mathbf{J}$ is the linear map of the first-order expansion $\boldsymbol{\phi}(\boldsymbol{\theta} + \mathbf{d}) = \boldsymbol{\phi}(\boldsymbol{\theta}) + \mathbf{J}(\boldsymbol{\theta}) \, \mathbf{d} + o(\|\mathbf{d}\|)$; for $d = 1$ it is the row vector $\nabla f^\top$. AD never forms $\mathbf{J}$. It computes one of two products:

- the *Jacobian–vector product* (JVP) $\mathbf{J} \, \mathbf{p} \in \mathbb{R}^d$ for an input perturbation $\mathbf{p} \in \mathbb{R}^P$, the directional derivative of $\boldsymbol{\phi}$ along $\mathbf{p}$, by *forward mode*;
- the *vector–Jacobian product* (VJP) $\mathbf{J}^\top \mathbf{s} \in \mathbb{R}^P$ for an output perturbation $\mathbf{s} \in \mathbb{R}^d$, the gradient of the scalar $\mathbf{s}^\top \boldsymbol{\phi}$, by *reverse mode*.

Materialising $\mathbf{J}$ is the special case of P JVPs against $\mathbf{e}_1, \ldots, \mathbf{e}_P$ (one per column) or d VJPs against $\mathbf{e}_1, \ldots, \mathbf{e}_d$ (one per row).

5.2 Forward and Reverse Mode on Chains

Consider a differentiable function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ that decomposes as a chain of K differentiable operations:

$$\mathbf{f} = f_K \circ f_{K-1} \circ \cdots \circ f_1$$

where each $f_k : \mathbb{R}^{d_{k-1}} \to \mathbb{R}^{d_k}$ is an elementary operation (matrix multiply, pointwise nonlinearity, addition, etc.) whose local Jacobian $\mathbf{J}_{f_k} \in \mathbb{R}^{d_k \times d_{k-1}}$ is known analytically. Let $\mathbf{x}_1 := \mathbf{x} \in \mathbb{R}^n$ denote the input (so $d_0 = n$) and $\mathbf{x}_{k+1} := f_k(\mathbf{x}_k) \in \mathbb{R}^{d_k}$ the intermediate values, so $\mathbf{f}(\mathbf{x}) = \mathbf{x}_{K+1} \in \mathbb{R}^{d_K} = \mathbb{R}^m$. The *primal trace* is the sequence $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{K+1}$ computed during a forward evaluation of the program. By the chain rule, the full Jacobian factorises as a product of local Jacobians:

$$\mathbf{J}_f(\mathbf{x}) = \mathbf{J}_{f_K}(\mathbf{x}_K) \cdots \mathbf{J}_{f_2}(\mathbf{x}_2)\, \mathbf{J}_{f_1}(\mathbf{x}_1) \in \mathbb{R}^{m \times n} \tag{22}$$

Computing $\mathbf{J}_f$ explicitly by multiplying these K matrices would require $O(mn)$ memory for the dense result and at least $O(\min(m,n) \cdot \sum_k d_k d_{k-1})$ arithmetic[10]. The key insight of AD is that we never need $\mathbf{J}_f$ itself — only its products with vectors, which can be evaluated by propagating vectors through the same chain of local Jacobians, one factor at a time.

FORWARD MODE (TANGENT PROPAGATION). Consider evaluating $\mathbf{f}$ along a parameterised path through the input space:

$$\mathbf{x}(\epsilon) = \mathbf{x}_0 + \epsilon\, \mathbf{p}, \qquad \mathbf{p} \in \mathbb{R}^n$$

Each intermediate variable becomes a function of ϵ: $\mathbf{x}_k(\epsilon) = f_{k-1}(\mathbf{x}_{k-1}(\epsilon))$. Define the *tangent* at each stage as the derivative with respect to ϵ, evaluated at $\epsilon = 0$:

$$\dot{\mathbf{x}}_k := \left.\frac{d\mathbf{x}_k}{d\epsilon}\right|_{\epsilon=0}, \quad \in \mathbb{R}^{d_{k-1}}$$

By the chain rule applied to $\mathbf{x}_{k+1}(\epsilon) = f_k(\mathbf{x}_k(\epsilon))$:

$$\dot{\mathbf{x}}_1 = \mathbf{p}, \qquad \dot{\mathbf{x}}_{k+1} = \mathbf{J}_{f_k}(\mathbf{x}_k)\, \dot{\mathbf{x}}_k, \qquad k = 1, \ldots, K \tag{23}$$

Unrolling this recursion gives $\dot{\mathbf{x}}_{K+1} = \mathbf{J}_{f_K} \cdots \mathbf{J}_{f_1}\, \mathbf{p} = \mathbf{J}_f\, \mathbf{p}$. That is, one forward pass with seed $\mathbf{p}$ computes exactly the JVP $\mathbf{J}_f \mathbf{p}$ — a directional derivative of $\mathbf{f}$ along $\mathbf{p}$. Setting $\mathbf{p} = \mathbf{e}_j$ (the j-th standard basis vector) extracts the j-th column of $\mathbf{J}_f$, so n forward passes recover the full Jacobian. The crucial observation is that step k of (23) is a single local Jacobian–vector product costing $O(d_k \cdot d_{k-1})$ operations, and requires only the current pair $(\mathbf{x}_k, \dot{\mathbf{x}}_k)$ — previous intermediates can be discarded. Hence: time is $O(\sum_k d_k d_{k-1})$, proportional to the cost of evaluating $\mathbf{f}$ itself; memory is $O(\max_k d_k)$, *independent of the chain length* K. More precisely, a single forward-mode pass costs at most $\sim 2.5\times$ the time cost of evaluating $\mathbf{f}$ [6, 7].

MATRIX PRODUCTS IN FORWARD MODE. Most of the work in a network is matrix products, and their tangent rule repeats the primal computation. For a step $\mathbf{y} = \mathbf{W}\mathbf{x}$ with fixed $\mathbf{W}$, the map is linear in $\mathbf{x}$, so its derivative is the map itself:

$$\begin{aligned} \mathbf{W}(\mathbf{x} + \epsilon\, \dot{\mathbf{x}}) &= \mathbf{W}\mathbf{x} + \epsilon\, \mathbf{W}\dot{\mathbf{x}} && \text{(linearity)} \\ \implies \quad \dot{\mathbf{y}} &= \mathbf{W}\dot{\mathbf{x}}. && \text{(coefficient of } \epsilon) \end{aligned}$$

The tangent pass applies the same matrices as the primal pass, to different inputs. Seeding r tangents at once stacks them as columns, $\mathbf{W}[\dot{\mathbf{x}}_1 \cdots \dot{\mathbf{x}}_r]$, which is one matrix product with r columns, exactly a primal pass at batch size r; seeding with all n columns of $\mathbf{I}_n$ is the batch that returns the full Jacobian. When we differentiate with respect to the parameters, the seed also perturbs the weights, $\mathbf{W} \to \mathbf{W} + \epsilon\, \dot{\mathbf{W}}$ with $\dot{\mathbf{W}}$ the block of the seed belonging to $\mathbf{W}$, so both factors of the product move and the product rule adds a second matrix product:

$$\begin{aligned} (\mathbf{W} + \epsilon\, \dot{\mathbf{W}})(\mathbf{x} + \epsilon\, \dot{\mathbf{x}}) &= \mathbf{W}\mathbf{x} + \epsilon\, (\mathbf{W}\dot{\mathbf{x}} + \dot{\mathbf{W}}\mathbf{x}) + \epsilon^2\, \dot{\mathbf{W}}\dot{\mathbf{x}} && \text{(expand)} \\ \implies \quad \dot{\mathbf{y}} &= \mathbf{W}\dot{\mathbf{x}} + \dot{\mathbf{W}}\mathbf{x}. && \text{(coefficient of } \epsilon) \end{aligned}$$

The new term takes the tangent $\dot{\mathbf{W}}$ as its matrix and the primal $\mathbf{x}$ as its input, and since each seed has its own $\dot{\mathbf{W}}$ it does not batch as a single product. This is the structure of the RNN tangent recursion of SOFO (§2 of the SOFO notes).

10 Accumulating right-to-left, the running product is $\mathbb{R}^{d_k \times n}$ at step k, so each multiply costs $O(d_k d_{k-1} n)$ and the total is $O(n \sum_k d_k d_{k-1})$. Accumulating left-to-right gives $O(m \sum_k d_k d_{k-1})$ by symmetric reasoning. Taking the cheaper order yields the $\min(m, n)$ factor.

REVERSE MODE (ADJOINT PROPAGATION). Define the *adjoint* of intermediate $\mathbf{x}_k$ as the sensitivity of the final output to perturbations at that stage:[11]

$$\bar{\mathbf{x}}_k := \frac{\partial \mathbf{f}}{\partial \mathbf{x}_k} \in \mathbb{R}^{m \times d_{k-1}}$$

Since $\mathbf{f}$ depends on $\mathbf{x}_k$ only through $\mathbf{x}_{k+1} = f_k(\mathbf{x}_k)$, the chain rule gives:

$$\bar{\mathbf{x}}_{K+1} = \mathbf{I}_m, \qquad \bar{\mathbf{x}}_k = \bar{\mathbf{x}}_{k+1} \mathbf{J}_{f_k}(\mathbf{x}_k), \qquad k = K, \dots, 1 \tag{24}$$

In practice, one does not initialise with $\mathbf{I}_m$ and compute the full Jacobian. Instead, given a seed $\mathbf{s} \in \mathbb{R}^m$, one initialises $\bar{\mathbf{x}}_{K+1} = \mathbf{s}^\top$ and propagates a *row vector* backward: $\bar{\mathbf{x}}_k = \bar{\mathbf{x}}_{k+1} \mathbf{J}_{f_k}(\mathbf{x}_k)$. Unrolling gives $\bar{\mathbf{x}}_1 = \mathbf{s}^\top \mathbf{J}_{f_K} \cdots \mathbf{J}_{f_1} = \mathbf{s}^\top \mathbf{J}_f$, or equivalently $\bar{\mathbf{x}}_1^\top = \mathbf{J}_f^\top \mathbf{s}$: the VJP, computed without ever forming $\mathbf{J}_f$. For a scalar loss ($m = 1$), one reverse pass with $\bar{\mathbf{x}}_{K+1} = 1$ yields the full gradient $\nabla_{\mathbf{x}} f = \bar{\mathbf{x}}_1^\top$ — *this is backpropagation*. However, evaluating (24) at step k requires the primal value $\mathbf{x}_k$ to compute the local Jacobian $\mathbf{J}_{f_k}(\mathbf{x}_k)$. Since the backward pass traverses $k = K, K-1, \dots, 1$, *all* intermediate values $\mathbf{x}_1, \dots, \mathbf{x}_K$ must be retained from the forward pass. This stored sequence is called the *tape*. Hence: time is $O(\sum_k d_k d_{k-1})$, the same as forward mode; but memory is $O(K \cdot \max_k d_k)$ for the tape — *linear in the chain length*, unlike forward mode's $O(\max_k d_k)$. A single reverse-mode pass costs at most $\sim 4\times$ the time cost of evaluating $\mathbf{f}$ [6, 7].

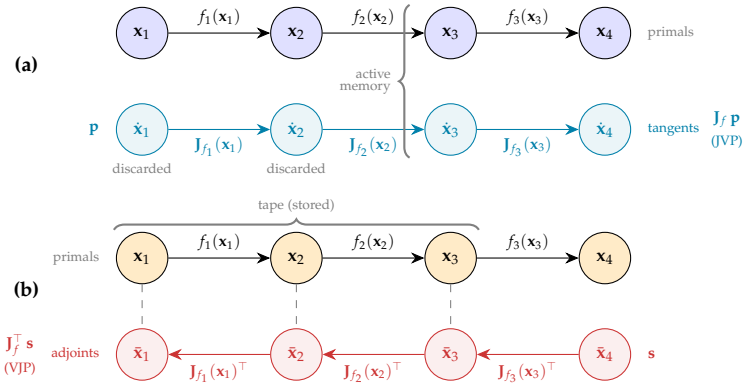


Figure 8: Forward and reverse mode AD on a chain $\mathbf{f} = f_3 \circ f_2 \circ f_1$. **(a)** Forward mode propagates tangents $\dot{\mathbf{x}}_k$ left-to-right alongside primals $\mathbf{x}_k$. At each step, only the current pair $(\mathbf{x}_k, \dot{\mathbf{x}}_k)$ is needed — previous values can be discarded. **(b)** Reverse mode propagates adjoints $\bar{\mathbf{x}}_k$ right-to-left, but each adjoint step $\bar{\mathbf{x}}_k = \bar{\mathbf{x}}_{k+1} \mathbf{J}_{f_k}(\mathbf{x}_k)$ requires the stored primal $\mathbf{x}_k$ (dashed lines), so all intermediates must be retained on a tape during the forward pass.

5.3 General Computation Graphs

The chain case suffices for unrolled RNNs, but general computation graphs are directed acyclic graphs (DAGs) where a node $\mathbf{v}_k = f_k(\mathbf{v}_{j_1}, \dots, \mathbf{v}_{j_p})$ may depend on multiple parents. The Jacobian product (22) no longer applies directly, but the propagation rules generalise naturally. In forward mode, the tangent at node $\mathbf{v}_k$ sums contributions from all parents:

$$\dot{\mathbf{v}}_k = \sum_{j \in \text{parents}(k)} \frac{\partial f_k}{\partial \mathbf{v}_j} \dot{\mathbf{v}}_j \tag{25}$$

In reverse mode, the adjoint at node $\mathbf{v}_j$ sums contributions from all children:

$$\bar{\mathbf{v}}_j = \sum_{k \in \text{children}(j)} \bar{\mathbf{v}}_k \frac{\partial f_k}{\partial \mathbf{v}_j} \tag{26}$$

The chain case (23) and (24) are the special case where each node has exactly one parent and one child. Reverse mode on a graph, seeded with the scalar loss, is exactly backpropagation. Figure 9 illustrates both rules on a small DAG.

11 For vector-valued $\mathbf{f}$, $\bar{\mathbf{x}}_k$ is a matrix. In the common case $m = 1$ (scalar loss), it reduces to a column vector.

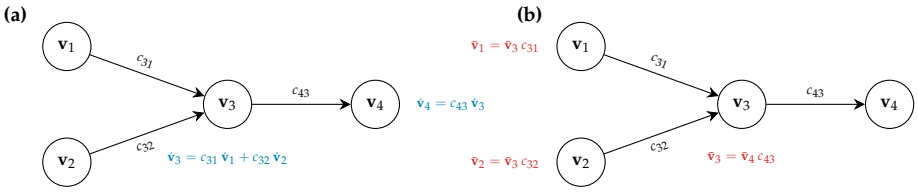


Figure 9: AD on a DAG where $\mathbf{v}_3$ has two parents $(\mathbf{v}_1, \mathbf{v}_2)$. Arc labels $c_{kj} \coloneqq \partial f_k / \partial \mathbf{v}_j$ are the local (elemental) partials. **(a)** Forward mode: the tangent $\dot{\mathbf{v}}_3$ sums over parents (Eq. 25). **(b)** Reverse mode: the adjoints $\bar{\mathbf{v}}_1, \bar{\mathbf{v}}_2$ each receive a contribution from their shared child $\mathbf{v}_3$ (Eq. 26).

5.4 Complexity Comparison

Both modes evaluate one local Jacobian–vector product per step, at the same per-step cost. The asymmetry lies in how many passes are needed and what must be stored:

	Forward mode	**Reverse mode**
Computes per pass	JVP: $\mathbf{J}_f\,\mathbf{p}$	VJP: $\mathbf{J}_f^\top\,\mathbf{s}$
Passes for full $\mathbf{J}_f$	n (one per input dim.)	m (one per output dim.)
Efficient when	$n \ll m$	$m \ll n$
Time per pass	$O(\sum_k d_k d_{k-1})$	$O(\sum_k d_k d_{k-1})$
Memory	$O(\max_k d_k)$	$O(K \cdot \max_k d_k)$

In machine learning, the loss is scalar ($m = 1$) and $n = P$ is large, so reverse mode dominates: one pass gives the full gradient. However, a single forward-mode pass still yields useful information: the directional derivative $\nabla f \cdot \mathbf{v}$ along a random direction $\mathbf{v}$. Baydin et al. [8] show that $g = (\nabla f \cdot \mathbf{v})\,\mathbf{v}$ is an unbiased gradient estimator computable without any backward pass, though with high variance.

5.5 Composing Modes: Hessian–Vector and GGN–Vector Products

Second-order methods need the action of a curvature matrix on a vector. For the Hessian $\mathbf{H}_{\boldsymbol{\theta}}$ of a scalar f and for the generalised Gauss–Newton matrix $\mathbf{G} = \mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}} \mathbf{J}$ of a composite $f = \mathcal{L}(\boldsymbol{\phi}(\boldsymbol{\theta}))$ (§15), the two products have fundamentally different costs. A full HVP $\mathbf{H}_{\boldsymbol{\theta}}\mathbf{v} = \nabla_{\boldsymbol{\theta}}[\nabla_{\boldsymbol{\theta}} f \cdot \mathbf{v}]$ is a second-order derivative requiring *forward-over-reverse* AD (Algorithm 1): the reverse pass must differentiate *through* a forward-mode computation — a nested AD call. The GGN's factored form avoids this: $\mathbf{G}\mathbf{v} = \mathbf{J}^\top(\mathbf{H}_{\boldsymbol{\phi}}(\mathbf{J}\mathbf{v}))$ decomposes into one JVP, an analytic multiply by $\mathbf{H}_{\boldsymbol{\phi}}$, and one VJP — all acting on the same original $\boldsymbol{\phi}$, none nested (Algorithm 2) [5, 6, 9].

Algorithm 1 HVP: $\mathbf{H}_{\boldsymbol{\theta}}\mathbf{v}$ via forward-over-reverse (**nested**)

1: **input:** $f : \mathbb{R}^P \to \mathbb{R}$, $\boldsymbol{\theta}$, $\mathbf{v} \in \mathbb{R}^P$
2: $s \leftarrow \texttt{JVP}(f, \boldsymbol{\theta}, \mathbf{v})$ ▷ forward: scalar $s = \nabla_{\boldsymbol{\theta}} f \cdot \mathbf{v}$; $\approx 1\times$ fwd
3: $\mathbf{H}_{\boldsymbol{\theta}}\mathbf{v} \leftarrow \texttt{VJP}(s, \boldsymbol{\theta})$ ▷ reverse *through* s (**nested**: s is output of line 2); $\approx 1\times$ rev
4: **return** $\mathbf{H}_{\boldsymbol{\theta}}\mathbf{v} \in \mathbb{R}^P$ total $\approx 5\times$ fwd eval

Algorithm 2 GVP: $\mathbf{G}\mathbf{v} = \mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}} \mathbf{J}\mathbf{v}$ via three independent stages (**no nesting**)

1: **input:** $\boldsymbol{\phi} : \mathbb{R}^P \to \mathbb{R}^d$, $\boldsymbol{\theta}$, $\mathbf{v} \in \mathbb{R}^P$
2: $\mathbf{u} \leftarrow \texttt{JVP}(\boldsymbol{\phi}, \boldsymbol{\theta}, \mathbf{v})$ ▷ $\mathbf{u} = \mathbf{J}\mathbf{v} \in \mathbb{R}^d$; $\leq 2\times$ fwd
3: $\mathbf{w} \leftarrow \mathbf{H}_{\boldsymbol{\phi}} \cdot \mathbf{u}$ ▷ analytic multiply, *no AD*; cheap
4: $\mathbf{G}\mathbf{v} \leftarrow \texttt{VJP}(\boldsymbol{\phi}, \boldsymbol{\theta}, \mathbf{w})$ ▷ reverse through $\boldsymbol{\phi}$ (**not nested**: same target as line 2); $\leq 3\times$ fwd
5: **return** $\mathbf{G}\mathbf{v} \in \mathbb{R}^P$ total $\leq 5\times$ fwd eval

The full $P \times P$ matrix is never formed — only its action on $\mathbf{v}$ is computed, at a total cost of ≤ 5 forward evaluations per vector [6, 9]. This is the basis of the Hessian-free methods of §19, which solve $(\mathbf{G} + \gamma\mathbf{I})\,\mathbf{d} = -\nabla_{\boldsymbol{\theta}} f$ by conjugate gradient: each CG iteration requires one such product, and the iterations are inherently sequential.

Part II.
First-Order Methods

6 LOCAL MODELS AND CONVERGENCE TEMPLATES

Almost every method in these notes, SOFO included, takes a step by minimising a local quadratic model of f, and the methods differ only in the curvature matrix of that model. This section sets up the model, then states three short templates that turn a per-step decrease into a convergence rate.

6.1 The Local Quadratic Model

At the iterate $\boldsymbol{\theta}_k$, form the *local quadratic model* [10]

$$M_k(\mathbf{d}) \;=\; f(\boldsymbol{\theta}_k) \;+\; \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)^\top \mathbf{d} \;+\; \frac{1}{2}\,\mathbf{d}^\top \mathbf{B}_k\,\mathbf{d}, \tag{27}$$

with a symmetric positive definite **curvature matrix** $\mathbf{B}_k$. Setting $\nabla_{\mathbf{d}} M_k = \mathbf{0}$ gives the step

$$\mathbf{d}_k \;=\; -\mathbf{B}_k^{-1}\,\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k). \tag{28}$$

The linear term alone is unbounded below; the quadratic term is a *proximity penalty* that keeps the step where the linear approximation is reliable, and $\mathbf{B}_k$ decides how distance is measured. Table 2 lists the choices made in the rest of these notes.

Curvature matrix $\mathbf{B}_k$	Method	Where
$\frac{1}{\alpha}\mathbf{I}$	gradient descent	§7
$\frac{1}{\alpha}\mathbf{P}_k^{-1}$, $\mathbf{P}_k$ diagonal from gradient statistics	Adagrad, RMSProp, Adam	§12
$\mathbf{H}_k$ (Hessian)	Newton	§13
$\mathbf{B} + \gamma\mathbf{I}$ (damped)	trust region, Levenberg–Marquardt	§14
$\mathbf{G}$ (generalised Gauss–Newton)	Gauss–Newton, Hessian-free	§15, §19
$\frac{1}{\alpha}\mathbf{F}$ (Fisher)	natural gradient	§16.4
Kronecker-factored blocks	KFAC, Shampoo, SOAP	§20
$\mathbf{G}$ restricted to a random subspace	SOFO	§4 of the SOFO notes

Table 2: Methods as choices of the curvature matrix in (27). Part V replaces the penalty $\frac{1}{2}\mathbf{d}^\top\mathbf{B}_k\mathbf{d}$ by $\frac{1}{2\alpha}\|\mathbf{d}\|^2$ under a general norm, which adds sign descent and Muon.

Momentum (§8) is the one family that does not fit the table: it keeps $\mathbf{B}_k = \frac{1}{\alpha}\mathbf{I}$ but adds memory of past steps.

6.2 Rate Taxonomy

We adopt the standard classification of error-decay rates. Let $\{e_k\}$ be a non-negative scalar error sequence (e.g. $e_k = f(\boldsymbol{\theta}_k) - f^*$ or $e_k = \|\nabla f(\boldsymbol{\theta}_k)\|^2$), and let $\varepsilon > 0$ be the target accuracy. The asymptotic forms describe the iteration count K needed to reach $e_K \le \varepsilon$:

Rate	Form	Iterations to reach $e_K \le \varepsilon$
Sublinear	$e_k \le C/k$	$O(1/\varepsilon)$
Linear (geometric)	$e_k \le (1-\omega)^k\, e_0,\ \omega \in (0,1]$	$O\big(\frac{1}{\omega}\log\frac{1}{\varepsilon}\big)$
Superlinear	$e_{k+1}/e_k \to 0$	faster than any linear
Quadratic	$e_{k+1} \le C\, e_k^2$	$O\big(\log\log\frac{1}{\varepsilon}\big)$

6.3 Linear Convergence under a PL-type Bound

A per-step descent in function value, combined with a PL lower bound on the gradient, gives a geometric rate.

Lemma 20 (Linear convergence under PL)**.** *Suppose f is bounded below by f^*, the iterates $\{\boldsymbol{\theta}_k\}$ satisfy the per-step descent*

$$f(\boldsymbol{\theta}_{k+1}) \;\le\; f(\boldsymbol{\theta}_k) - c\,\|\nabla f(\boldsymbol{\theta}_k)\|^2 \qquad (c>0),$$

and f satisfies the PL inequality $\|\nabla f(\boldsymbol{\theta})\|^2 \ge 2\mu\,(f(\boldsymbol{\theta}) - f^*)$ with $\mu > 0$. Then

$$f(\boldsymbol{\theta}_k) - f^* \;\le\; (1-2c\mu)^k\,\big(f(\boldsymbol{\theta}_0) - f^*\big). \tag{29}$$

Proof. Substituting PL into the per-step descent, $f(\boldsymbol{\theta}_{k+1}) \le f(\boldsymbol{\theta}_k) - 2c\mu(f(\boldsymbol{\theta}_k) - f^*)$; subtracting f^* gives $f(\boldsymbol{\theta}_{k+1}) - f^* \le (1-2c\mu)(f(\boldsymbol{\theta}_k) - f^*)$, and iterating from $k=0$ gives (29). □

6.4 Sublinear Convergence under Convexity

When f is convex but not strongly so (no PL-type lower bound), the geometric rate degrades to the sublinear $O(1/k)$ rate familiar from gradient descent on smooth convex objectives.

Lemma 21 (Sublinear convergence, convex)**.** *Suppose f is convex with a minimiser $\boldsymbol{\theta}^*$, the iterates satisfy the per-step descent*

$$f(\boldsymbol{\theta}_{k+1}) \;\le\; f(\boldsymbol{\theta}_k) \;-\; c\,\|\nabla f(\boldsymbol{\theta}_k)\|^2 \tag{30}$$

for some $c > 0$, and the iterates remain bounded:[12] *$R := \sup_{k\ge 0}\|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\| < \infty$. Then*

$$f(\boldsymbol{\theta}_k) - f^* \;\le\; \frac{R^2}{c\,k} \qquad \forall\, k \ge 1. \tag{31}$$

The proof combines first-order convexity, $f(\boldsymbol{\theta}_k) - f^* \le \|\nabla f(\boldsymbol{\theta}_k)\|\,R$, with the per-step descent to get $\Delta_{k+1} \le \Delta_k - (c/R^2)\Delta_k^2$ for $\Delta_k = f(\boldsymbol{\theta}_k) - f^*$, a recursion whose solution decays like $1/k$ [2, §2.1.5].

6.5 Stationarity in the Non-Convex Case

When neither convexity nor PL is available — the practically relevant setting for neural-network losses — the best one can claim is that the gradient becomes small along the iterates.

Lemma 22 (Non-convex stationarity)**.** *Suppose f is bounded below by f^* and the iterates satisfy the per-step descent*

$$f(\boldsymbol{\theta}_{k+1}) \;\le\; f(\boldsymbol{\theta}_k) - c\,\|\nabla f(\boldsymbol{\theta}_k)\|^2 \qquad (c>0).$$

Then for any $K \ge 1$,

$$\min_{0\le k<K} \|\nabla f(\boldsymbol{\theta}_k)\|^2 \;\le\; \frac{f(\boldsymbol{\theta}_0) - f^*}{c\,K}. \tag{32}$$

Proof. Summing the per-step descent from $k=0$ to $K-1$ telescopes:

$$c\sum_{k=0}^{K-1}\|\nabla f(\boldsymbol{\theta}_k)\|^2 \;\le\; f(\boldsymbol{\theta}_0) - f(\boldsymbol{\theta}_K) \;\le\; f(\boldsymbol{\theta}_0) - f^*.$$

Since the minimum of a non-negative sequence is at most its average, $\min_{k<K}\|\nabla f(\boldsymbol{\theta}_k)\|^2 \le \frac{1}{K}\sum_{k<K}\|\nabla f(\boldsymbol{\theta}_k)\|^2 \le (f(\boldsymbol{\theta}_0) - f^*)/(cK)$. □

7 GRADIENT DESCENT

Set $\mathbf{B}_k = \frac{1}{\alpha}\mathbf{I}$ — isotropic curvature, treating all parameter directions as equally sensitive. The local model (27) becomes

$$M_k(\mathbf{d}) \;=\; f(\boldsymbol{\theta}_k) + \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)^\top \mathbf{d} + \frac{1}{2\alpha}\|\mathbf{d}\|^2.$$

The first-order Taylor approximation $f(\boldsymbol{\theta}_k + \mathbf{d}) \approx f(\boldsymbol{\theta}_k) + \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)^\top \mathbf{d}$ is linear and unbounded below; the isotropic penalty $\frac{1}{2\alpha}\|\mathbf{d}\|^2$ keeps the step finite. From (28), $\mathbf{d}_k = -\alpha\,\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$, giving the gradient descent update:

$$\boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k - \alpha_k\,\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k) \tag{33}$$

where $\alpha_k > 0$ may be fixed or chosen by line search.

12 The hypothesis $R < \infty$ holds automatically when the method is a descent method and f is coercive: $\{\boldsymbol{\theta}_k\}$ stays in the initial sublevel set $\{f \le f(\boldsymbol{\theta}_0)\}$, which is bounded.

7.1 Per-Step Descent

All four convergence regimes for GD on L-smooth f follow from a single per-step inequality, obtained by substituting the GD update with constant step $\alpha = 1/L$ into the descent lemma (17).

Lemma 23 (GD per-step descent). *If f is L-smooth and gradient descent is run with step size $\alpha = 1/L$, then*

$$f(\boldsymbol{\theta}_{k+1}) \;\le\; f(\boldsymbol{\theta}_k) \;-\; \frac{1}{2L}\,\|\nabla f(\boldsymbol{\theta}_k)\|^2. \tag{34}$$

Proof. Apply the descent lemma (17) at $\boldsymbol{\theta}' = \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - (1/L)\nabla f(\boldsymbol{\theta}_k)$:

$$f(\boldsymbol{\theta}_{k+1}) \;\le\; f(\boldsymbol{\theta}_k) - \tfrac{1}{L}\|\nabla f(\boldsymbol{\theta}_k)\|^2 + \tfrac{L}{2}\cdot\tfrac{1}{L^2}\|\nabla f(\boldsymbol{\theta}_k)\|^2 \;=\; f(\boldsymbol{\theta}_k) - \tfrac{1}{2L}\|\nabla f(\boldsymbol{\theta}_k)\|^2. \quad \square$$

The constant $c = 1/(2L)$ is what each of the four regimes below feeds into a convergence lemma from §6.

7.2 Convergence in Four Regimes

Theorem 24 (GD, non-convex L-smooth). *If f is L-smooth and bounded below by f^*, then GD with $\alpha = 1/L$ satisfies, for any $K \ge 1$,*

$$\min_{0\le k<K} \|\nabla f(\boldsymbol{\theta}_k)\|^2 \;\le\; \frac{2L\,(f(\boldsymbol{\theta}_0) - f^*)}{K}. \tag{35}$$

Proof. Apply the non-convex stationarity lemma (Lemma 22) with $c = 1/(2L)$, per-step descent supplied by Lemma 23. $\square$

Theorem 25 (GD, convex + L-smooth). *If f is L-smooth and convex with minimiser $\boldsymbol{\theta}^*$, then GD with $\alpha = 1/L$ satisfies, for any $k \ge 1$,*

$$f(\boldsymbol{\theta}_k) - f^* \;\le\; \frac{2L\,\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|^2}{k}. \tag{36}$$

Proof. For L-smooth convex f the GD iterates never move away from $\boldsymbol{\theta}^*$ [2, Thm. 2.1.5], so $R = \|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|$. Apply the sublinear convex lemma (Lemma 21) with $c = 1/(2L)$, per-step descent from Lemma 23. $\square$

Theorem 26 (GD, μ-strongly convex + L-smooth). *If f is L-smooth and μ-strongly convex, then GD with $\alpha = 1/L$ satisfies*

$$f(\boldsymbol{\theta}_k) - f^* \;\le\; \left(1 - \tfrac{\mu}{L}\right)^k (f(\boldsymbol{\theta}_0) - f^*). \tag{37}$$

Proof. Strong convexity implies PL with the same μ (Corollary 16). Apply the linear-convergence lemma (Lemma 20) with $c = 1/(2L)$ and μ from PL: rate $1 - 2c\mu = 1 - \mu/L$. $\square$

Theorem 27 (GD under PL — no convexity required). *If f is L-smooth and satisfies the PL inequality with constant $\mu > 0$ (Definition 15), then GD with $\alpha = 1/L$ satisfies* [13]

$$f(\boldsymbol{\theta}_k) - f^* \;\le\; \left(1 - \tfrac{\mu}{L}\right)^k (f(\boldsymbol{\theta}_0) - f^*). \tag{38}$$

Proof. Identical to Theorem 26: PL is exactly the hypothesis of the linear-convergence lemma (Lemma 20), and strong convexity was used in Theorem 26 *only* to supply that bound. $\square$

Remark 28 (Sharp rate under SC + L-smooth). A finer analysis with step size $\alpha = 2/(L+\mu)$ [2, Thm. 2.1.15] gives the optimal first-order rate

$$\|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|^2 \;\le\; \left(\tfrac{L-\mu}{L+\mu}\right)^{2k}\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|^2, \qquad f(\boldsymbol{\theta}_k) - f^* \;\le\; \tfrac{L}{2}\left(\tfrac{\kappa-1}{\kappa+1}\right)^{2k}\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|^2, \tag{39}$$

with $\kappa = L/\mu$. This is the classical Kantorovich rate. [14]

The linear rate $1 - \mu/L = 1 - 1/\kappa$ in Theorems 26 and 27 is dominated by the condition number $\kappa = L/\mu$. For neural-network losses κ can be very large (§18), giving rates close to 1; this is the practical motivation for the curvature-aware methods of later sections. Geometrically, gradient descent *zig-zags* on elongated elliptical contours: each step makes good progress perpendicular to the dominant valley but almost none along it. Newton's method (§13) eliminates this by rescaling so all directions have unit curvature.

13 The equivalence between Theorems 26 and 27 is the central insight of Karimi et al. [3]: the geometric convergence of GD does *not* require convexity, only the PL inequality, which is far more plausible for over-parameterised neural-network losses (Definition 15, Lemma 17).

14 The simpler $(1-\mu/L)^k$ form of Theorem 26 is what generalises cleanly to the sketched subspace methods of §3.2–§4, rather than the sharp $((\kappa-1)/(\kappa+1))^2$ form.

7.3 Line Search

A fixed step $\alpha = 1/L$ needs the smoothness constant L, which is rarely known. Line search instead chooses α_k along the current direction $\mathbf{d}_k$ by solving the one-dimensional problem

$$\alpha_k \;=\; \arg\min_{\alpha>0}\; \varphi(\alpha), \qquad \varphi(\alpha) \;:=\; f(\boldsymbol{\theta}_k + \alpha\,\mathbf{d}_k).$$

Exact line search solves this problem exactly, which has a closed form for a quadratic $f(\boldsymbol{\theta}) = \tfrac{1}{2}\boldsymbol{\theta}^\top \mathbf{A}\boldsymbol{\theta} - \mathbf{b}^\top\boldsymbol{\theta}$ with $\mathbf{A} \succ 0$. Writing $\mathbf{g}_k := \nabla f(\boldsymbol{\theta}_k) = \mathbf{A}\boldsymbol{\theta}_k - \mathbf{b}$:

$$\begin{aligned} \varphi(\alpha) &= f(\boldsymbol{\theta}_k) + \alpha\,\mathbf{g}_k^\top \mathbf{d}_k + \tfrac{\alpha^2}{2}\,\mathbf{d}_k^\top \mathbf{A}\,\mathbf{d}_k \quad \text{((5) is exact: constant Hessian } \mathbf{A}) \\ \varphi'(\alpha) &= \mathbf{g}_k^\top \mathbf{d}_k + \alpha\,\mathbf{d}_k^\top \mathbf{A}\,\mathbf{d}_k = 0 \\ \Longrightarrow \quad \alpha_k &= -\frac{\mathbf{g}_k^\top \mathbf{d}_k}{\mathbf{d}_k^\top \mathbf{A}\,\mathbf{d}_k}. \end{aligned}$$

For gradient descent, $\mathbf{d}_k = -\mathbf{g}_k$ gives $\alpha_k = \|\mathbf{g}_k\|^2/\mathbf{g}_k^\top \mathbf{A}\,\mathbf{g}_k = 1/R_{\mathbf{A}}(\mathbf{g}_k)$, the inverse Rayleigh quotient (Definition 5), so $\alpha_k \in [1/\lambda_{\max}(\mathbf{A}),\, 1/\lambda_{\min}(\mathbf{A})]$ by Theorem 4.

For any differentiable f, exact line search makes successive gradient-descent directions orthogonal. At the minimiser α_k of φ,

$$\begin{aligned} 0 \;&=\; \varphi'(\alpha_k) = \nabla f(\boldsymbol{\theta}_k + \alpha_k \mathbf{d}_k)^\top \mathbf{d}_k = \nabla f(\boldsymbol{\theta}_{k+1})^\top \mathbf{d}_k \quad \text{(chain rule)} \\ \Longrightarrow \quad \mathbf{d}_{k+1}^\top \mathbf{d}_k &= -\nabla f(\boldsymbol{\theta}_{k+1})^\top \mathbf{d}_k = 0 \qquad\qquad (\mathbf{d}_{k+1} = -\nabla f(\boldsymbol{\theta}_{k+1})) \end{aligned}$$

The step stops where $\mathbf{d}_k$ is tangent to a level set of f, and the next gradient is normal to that level set. On elongated contours (large κ) the iterates therefore alternate between two perpendicular directions, crossing the valley rather than travelling along it: this is the zig-zag of §7, and choosing the step optimally does not remove it. Momentum (§8) and curvature do.

Outside quadratics exact line search is rarely worth its cost, and any step giving *sufficient decrease* is accepted instead. The *Armijo condition* requires the actual decrease to be at least a fraction c of the decrease predicted by the linear model:

$$f(\boldsymbol{\theta}_k + \alpha\,\mathbf{d}_k) \;\le\; f(\boldsymbol{\theta}_k) + c\,\alpha\,\nabla f(\boldsymbol{\theta}_k)^\top \mathbf{d}_k, \qquad c \in (0,1),$$

typically $c = 10^{-4}$. For a descent direction it holds for all small enough α, by the first-order Taylor expansion. *Backtracking* starts from an initial α (e.g. 1) and shrinks $\alpha \leftarrow \tau\alpha$ with $\tau \in (0,1)$ until the Armijo condition holds [1, 11]. For L-smooth f and $\mathbf{d}_k = -\nabla f(\boldsymbol{\theta}_k)$, the descent lemma (17) shows the condition holds for every $\alpha \le 2(1-c)/L$, so backtracking finds a step of order $1/L$ without knowing L.

8 MOMENTUM AND ACCELERATION

8.1 Heavy Ball

Each gradient-descent step forgets the previous one, which is why the zig-zag survives even an optimal step size. The *heavy-ball* method [12] adds a multiple of the previous displacement to the gradient step:

$$\boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k - \alpha\,\nabla f(\boldsymbol{\theta}_k) + \beta\,(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1}), \qquad \alpha > 0,\;\; \beta \in [0,1), \tag{40}$$

with $\boldsymbol{\theta}_{-1} := \boldsymbol{\theta}_0$; $\beta = 0$ is gradient descent. Defining the *momentum* $\mathbf{m}_{k+1} := (\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k+1})/\alpha$, so that $\mathbf{m}_0 = \mathbf{0}$ and $\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1} = -\alpha\,\mathbf{m}_k$, the update becomes

$$\begin{aligned} \boldsymbol{\theta}_{k+1} &= \boldsymbol{\theta}_k - \alpha\,\big(\beta\,\mathbf{m}_k + \nabla f(\boldsymbol{\theta}_k)\big) \qquad\qquad \text{(substitute into (40))} \\ \Longrightarrow \quad \mathbf{m}_{k+1} &= \beta\,\mathbf{m}_k + \nabla f(\boldsymbol{\theta}_k) = \sum_{j=0}^{k} \beta^{k-j}\,\nabla f(\boldsymbol{\theta}_j) \quad \text{(unroll from } \mathbf{m}_0 = \mathbf{0}) \end{aligned}$$

Each step is therefore an exponentially weighted sum of past gradients. Components along which the gradient keeps its sign accumulate, while components that flip sign from step to step, such as the zig-zag across a valley, cancel [13].

The name comes from physics: (40) discretises the motion of a ball rolling in the potential f with friction, $\ddot{\boldsymbol{\theta}} + a\,\dot{\boldsymbol{\theta}} + \nabla f(\boldsymbol{\theta}) = \mathbf{0}$. Replacing the derivatives by finite differences with time step h gives (40) with $\alpha = h^2$ and $\beta = 1 - ah$.

Theorem 29 (Heavy ball on quadratics [12]). *Let* $f(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{A}\boldsymbol{\theta} - \mathbf{b}^\top\boldsymbol{\theta}$ *with* $\mu\mathbf{I} \preceq \mathbf{A} \preceq L\mathbf{I}$, $\mu > 0$, $\kappa = L/\mu$, *and minimiser* $\boldsymbol{\theta}^* = \mathbf{A}^{-1}\mathbf{b}$. *Run* (40) *with*

$$\alpha = \frac{4}{(\sqrt{L}+\sqrt{\mu})^2}, \qquad \beta = \Big(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\Big)^2.$$

Then the error $\mathbf{e}_k \coloneqq \boldsymbol{\theta}_k - \boldsymbol{\theta}^*$ *obeys a linear recursion whose iteration matrix has spectral radius*

$$\rho \;=\; \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}. \tag{41}$$

The proof writes the error recursion $\mathbf{e}_{k+1} = ((1+\beta)\mathbf{I} - \alpha\mathbf{A})\mathbf{e}_k - \beta\mathbf{e}_{k-1}$ in the eigenbasis of $\mathbf{A}$, where it splits into one 2×2 linear recursion per eigenvalue λ_i; the stated (α,β) are the smallest choice that makes both roots of every such recursion complex with modulus $\sqrt{\beta}$ [12, 13].

Gradient descent with its best fixed step contracts the distance to $\boldsymbol{\theta}^*$ by $(\kappa-1)/(\kappa+1)$ per step (Remark 28). For large κ, $-\ln\frac{\kappa-1}{\kappa+1} \approx 2/\kappa$ while $-\ln\rho \approx 2/\sqrt{\kappa}$, so reducing the error by a factor e takes about $\kappa/2$ gradient-descent steps but only $\sqrt{\kappa}/2$ heavy-ball steps: 5000 versus 50 at $\kappa = 10^4$. The spectral radius is only an asymptotic rate: at the extreme eigenvalues μ and L the two roots of the per-eigenvalue recursion coincide, so the error can behave like $k\rho^k$ rather than ρ^k. Finally, choosing (α_k,β_k) afresh at every step to minimise $f(\boldsymbol{\theta}_{k+1})$ turns heavy ball on a quadratic into the conjugate gradient method (§19.1), which terminates in at most P steps [13].

8.2 Nesterov Accelerated Gradient

Nesterov's accelerated gradient [14] uses the same two ingredients as heavy ball in the opposite order. It first moves along the momentum, then takes a gradient step evaluated at that look-ahead point:

$$\begin{aligned} \tilde{\boldsymbol{\theta}}_k &= \boldsymbol{\theta}_k + \beta_k\,(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1}) && \text{(momentum step)} \\ \boldsymbol{\theta}_{k+1} &= \tilde{\boldsymbol{\theta}}_k - \alpha\,\nabla f(\tilde{\boldsymbol{\theta}}_k) && \text{(gradient step at the look-ahead point)} \end{aligned} \tag{42}$$

Substituting the first line into the second,

$$\boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k - \alpha\,\nabla f\big(\boldsymbol{\theta}_k + \beta_k(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1})\big) + \beta_k\,(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1}),$$

which differs from heavy ball (40) only in where the gradient is evaluated [13]. In terms of the velocity $\mathbf{v}_{k+1} \coloneqq \boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k$, the update reads $\mathbf{v}_{k+1} = \beta_k\mathbf{v}_k - \alpha\,\nabla f(\boldsymbol{\theta}_k + \beta_k\mathbf{v}_k)$, $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \mathbf{v}_{k+1}$ [1]. Measuring the gradient where the momentum is about to carry the iterate lets the method correct an overshoot one step earlier than heavy ball.

Theorem 30 (Nesterov, smooth convex [2, 14]). *If* f *is convex and* L*-smooth with minimiser* $\boldsymbol{\theta}^*$, *then* (42) *with* $\alpha = 1/L$ *and a suitable increasing sequence* $\beta_k \to 1$ *satisfies*[15]

$$f(\boldsymbol{\theta}_k) - f^* \;=\; O\!\left(\frac{L\,\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|^2}{k^2}\right).$$

This improves the $O(1/k)$ of gradient descent (Theorem 25) to $O(1/k^2)$ at the same cost per step. The gain is fragile in practice: with α or β_k misspecified, Nesterov's method can be slower than gradient descent, or unstable [1].

Remark 31 (Oracle lower bounds). A first-order method sees f only through the values and gradients it queries. Nemirovski and Yudin showed that on L-smooth convex f, no such method can guarantee better than $f(\boldsymbol{\theta}_k) - f^* = \Omega\big(L\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|^2/k^2\big)$ [2, 15]. Nesterov's method attains this, so it is optimal, and the gap to gradient descent's $O(1/k)$ is real. With a noisy gradient oracle the lower bound drops to $\Omega(1/k)$, even for strongly convex f [13], which stochastic gradient descent already attains up to constants (§10).

8.3 Momentum in the Stochastic Setting

Replacing $\nabla f(\boldsymbol{\theta}_k)$ in (40) by a stochastic gradient $\mathbf{g}_k$ (§10) gives stochastic gradient descent with momentum:

$$\mathbf{m}_{k+1} = \beta\,\mathbf{m}_k + \mathbf{g}_k, \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha\,\mathbf{m}_{k+1}.$$

The exponential average now also averages noise. Directions in which successive stochastic gradients agree accumulate, while the zero-mean noise in each $\mathbf{g}_k$ is partly cancelled by its neighbours, so the iterates keep moving along directions that the stochastic gradients consistently report as improving [13]. In this sense momentum acts like a larger minibatch, averaging gradients over recent steps without paying to compute them [1]. Normalising the sum to a weighted mean, $\mathbf{m}_{k+1} = \beta\,\mathbf{m}_k + (1-\beta)\,\mathbf{g}_k$ (a rescaling of α by $1-\beta$), gives the first-moment estimate of Adam (§12.5).

15 Gärtner and Jaggi [15, Thm. 2.9] prove the explicit bound $2L\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|^2/(k(k+1))$ for a three-sequence variant of the method, with a short potential-function argument.

Part III.
Stochastic Optimization

9 WHY STOCHASTIC METHODS

Parts I and II assumed exact gradients of a deterministic f. In machine learning f is an average over data, and exact gradients are neither affordable nor, for the quantity we actually care about, available. This part follows Bottou, Curtis and Nocedal [13] for the theory, then turns to the practice of schedules, large batches, and adaptive methods.

9.1 Expected and Empirical Risk

Let a random *seed* ξ stand for one sample $(\mathbf{x}, \mathbf{y})$, or a set of samples, drawn from the data distribution, and let $f(\boldsymbol{\theta}; \xi)$ be the loss of the parameters $\boldsymbol{\theta}$ on ξ. For a network with outputs $\boldsymbol{\phi}$ and a convex loss $\mathcal{L}$ (§15), $f(\boldsymbol{\theta}; \xi) = \mathcal{L}(\boldsymbol{\phi}(\boldsymbol{\theta}; \mathbf{x}); \mathbf{y})$. Two objectives are built from it [13, §3.1]:

$$R(\boldsymbol{\theta}) \;=\; \mathbb{E}_{\xi}\big[f(\boldsymbol{\theta}; \xi)\big] \quad \text{(expected risk)}, \qquad R_n(\boldsymbol{\theta}) \;=\; \frac{1}{n}\sum_{i=1}^{n} f_i(\boldsymbol{\theta}), \quad f_i(\boldsymbol{\theta}) := f(\boldsymbol{\theta}; \xi_{[i]}) \quad \text{(empirical risk)}, \tag{43}$$

where $\xi_{[1]}, \dots, \xi_{[n]}$ are the n training samples. We want a small R but can only evaluate R_n. In the analysis below f (Bottou's $F(w)$) stands for either: sampling ξ uniformly from the training set makes the algorithms minimise $f = R_n$, and drawing fresh samples from the data distribution makes them minimise $f = R$ [13, §4].

9.2 Stochastic versus Batch

The two prototypical methods for minimising R_n are the *stochastic gradient* (SG) method of Robbins and Monro [16] and the batch gradient descent of §7:

$$\boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k - \alpha_k \nabla f_{i_k}(\boldsymbol{\theta}_k) \quad \text{(SG)}, \qquad \boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k - \frac{\alpha_k}{n}\sum_{i=1}^{n} \nabla f_i(\boldsymbol{\theta}_k) \quad \text{(batch)}, \tag{44}$$

with i_k drawn uniformly from $\{1, \dots, n\}$. An SG step costs one sample gradient, a batch step costs n. The SG direction $-\nabla f_{i_k}$ need not be a descent direction for R_n; it is one only in expectation, which §10 shows is enough. Bottou et al. give three reasons to prefer SG [13, §3.3].

INTUITIVE. Real data is highly redundant. If the training set were ten copies of a smaller set, every batch step would cost ten times more for exactly the same step, while SG would be unaffected.

PRACTICAL. SG makes rapid progress in the first epochs and then slows sharply. Bertsekas' example explains both halves (Figure 10): let each f_i be a one-dimensional convex quadratic with its minimiser in $[-1, 1]$, so R_n is minimised at $\theta^* = 0$. Far from $[-1, 1]$ every f_i has the same slope sign, so an SG step moves the right way whichever sample is drawn. Inside $[-1, 1]$, the *region of confusion*, the sampled slopes disagree in sign and a step is about as likely to move away from θ^* as towards it. Progress there needs either the full gradient or a shrinking step (§10).

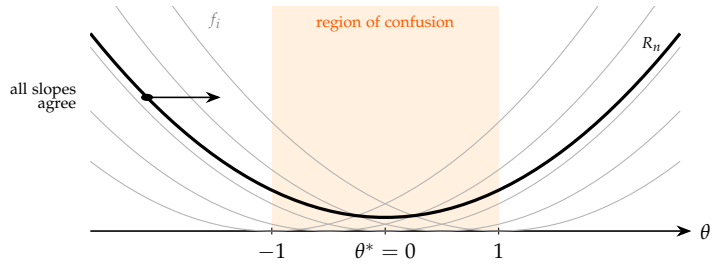


Figure 10: Bertsekas' example [13]. Component losses f_i (grey) are convex quadratics with minimisers spread over $[-1, 1]$; their average R_n (black) is minimised at $\theta^* = 0$. Far from θ^* every component slope has the same sign, so a stochastic step makes the same progress as a batch step. Inside $[-1, 1]$ the component slopes disagree, and stochastic steps no longer reliably approach θ^*.

THEORETICAL. For strongly convex R_n, batch gradient descent converges linearly (Theorem 26), so accuracy ε takes $O(\log(1/\varepsilon))$ iterations of cost n: total work proportional to $n\log(1/\varepsilon)$. SG converges only as $O(1/k)$ (Theorem 35), but neither its cost per iteration nor this rate depends on n, so its total work is proportional to $1/\varepsilon$, which is smaller for large n. Run on fresh samples, SG attains the same rate on the expected risk R, which batch methods cannot even evaluate.

9.3 Work Complexity for Large-Scale Learning

The comparison can be made precise with unlimited samples and a fixed time budget $T_{\max}$ [13, §4.4]. Let $\boldsymbol{\theta}^*$ minimise R, $\boldsymbol{\theta}_n$ minimise R_n, and $\tilde{\boldsymbol{\theta}}_n$ be what the optimizer returns. The expected risk of the returned point splits into three errors:

$$\mathbb{E}[R(\tilde{\boldsymbol{\theta}}_n)] \;=\; \underbrace{R(\boldsymbol{\theta}^*)}_{\mathcal{E}_{\text{app}}} \;+\; \underbrace{\mathbb{E}[R(\boldsymbol{\theta}_n) - R(\boldsymbol{\theta}^*)]}_{\mathcal{E}_{\text{est}}} \;+\; \underbrace{\mathbb{E}[R(\tilde{\boldsymbol{\theta}}_n) - R(\boldsymbol{\theta}_n)]}_{\mathcal{E}_{\text{opt}}}. \tag{45}$$

The approximation error is fixed by the model class; the estimation error shrinks with the number of samples n and the optimization error with the optimization accuracy ε, and both cost time. Under the most favourable statistical rate the excess error behaves like $1/n + \varepsilon$, minimised subject to the computing time $T(n,\varepsilon) \le T_{\max}$ (Table 3).

	Batch	**Stochastic**
Time to reach accuracy ε, $T(n,\varepsilon)$	$n\log(1/\varepsilon)$	$1/\varepsilon$
Best excess error within $T_{\max}$, $\mathcal{E}^*$	$\log(T_{\max})/T_{\max} + 1/T_{\max}$	$1/T_{\max}$

Table 3: Idealised work complexity of batch gradient descent and SG for large-scale learning [13, Table 4.1].

SG's time does not depend on n, so it uses every sample it can process, $n \sim T_{\max}$, and both error terms are of order $1/T_{\max}$. The batch method can only afford $n \sim T_{\max}/\log(1/\varepsilon)$ samples, which costs the extra $\log T_{\max}$. SG therefore wins as a learning algorithm, and matches the $\Omega(1/k)$ lower bound for noisy gradient oracles (Remark 31). The advantage is fragile in one direction that matters for these notes: the constants in SG's $O(1/k)$ grow with the condition number L/μ and the noise-to-curvature ratio M/μ (§10.3), which is what curvature (Part IV) and the adaptive methods of §12 try to fix [13, §4.5].

10 CONVERGENCE OF STOCHASTIC GRADIENT

10.1 The Generalised SG Algorithm and Assumptions

Replacing the gradient by a noisy estimate changes the per-step descent of §7 in one way: it holds only in expectation, and it carries an extra noise term that does not vanish at the optimum. With a fixed step the iterates therefore converge linearly, but only to a *noise floor* proportional to the step size; reaching the optimum itself requires a decreasing step and costs the linear rate. This section makes both statements precise, following Bottou et al. [13, §4].

The analysis covers a generalised SG method (Algorithm 3) in which $\mathbf{g}(\boldsymbol{\theta}_k, \xi_k)$ is any stochastic direction, for example

$$\mathbf{g}(\boldsymbol{\theta}_k,\xi_k) \;\in\; \Big\{ \nabla f(\boldsymbol{\theta}_k;\xi_k),\;\; \frac{1}{b}\sum_{i=1}^{b} \nabla f(\boldsymbol{\theta}_k;\xi_{k,i}),\;\; \mathbf{P}_k \frac{1}{b}\sum_{i=1}^{b} \nabla f(\boldsymbol{\theta}_k;\xi_{k,i}) \Big\}, \tag{46}$$

a single-sample gradient, a minibatch gradient over b samples, or a minibatch gradient rescaled by a positive definite preconditioner $\mathbf{P}_k$ (§12). We write $\mathbf{g}_k := \mathbf{g}(\boldsymbol{\theta}_k,\xi_k)$, and $\mathbb{E}_k[\cdot]$ for the expectation over ξ_k given $\boldsymbol{\theta}_k$. The seeds $\xi_0, \xi_1, \ldots$ are independent.

Algorithm 3 Generalised stochastic gradient (SG) method [13, Alg. 4.1]

input: initial iterate $\boldsymbol{\theta}_0$
for $k = 0, 1, 2, \ldots$ **do**
  generate a realisation of the seed ξ_k ▷ *one sample, a minibatch, ...*
  compute a stochastic direction $\mathbf{g}_k = \mathbf{g}(\boldsymbol{\theta}_k, \xi_k)$ ▷ *e.g. one of* (46)
  choose a step size $\alpha_k > 0$
  $\boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k - \alpha_k \mathbf{g}_k$
end for

Two assumptions carry all the results. The first is the smoothness of Definition 13, through the descent lemma (17).

Assumption 1 (Lipschitz gradient)**.** f is continuously differentiable and L-smooth: $\|\nabla f(\boldsymbol{\theta}) - \nabla f(\boldsymbol{\theta}')\| \le L\,\|\boldsymbol{\theta} - \boldsymbol{\theta}'\|$ for all $\boldsymbol{\theta}, \boldsymbol{\theta}'$.

The second restricts the first two moments of $\mathbf{g}_k$. With the variance $\mathbb{V}_k[\mathbf{g}_k] := \mathbb{E}_k\|\mathbf{g}_k\|^2 - \|\mathbb{E}_k\mathbf{g}_k\|^2$:

Assumption 2 (First and second moments [13, Assn. 4.3])**.**

(a) The iterates stay in an open set on which f is bounded below by $f_{\inf}$.
(b) There exist $\nu_G \ge \nu > 0$ such that, for all k,

$$\nabla f(\boldsymbol{\theta}_k)^\top \mathbb{E}_k[\mathbf{g}_k] \;\ge\; \nu\,\|\nabla f(\boldsymbol{\theta}_k)\|^2, \qquad \|\mathbb{E}_k[\mathbf{g}_k]\| \;\le\; \nu_G\,\|\nabla f(\boldsymbol{\theta}_k)\|.$$

(c) There exist $M, M_V \ge 0$ such that $\mathbb{V}_k[\mathbf{g}_k] \le M + M_V\,\|\nabla f(\boldsymbol{\theta}_k)\|^2$ for all k.

Condition (b) says $-\mathbf{g}_k$ is a descent direction in expectation, with a norm comparable to the gradient's. An unbiased estimate satisfies it with $\nu = \nu_G = 1$, and so does a preconditioned one whose $\mathbf{P}_k$ has eigenvalues in a fixed positive interval and is independent of ξ_k. Condition (c) lets the variance stay nonzero at a stationary point (the constant M) and grow with the gradient. Combining (b) and (c) bounds the second moment:

$$\mathbb{E}_k\|\mathbf{g}_k\|^2 \;=\; \mathbb{V}_k[\mathbf{g}_k] + \|\mathbb{E}_k\mathbf{g}_k\|^2 \;\le\; M + M_G\,\|\nabla f(\boldsymbol{\theta}_k)\|^2, \qquad M_G := M_V + \nu_G^2 \;\ge\; \nu^2. \tag{47}$$

In Bottou's notation, ν, ν_G are his μ, μ_G, and the strong-convexity (here PL) constant μ is his c.

10.2 Two Fundamental Lemmas

Lemma 32 (Expected one-step change [13, Lem. 4.2])**.** *Under Assumption 1, the iterates of Algorithm 3 satisfy, for all k,*

$$\mathbb{E}_k[f(\boldsymbol{\theta}_{k+1})] - f(\boldsymbol{\theta}_k) \;\le\; -\alpha_k\,\nabla f(\boldsymbol{\theta}_k)^\top \mathbb{E}_k[\mathbf{g}_k] \;+\; \tfrac{1}{2}\alpha_k^2 L\,\mathbb{E}_k\|\mathbf{g}_k\|^2. \tag{48}$$

Proof. Apply the descent lemma (17) with $\boldsymbol{\theta}' = \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha_k\mathbf{g}_k$:

$$f(\boldsymbol{\theta}_{k+1}) - f(\boldsymbol{\theta}_k) \;\le\; \nabla f(\boldsymbol{\theta}_k)^\top(\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k) + \tfrac{L}{2}\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k\|^2 \;=\; -\alpha_k\,\nabla f(\boldsymbol{\theta}_k)^\top \mathbf{g}_k + \tfrac{1}{2}\alpha_k^2 L\,\|\mathbf{g}_k\|^2.$$

Take $\mathbb{E}_k$ of both sides; $\boldsymbol{\theta}_k$ is fixed under $\mathbb{E}_k$, and only $\mathbf{g}_k$ is random. □

Lemma 33 (Descent minus noise [13, Lem. 4.4])**.** *Under Assumptions 1 and 2, for all k,*

$$\mathbb{E}_k[f(\boldsymbol{\theta}_{k+1})] - f(\boldsymbol{\theta}_k) \;\le\; -\nu\alpha_k\,\|\nabla f(\boldsymbol{\theta}_k)\|^2 + \tfrac{1}{2}\alpha_k^2 L\,\mathbb{E}_k\|\mathbf{g}_k\|^2 \tag{49}$$

$$\;\le\; -\big(\nu - \tfrac{1}{2}\alpha_k L M_G\big)\alpha_k\,\|\nabla f(\boldsymbol{\theta}_k)\|^2 + \tfrac{1}{2}\alpha_k^2 L M. \tag{50}$$

Proof. In (48), the first moment bound of Assumption 2(b) gives $-\alpha_k\nabla f(\boldsymbol{\theta}_k)^\top\mathbb{E}_k[\mathbf{g}_k] \le -\nu\alpha_k\|\nabla f(\boldsymbol{\theta}_k)\|^2$, which is (49). Substituting the second moment bound (47) into (49) and collecting the $\|\nabla f(\boldsymbol{\theta}_k)\|^2$ terms gives (50). □

The right-hand side of (50) has two terms, and every result below balances them. The first is the descent of gradient descent, negative for small α_k and proportional to $\|\nabla f\|^2$. The second is the price of noise, $\frac{1}{2}\alpha_k^2 LM$, which does not shrink as $\nabla f \to \mathbf{0}$, so near a minimiser it wins and the objective can increase. With $M = 0$ (no noise at the optimum) and α_k small, (50) is the per-step descent hypothesis of the convergence lemmas in §6, holding in expectation.

10.3 Objectives Satisfying PL

Bottou et al. state the next two theorems for strongly convex f, but their proofs use strong convexity only through the inequality $\|\nabla f\|^2 \ge 2\mu(f - f^*)$, which is the PL inequality (Definition 15). We state them under PL, as Theorem 27 did for gradient descent.[16]

Theorem 34 (SG with a fixed step under PL [13, Thm. 4.6])**.** *Let Assumptions 1 and 2 hold with $f_{\inf} = f^*$, and let f satisfy the PL inequality with constant μ. Run Algorithm 3 with a fixed step $\alpha_k = \alpha$, where*

$$0 \;<\; \alpha \;\le\; \frac{\nu}{LM_G}.$$

Then, with the noise floor $F_\alpha := \alpha LM/(2\mu\nu)$*, for all $k \ge 0$,*

$$\mathbb{E}[f(\boldsymbol{\theta}_k) - f^*] \;\le\; F_\alpha + (1 - \alpha\mu\nu)^k\big(f(\boldsymbol{\theta}_0) - f^* - F_\alpha\big) \;\xrightarrow{k\to\infty}\; F_\alpha. \tag{51}$$

16 Under L-smoothness and PL, $\mu \le L$: the descent lemma at $\boldsymbol{\theta}' = \boldsymbol{\theta} - \nabla f(\boldsymbol{\theta})/L$ gives $f(\boldsymbol{\theta}) - f^* \ge \|\nabla f(\boldsymbol{\theta})\|^2/(2L)$, and PL gives $f(\boldsymbol{\theta}) - f^* \le \|\nabla f(\boldsymbol{\theta})\|^2/(2\mu)$.

Proof. One step. Starting from Lemma 33:

$$\begin{aligned}
\mathbb{E}_k[f(\boldsymbol{\theta}_{k+1})] - f(\boldsymbol{\theta}_k) &\le -\big(\nu - \tfrac{1}{2}\alpha L M_G\big)\alpha \,\|\nabla f(\boldsymbol{\theta}_k)\|^2 + \tfrac{1}{2}\alpha^2 L M && (50)\\
&\le -\tfrac{1}{2}\nu\alpha \,\|\nabla f(\boldsymbol{\theta}_k)\|^2 + \tfrac{1}{2}\alpha^2 L M && (\alpha L M_G \le \nu, \text{ so } \tfrac{1}{2}\alpha L M_G \le \tfrac{1}{2}\nu)\\
&\le -\alpha\mu\nu\,\big(f(\boldsymbol{\theta}_k) - f^*\big) + \tfrac{1}{2}\alpha^2 L M && (\text{PL: } \|\nabla f\|^2 \ge 2\mu(f - f^*))
\end{aligned}$$

Recursion and fixed point. Add $f(\boldsymbol{\theta}_k) - f^*$ to both sides, take the total expectation and write $e_k := \mathbb{E}[f(\boldsymbol{\theta}_k)] - f^*$. The constant F_α is chosen so that $\alpha\mu\nu F_\alpha = \frac{1}{2}\alpha^2 L M$, so subtracting it removes the noise term:

$$\begin{aligned}
e_{k+1} &\le (1 - \alpha\mu\nu)\, e_k + \tfrac{1}{2}\alpha^2 L M\\
e_{k+1} - F_\alpha &\le (1 - \alpha\mu\nu)\, e_k + \alpha\mu\nu F_\alpha - F_\alpha \;=\; (1 - \alpha\mu\nu)\,(e_k - F_\alpha).
\end{aligned}$$

Contraction. The step condition and $M_G \ge \nu^2$ (47) give $0 < \alpha\mu\nu \le \mu\nu^2/(LM_G) \le \mu/L \le 1$, so $1 - \alpha\mu\nu \in [0, 1)$; iterating from $k = 0$ gives (51). □

With $M = 0$ this is Lemma 20 with $c = \nu\alpha/2$, in expectation; noise adds $\frac{1}{2}\alpha^2 L M$ to each step and turns convergence to f^* into convergence to $f^* + F_\alpha$. A smaller α lowers the floor $F_\alpha \propto \alpha$ but slows the contraction $1 - \alpha\mu\nu$.

HALVING THE STEP. This trade-off suggests running with a fixed step until progress stalls near $2F_\alpha$, then halving it [13, §4.2]. Each halving halves the floor but also halves $\alpha\mu\nu$, so each phase lasts about twice as long as the previous one, and the gap falls like $1/k$ (Figure 11).

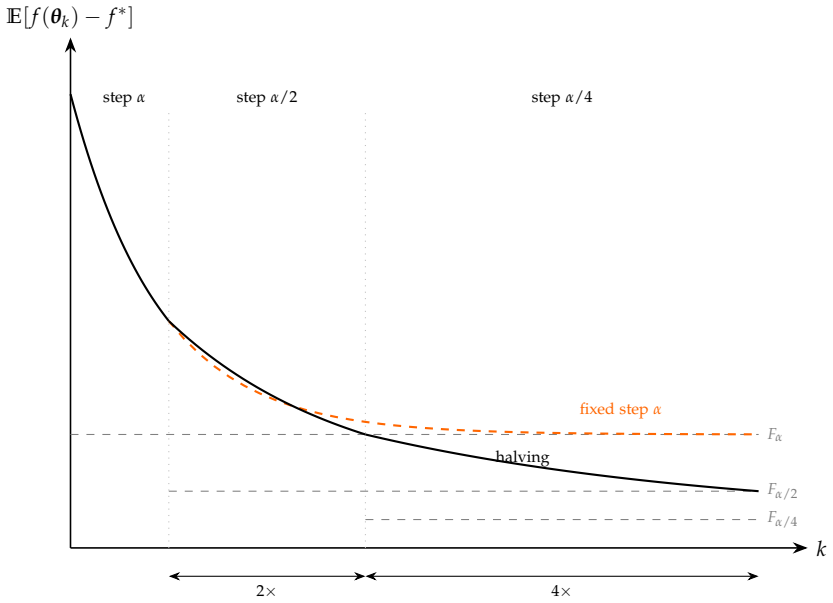


Figure 11: Fixed step versus step halving [13, Fig. 4.1]. With a fixed step α (orange, dashed) the expected gap decays geometrically to the noise floor $F_\alpha \propto \alpha$ and stalls there. Halving the step whenever the gap reaches twice the current floor (black) halves the floor, but the contraction factor $1 - \alpha\mu\nu$ also weakens, so each phase lasts twice as long as the previous one. The gap halves each time the iteration count doubles: an $O(1/k)$ rate.

DIMINISHING STEPS. The same conclusion holds for a smooth schedule. The classical requirement of Robbins and Monro [16] is

$$\sum_{k=0}^{\infty} \alpha_k \;=\; \infty, \qquad \sum_{k=0}^{\infty} \alpha_k^2 \;<\; \infty. \tag{52}$$

The first condition lets the iterates travel arbitrarily far, so they can reach a minimiser from any start; the second makes the accumulated noise $\sum_k \frac{1}{2}\alpha_k^2 L M$ finite. The step $\alpha_k \propto 1/k$ satisfies both.[17]

Theorem 35 (SG with diminishing steps under PL [13, Thm. 4.7])**.** *Under the assumptions of Theorem 34, run Algorithm 3 with $\alpha_k = \beta/(\gamma + k)$, where $\beta > 1/(\mu\nu)$ and $\gamma > 0$ is large enough that $\alpha_0 \le \nu/(LM_G)$. Then* $\mathbb{E}[f(\boldsymbol{\theta}_k) - f^*] = O(1/k)$.

The proof is an induction on the recursion $e_{k+1} \le (1 - \alpha_k\mu\nu)e_k + \frac{1}{2}\alpha_k^2 L M$ from the proof of Theorem 34. With the best choice of β the constant in $O(1/k)$ grows with the condition number L/μ and the noise-to-curvature ratio M/μ [13, §4.2].

17 Murphy [1, (8.61)] prints this condition as a ratio; (52) is the standard form.

10.4 General Nonconvex Objectives

Without PL, SG can only be shown to find points with small gradients, as for gradient descent (Theorem 24).

Theorem 36 (SG with a fixed step, nonconvex [13, Thm. 4.8]). *Under Assumptions 1 and 2, run Algorithm 3 with a fixed step $0 < \alpha \le \nu/(LM_G)$. Then for all $K \ge 1$,*

$$\frac{1}{K}\sum_{k=0}^{K-1}\mathbb{E}\|\nabla f(\boldsymbol{\theta}_k)\|^2 \;\le\; \frac{\alpha LM}{\nu} + \frac{2\big(f(\boldsymbol{\theta}_0) - f_{\inf}\big)}{K\nu\alpha} \;\xrightarrow{K\to\infty}\; \frac{\alpha LM}{\nu}. \tag{53}$$

Proof. Take the total expectation of (50) with $\alpha LM_G \le \nu$, as in the proof of Theorem 34, and sum over $k = 0, \dots, K-1$; the left-hand side telescopes and Assumption 2(a) bounds it below:

$$f_{\inf} - f(\boldsymbol{\theta}_0) \;\le\; \mathbb{E}[f(\boldsymbol{\theta}_K)] - f(\boldsymbol{\theta}_0) \;\le\; -\tfrac{1}{2}\nu\alpha\sum_{k=0}^{K-1}\mathbb{E}\|\nabla f(\boldsymbol{\theta}_k)\|^2 + \tfrac{1}{2}K\alpha^2 LM.$$

Rearranging and dividing by $\frac{1}{2}K\nu\alpha$ gives (53). □

Theorem 37 (SG with diminishing steps, nonconvex [13, Thm. 4.10]). *Under Assumptions 1 and 2, run Algorithm 3 with steps satisfying the Robbins–Monro conditions (52), and let $A_K \coloneqq \sum_{k=0}^{K-1}\alpha_k$. Then*

$$\lim_{K\to\infty}\mathbb{E}\Big[\sum_{k=0}^{K-1}\alpha_k\|\nabla f(\boldsymbol{\theta}_k)\|^2\Big] \;<\; \infty \qquad \textit{and therefore} \qquad \frac{1}{A_K}\,\mathbb{E}\Big[\sum_{k=0}^{K-1}\alpha_k\|\nabla f(\boldsymbol{\theta}_k)\|^2\Big] \;\xrightarrow{K\to\infty}\; 0.$$

With $M = 0$, Theorem 36 is Lemma 22 with $c = \nu\alpha/2$, in expectation. With noise and a fixed step, the average squared gradient approaches a floor proportional to α: SG spends more and more of its time where the gradient is small, but cannot drive it to zero. Diminishing steps remove the floor, since the summed noise $\frac{1}{2}LM\sum_k \alpha_k^2$ is finite while $A_K \to \infty$, so the weighted average of the squared gradients tends to zero [13, Thm. 4.9].

10.5 Minibatching Trade-offs

A minibatch of b independent samples reduces the noise by a factor of b. Let the per-sample gradients be unbiased with covariance

$$\boldsymbol{\Sigma}(\boldsymbol{\theta}) \;\coloneqq\; \mathbb{E}_\xi\big[(\nabla f(\boldsymbol{\theta};\xi) - \nabla f(\boldsymbol{\theta}))(\nabla f(\boldsymbol{\theta};\xi) - \nabla f(\boldsymbol{\theta}))^\top\big], \qquad \mathbb{E}_\xi\|\nabla f(\boldsymbol{\theta};\xi) - \nabla f(\boldsymbol{\theta})\|^2 = \operatorname{tr}\boldsymbol{\Sigma}(\boldsymbol{\theta}). \tag{54}$$

For $\mathbf{g} = \frac{1}{b}\sum_{i=1}^{b}\nabla f(\boldsymbol{\theta};\xi_i)$ with independent ξ_i, write $\boldsymbol{\delta}_i \coloneqq \nabla f(\boldsymbol{\theta};\xi_i) - \nabla f(\boldsymbol{\theta})$, which are independent with mean zero:

$$\begin{aligned}
\mathbb{E}\|\mathbf{g} - \nabla f(\boldsymbol{\theta})\|^2 &= \mathbb{E}\Big\|\frac{1}{b}\sum_{i=1}^{b}\boldsymbol{\delta}_i\Big\|^2 = \frac{1}{b^2}\sum_{i=1}^{b}\sum_{j=1}^{b}\mathbb{E}\,[\boldsymbol{\delta}_i^\top\boldsymbol{\delta}_j] \\
&= \frac{1}{b^2}\sum_{i=1}^{b}\mathbb{E}\|\boldsymbol{\delta}_i\|^2 \qquad\qquad (\mathbb{E}[\boldsymbol{\delta}_i^\top\boldsymbol{\delta}_j] = 0 \text{ for } i \ne j) \\
&= \frac{1}{b}\operatorname{tr}\boldsymbol{\Sigma}(\boldsymbol{\theta}), \qquad \text{and likewise} \qquad \operatorname{Cov}[\mathbf{g}] = \frac{1}{b}\,\boldsymbol{\Sigma}(\boldsymbol{\theta}).
\end{aligned} \tag{55}$$

The variance constants of Assumption 2(c) therefore become M/b and M_V/b [13, §4.2], [15, §5.6].

In theory this does not pay for the b-fold cost. With a fixed step α, Theorem 34 gives minibatch SG the floor $\alpha LM/(2\mu\nu b)$ and contraction $1 - \alpha\mu\nu$. Single-sample SG with step α/b has the same floor and contraction $1 - \alpha\mu\nu/b$, so it needs about b times as many iterations, each b times cheaper: the same total work. Minibatching would win if its step could grow with b, but the step condition $\alpha \le \nu/(LM_G)$ with $M_G \ge \nu^2$ caps it at $1/(\nu L)$ whatever b is [13, §4.2]. In practice minibatching is essential, because the b sample gradients are computed in parallel and a GPU is only fully used with sizeable batches. How far the step can grow with b before the cap binds is the subject of §11.2 and §11.4.

10.6 Noise Reduction

The noise floor comes from the constant M in Assumption 2(c). Methods that drive the variance of $\mathbf{g}_k$ to zero recover a linear rate with a fixed step [13, §5]; they are standard for convex finite sums but have not displaced SG in deep learning.

DYNAMIC SAMPLING. Grow the minibatch geometrically, $b_k = \lceil \tau^k \rceil$ with $\tau > 1$. By (55) the noise term in the proof of Theorem 34 then decays geometrically instead of staying constant, so the gap converges linearly to zero with a fixed step, while the total number of sample gradients stays $O(1/\varepsilon)$, as for SG [13, Thm. 5.3]. Practical versions grow the batch adaptively: $-\mathbf{g}_k$ is a descent direction when $\|\mathbf{g}_k - \nabla f(\boldsymbol{\theta}_k)\| \le \chi \|\mathbf{g}_k\|$ for some $\chi \in [0,1)$, and since $\mathbb{E}\|\mathbf{g}_k - \nabla f\|^2 = \operatorname{tr}\boldsymbol{\Sigma}/b_k$, the batch is enlarged whenever the sample estimate violates

$$\frac{\operatorname{tr}\widehat{\boldsymbol{\Sigma}}_k}{b_k} \;\le\; \chi^2 \,\|\mathbf{g}_k\|^2. \tag{56}$$

The ratio $\operatorname{tr}\boldsymbol{\Sigma}/\|\nabla f\|^2$ reappears in §11.4 as the noise scale that sets the critical batch size.

GRADIENT AGGREGATION: SAGA. For a finite sum R_n, past gradients can serve as a *control variate*: subtract a correlated quantity of known mean to reduce variance without adding bias. SAGA [17] stores the latest gradient $\nabla f_i(\boldsymbol{\theta}_{[i]})$ of every component, where $\boldsymbol{\theta}_{[i]}$ is the iterate at which f_i was last evaluated, draws j uniformly, uses

$$\mathbf{g}_k \;=\; \nabla f_j(\boldsymbol{\theta}_k) - \nabla f_j(\boldsymbol{\theta}_{[j]}) + \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\boldsymbol{\theta}_{[i]}), \tag{57}$$

and overwrites the stored ∇f_j with $\nabla f_j(\boldsymbol{\theta}_k)$. The last two terms have mean zero over j, so $\mathbf{g}_k$ is unbiased; as the iterates converge the first two terms cancel and the variance vanishes, giving a linear rate on strongly convex R_n. The price is storing n gradients, and the total work grows with n, so in the big-data regime of §9.3 it does not beat SG [13, §5.3].

WHY VARIANCE REDUCTION FAILS IN DEEP LEARNING. These methods assume f_i is a fixed function of $\boldsymbol{\theta}$, so that a stored $\nabla f_i(\boldsymbol{\theta}_{[i]})$ stays a valid estimate later. Batch normalisation, dropout and data augmentation all break this: the loss on sample i depends on its minibatch or changes randomly between visits, so the stored gradients stop tracking the current ones [1, §8.4.5.3].

ITERATE AVERAGING. Instead of reducing the variance of each step, one can average the iterates, $\bar{\boldsymbol{\theta}}_k = \frac{1}{k+1}\sum_{i=0}^{k} \boldsymbol{\theta}_i$. Polyak and Juditsky show that this Polyak–Ruppert average attains the best possible asymptotic rate among SG methods, matching methods that use the Hessian [18]; deep-learning practice uses the same idea as an exponential moving average of the weights, or as stochastic weight averaging [1, §8.4.4].

11 LEARNING-RATE SCHEDULES AND LARGE-BATCH TRAINING

The theory of §10 fixes how the step must evolve; this section covers what practice does with it, and how the step should change with the batch size.

11.1 Schedules

With a fixed step SG stalls at a noise floor $F_\alpha \propto \alpha$ (Theorem 34), and only a step that shrinks to zero, while still summing to infinity, converges (52). A *learning-rate schedule* fixes α_k in advance. The common choices are [1, §8.4.3]

$$\begin{aligned} &\alpha_k = \alpha_i \quad \text{for } k_i \le k < k_{i+1} && \text{(piecewise constant)} \\ &\alpha_k = \alpha_0\, e^{-\lambda k} && \text{(exponential decay)} \\ &\alpha_k = \alpha_0\,(\beta k + 1)^{-a} && \text{(polynomial decay)} \end{aligned} \tag{58}$$

Step decay sets $\alpha_i = \alpha_0 \gamma^i$ at fixed milestones, or lowers the step when the loss plateaus; it is the halving strategy of Figure 11. Exponential decay is usually too fast: $\sum_k \alpha_0 e^{-\lambda k} = \alpha_0/(1-e^{-\lambda})$ is finite, violating the first Robbins–Monro condition, so the iterates can travel only a bounded distance. Polynomial decay satisfies $\sum_k \alpha_k = \infty$ for $a \le 1$ and $\sum_k \alpha_k^2 < \infty$ for $a > \frac{1}{2}$; the common square-root schedule $a = \frac{1}{2}$ meets the first condition but not the second.

Deep-learning practice usually trains for a fixed budget of K steps rather than to convergence, and combines a short linear *warmup* (§11.3) with a cosine decay to a small final value [19, 20]:

$$\alpha_k \;=\; \begin{cases} \alpha_{\max}\, k/K_w, & k \le K_w, \\ \alpha_{\min} + \tfrac{1}{2}(\alpha_{\max} - \alpha_{\min})\Big(1 + \cos\Big(\pi\,\dfrac{k - K_w}{K - K_w}\Big)\Big), & K_w < k \le K. \end{cases} \tag{59}$$

The cosine stays near $\alpha_{\max}$ for a long time, then falls quickly and flattens near $\alpha_{\min}$. *Warm restarts* (SGDR) repeat the cosine phase, resetting to $\alpha_{\max}$ and lengthening each phase [19]. The peak step is often chosen by an *LR range test*: train briefly with α increasing geometrically, and pick a value slightly below the one with the lowest loss [1, §8.4.3].

11.2 The Linear Scaling Rule

Data-parallel training spreads each minibatch over many workers, so the batch size grows with the hardware. Goyal et al. [21] found a simple rule for keeping the same step-by-step behaviour:

When the minibatch size is multiplied by q, multiply the learning rate by q.

To see why, compare q steps of minibatch SG with batch size b and step α against one step with batch size qb and step $\hat{\alpha}$, both from $\boldsymbol{\theta}_k$ and using the same samples $\mathcal{S}_k, \dots, \mathcal{S}_{k+q-1}$:

$$\begin{aligned} \boldsymbol{\theta}_{k+q} &= \boldsymbol{\theta}_k - \alpha \frac{1}{b} \sum_{j=0}^{q-1} \sum_{i \in \mathcal{S}_{k+j}} \nabla f_i(\boldsymbol{\theta}_{k+j}) \quad (q \text{ small steps}) \\ \hat{\boldsymbol{\theta}}_{k+1} &= \boldsymbol{\theta}_k - \hat{\alpha} \frac{1}{qb} \sum_{j=0}^{q-1} \sum_{i \in \mathcal{S}_{k+j}} \nabla f_i(\boldsymbol{\theta}_k) \quad (\text{one large step}) \end{aligned} \tag{60}$$

The two differ only in where the gradients are evaluated. If the parameters change little over q small steps, so that $\nabla f_i(\boldsymbol{\theta}_{k+j}) \approx \nabla f_i(\boldsymbol{\theta}_k)$, then $\hat{\boldsymbol{\theta}}_{k+1} \approx \boldsymbol{\theta}_{k+q}$ exactly when $\hat{\alpha} = q\alpha$. Scaled this way, small- and large-batch runs follow nearly identical training curves as a function of samples seen. The approximation fails in two places: early in training, when the parameters change rapidly (addressed by warmup), and beyond a task-dependent batch size, where accuracy degrades quickly. The alternative *square-root* rule $\hat{\alpha} = \sqrt{q}\,\alpha$, which matches the step to the $1/\sqrt{q}$ reduction in the gradient's standard deviation (55), performs clearly worse [21].

This seems at odds with §10.5, where the step could not grow with b beyond $1/(\nu L)$. The two agree once the noise level is taken into account: at small batches the step is limited by noise and can grow linearly with b, until it reaches the curvature limit set by L, after which larger batches no longer help. §11.4 makes this precise.

11.3 Warmup

Warmup uses smaller steps at the start of training, where the linear scaling approximation fails. Training at the small-batch rate α for a few epochs and then jumping to $q\alpha$ makes the training error spike at the switch, whereas increasing the step gradually from α to $q\alpha$, as in (59), lets the large-batch run match both the training curve and the validation error of the small-batch run: the difficulty of large batches is one of optimization, not generalization [21]. One intuition is that training starts in a poorly conditioned region where a large step bounces around, and small steps first reach better-conditioned regions where large steps are safe [1, §8.4.3].

11.4 Critical Batch Size

How large can the batch be before larger batches stop paying? McCandlish et al. [22] answer with a short calculation at a single point $\boldsymbol{\theta}$. Write $\mathbf{H}$ for the Hessian at $\boldsymbol{\theta}$ and $\mathbf{g}$ for a minibatch gradient of size b, so that $\mathbb{E}[\mathbf{g}] = \nabla f$ and $\operatorname{Cov}[\mathbf{g}] = \boldsymbol{\Sigma}/b$ by (55). The second-order Taylor expansion (5) along the step $-\alpha\mathbf{g}$, in expectation over the batch, is

$$\begin{aligned} \mathbb{E}\big[f(\boldsymbol{\theta} - \alpha\mathbf{g})\big] &\approx f(\boldsymbol{\theta}) - \alpha\, \nabla f^{\top} \mathbb{E}[\mathbf{g}] + \tfrac{1}{2}\alpha^2\, \mathbb{E}\big[\mathbf{g}^{\top}\mathbf{H}\mathbf{g}\big] && \text{(Taylor, then expectation)} \\ &= f(\boldsymbol{\theta}) - \alpha\, \|\nabla f\|^2 + \tfrac{1}{2}\alpha^2 \operatorname{tr}\big(\mathbf{H}\, \mathbb{E}[\mathbf{g}\mathbf{g}^{\top}]\big) && (\mathbf{g}^{\top}\mathbf{H}\mathbf{g} = \operatorname{tr}(\mathbf{H}\mathbf{g}\mathbf{g}^{\top})) \\ &= f(\boldsymbol{\theta}) - \alpha\, \|\nabla f\|^2 + \tfrac{1}{2}\alpha^2 \Big(\nabla f^{\top}\mathbf{H}\nabla f + \frac{\operatorname{tr}(\mathbf{H}\boldsymbol{\Sigma})}{b}\Big) && (\mathbb{E}[\mathbf{g}\mathbf{g}^{\top}] = \nabla f\, \nabla f^{\top} + \boldsymbol{\Sigma}/b). \end{aligned} \tag{61}$$

This is a quadratic in α. Setting its derivative to zero gives the best step and the resulting expected decrease:

$$\alpha_{\text{opt}}(b) \;=\; \frac{\|\nabla f\|^2}{\nabla f^{\top}\mathbf{H}\nabla f + \operatorname{tr}(\mathbf{H}\boldsymbol{\Sigma})/b} \;=\; \frac{\alpha_{\max}}{1 + B_{\text{noise}}/b}, \qquad \Delta f_{\text{opt}}(b) \;=\; \frac{\Delta f_{\max}}{1 + B_{\text{noise}}/b}, \tag{62}$$

where $\alpha_{\max} = \|\nabla f\|^2/\nabla f^{\top}\mathbf{H}\nabla f$ and $\Delta f_{\max} = \tfrac{1}{2}\|\nabla f\|^4/\nabla f^{\top}\mathbf{H}\nabla f$ are the step and decrease with the exact gradient ($\alpha_{\max}$ is the exact line-search step of §7.3), and the *gradient noise scale* is

$$B_{\text{noise}} \;:=\; \frac{\operatorname{tr}(\mathbf{H}\boldsymbol{\Sigma})}{\nabla f^{\top}\mathbf{H}\nabla f}, \qquad B_{\text{simple}} \;:=\; \frac{\operatorname{tr}\boldsymbol{\Sigma}}{\|\nabla f\|^2} \quad (\text{the value when } \mathbf{H} \propto \mathbf{I}). \tag{63}$$

Two regimes follow from (62).

- *Small batches*, $b \ll B_{\text{noise}}$: $\alpha_{\text{opt}} \approx \alpha_{\max}\, b/B_{\text{noise}}$ and $\Delta f_{\text{opt}} \approx \Delta f_{\max}\, b/B_{\text{noise}}$. The best step grows linearly with b, which is the linear scaling rule of §11.2, and so does the progress per step: doubling the batch halves the number of steps, at no extra total computation.

- *Large batches*, $b \gg B_{\text{noise}}$: $\alpha_{\text{opt}} \approx \alpha_{\max}$ and $\Delta f_{\text{opt}} \approx \Delta f_{\max}$. The step has reached the curvature limit, and larger batches only waste computation.

B_{simple} is cheap to estimate and is the ratio in the dynamic-sampling test (56): by (55), $\mathbb{E}\|\mathbf{g} - \nabla f\|^2/\|\nabla f\|^2 = B_{\text{simple}}/b$, so a batch well above B_{simple} estimates the gradient with an error much smaller than the gradient itself.

Averaging this per-step picture over a training run gives a trade-off between the number of optimization steps S and the number of training examples processed $E = bS$ to reach a given loss [22]:

$$\Big(\frac{S}{S_{\min}} - 1\Big)\Big(\frac{E}{E_{\min}} - 1\Big) \;=\; 1, \qquad B_{\text{crit}} \;:=\; \frac{E_{\min}}{S_{\min}} \;\approx\; B_{\text{noise}}, \tag{64}$$

where $S_{\min}$ is the fewest steps (at very large batch) and $E_{\min}$ the fewest examples (at very small batch); training at $b = B_{\text{crit}}$ costs twice the minimum of each. Empirically B_{simple} predicts the measured critical batch size to within an order of magnitude across very different tasks. Because $\|\nabla f\|$ shrinks during training while $\operatorname{tr}\boldsymbol{\Sigma}$ stays roughly constant, the critical batch size grows as training proceeds. The calculation is local and assumes a well-tuned step, so it is a guide rather than a theorem. The critical batch size also depends on the optimizer; for the second-order method SOAP see §20.4.

12 ADAPTIVE DIAGONAL METHODS

A single learning rate treats every direction alike, which is what makes SG's constants grow with the condition number (§9.3). The methods of this section adapt the step to the observed gradients: first with one scalar (clipping), then with one step size per coordinate.

12.1 Gradient Clipping

The simplest adaptive step rescales the whole gradient by a scalar set from its norm. *Clipping by norm* with threshold $c > 0$ takes

$$\boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k - \alpha\, \min\Big(1, \frac{c}{\|\mathbf{g}_k\|}\Big)\, \mathbf{g}_k, \tag{65}$$

which is plain SG while $\|\mathbf{g}_k\| \le c$ and caps the step length at αc otherwise. It was introduced for recurrent networks, where the backward chain of BPTT makes gradients occasionally explode (§2.2 of the SOFO notes), and clipping keeps such steps bounded [23].

WHY CLIPPING SPEEDS UP TRAINING. The analyses of §10 use one global smoothness constant L (Assumption 1), and the safe step is $1/L$ (Lemma 23). Along real training trajectories, especially of LSTM language models, Zhang et al. find instead that the local smoothness varies widely and grows with the gradient norm [23]. They model this as (L_0, L_1)*-smoothness*,

$$\|\nabla^2 f(\boldsymbol{\theta})\|_{\text{op}} \;\le\; L_0 + L_1\, \|\nabla f(\boldsymbol{\theta})\|, \tag{66}$$

which is weaker than L-smoothness: a univariate polynomial of degree three or more satisfies it, although its curvature is unbounded. The curvature near $\boldsymbol{\theta}$ is now about $L_0 + L_1\|\nabla f(\boldsymbol{\theta})\|$, so the locally safe step is about $1/(L_0 + L_1\|\nabla f(\boldsymbol{\theta})\|)$, which is short where the gradient is large and long where it is small. Clipping produces exactly this step. With $\alpha = 1/L_0$ and $c = L_0/L_1$ in (65), and $\mathbf{g}$ the current gradient,

$$\alpha\, \min\Big(1, \frac{c}{\|\mathbf{g}\|}\Big) = \min\Big(\frac{1}{L_0}, \frac{1}{L_1\|\mathbf{g}\|}\Big) = \frac{1}{\max(L_0,\, L_1\|\mathbf{g}\|)} \qquad \text{(substitute } \alpha, c\text{)}$$

$$\frac{1}{L_0 + L_1\|\mathbf{g}\|} \le \frac{1}{\max(L_0,\, L_1\|\mathbf{g}\|)} \le \frac{2}{L_0 + L_1\|\mathbf{g}\|} \qquad \big(\tfrac{1}{2}(a+b) \le \max(a,b) \le a + b \text{ for } a, b \ge 0\big)$$

so the clipped step is the locally safe step up to a factor of two. A fixed step has no such freedom: it must be safe in the sharpest region the iterates can reach, $\alpha \lesssim 1/(L_0 + L_1 G)$ with G the largest gradient norm on the initial sublevel set, and it is then far too short wherever the gradient is small. Making this precise, with the curvature bounded along the whole step rather than only at its start, Zhang et al. show that fixed-step gradient descent needs a number of iterations growing with $L_1 G$, while the leading term for clipped gradient descent depends on L_0 alone, so clipping can be arbitrarily faster [23, Thms. 3, 4].

Up to the same factor of two, clipping is normalised gradient descent, $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta\, \mathbf{g}_k/(\|\mathbf{g}_k\| + \beta)$ [23]: a single scalar normalisation, which RMSProp and Adam below apply per coordinate.

12.2 Preconditioned SGD

Preconditioned SGD rescales each stochastic gradient by a positive definite matrix $\mathbf{P}_k$ [1, §8.4.6]:

$$\boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k - \alpha_k \, \mathbf{P}_k \, \mathbf{g}_k, \tag{67}$$

which is the local-model step (28) with curvature matrix $\mathbf{B}_k = (\alpha_k \mathbf{P}_k)^{-1}$. A full $\mathbf{P}_k$ needs $O(P^2)$ memory, and the noise in $\mathbf{g}_k$ makes the Hessian hard to estimate, so the methods below restrict $\mathbf{P}_k$ to be diagonal and build it from running statistics of the gradient itself. A diagonal $\mathbf{P}_k$ rescales each coordinate but cannot undo correlations between parameters, so it does not fix ill-conditioning in general [1, §8.4.6.5]; Bottou et al. call these methods diagonal rescaling *without* curvature [13, §6.5], and §21 relates them to curvature. In this section all products, squares, square roots and divisions of vectors are elementwise.

12.3 Adagrad

Adagrad [24] divides each coordinate by the root of its accumulated squared gradients:

$$\mathbf{s}_k \;=\; \sum_{j=0}^{k} \mathbf{g}_j \odot \mathbf{g}_j, \qquad \boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k - \alpha \, \frac{\mathbf{g}_k}{\sqrt{\mathbf{s}_k} + \epsilon}, \tag{68}$$

with a small $\epsilon > 0$ to avoid division by zero. If the gradient magnitudes in coordinate i are roughly stable, $s_{k,i}$ grows linearly in k and the per-coordinate step decays like $1/\sqrt{k}$. Adagrad was designed for convex problems with sparse features: a coordinate whose gradient is usually zero accumulates a small $s_{k,i}$ and takes large steps whenever its feature appears. In deep learning the accumulated sum grows too quickly and the steps become too small too early [13, §6.5].

Adagrad has a full-matrix form that keeps the outer products instead of their diagonal [24]:

$$\mathbf{S}_k \;=\; \sum_{j=0}^{k} \mathbf{g}_j \mathbf{g}_j^{\top}, \qquad \boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k - \alpha \, \mathbf{S}_k^{-1/2} \, \mathbf{g}_k, \tag{69}$$

with $\mathbf{S}_k^{-1/2}$ the inverse PSD square root (Definition 7), regularised by adding $\epsilon \mathbf{I}$ in practice. It adapts to correlations between parameters but needs $O(P^2)$ memory and an $O(P^3)$ matrix root; Shampoo (§20) approximates it per layer with a Kronecker product.

12.4 RMSProp and AdaDelta

RMSProp [25] replaces Adagrad's sum by an exponentially weighted moving average (EWMA), so old gradients are forgotten and the step does not decay on its own:

$$\mathbf{v}_k \;=\; \beta \, \mathbf{v}_{k-1} + (1-\beta) \, \mathbf{g}_k \odot \mathbf{g}_k, \qquad \boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k - \alpha_k \, \frac{\mathbf{g}_k}{\sqrt{\mathbf{v}_k} + \epsilon}. \tag{70}$$

Now $\sqrt{v_{k,i}}$ is approximately the root mean square of recent gradients in coordinate i, so every rescaled coordinate has second moment close to one: RMSProp makes equal progress along each axis rather than approximating curvature [13, §6.5]. Dividing a coordinate by its own magnitude is close to taking its sign, as in its predecessor RPROP [1, §8.4.6.2], a connection Part V makes exact. AdaDelta multiplies the RMSProp step by the root of an EWMA of past squared updates, so numerator and denominator have the same units and the global step can in principle be set to one [1, §8.4.6.2].

12.5 Adam

Adam [26] combines momentum (§8) with RMSProp scaling: it keeps EWMAs of both the gradient and the squared gradient, corrects them for their zero initialisation, and divides one by the root of the other. Algorithm 4 states it together with AdamW (§12.6), which differs only in the decay term of the last line. Following the paper, iterations start at $k = 1$ so that the bias correction reads $1 - \beta^k$.

Algorithm 4 Adam [26] and AdamW [20]

input: $\boldsymbol{\theta}_0$, schedule α_k, $\beta_1, \beta_2 \in [0,1)$, $\epsilon > 0$, weight decay $\lambda \geq 0$ ▷ *defaults* 0.9, 0.999, 10^{-8}
$\mathbf{m}_0 \leftarrow \mathbf{0}$, $\mathbf{v}_0 \leftarrow \mathbf{0}$
for $k = 1, 2, \ldots$ **do**
  $\mathbf{g}_k \leftarrow \nabla f(\boldsymbol{\theta}_{k-1}; \xi_k)$ ▷ *Adam with ℓ_2: add $\lambda\boldsymbol{\theta}_{k-1}$ here instead*
  $\mathbf{m}_k \leftarrow \beta_1 \mathbf{m}_{k-1} + (1-\beta_1)\,\mathbf{g}_k$ ▷ *EWMA of gradients*
  $\mathbf{v}_k \leftarrow \beta_2 \mathbf{v}_{k-1} + (1-\beta_2)\,\mathbf{g}_k \odot \mathbf{g}_k$ ▷ *EWMA of squared gradients*
  $\hat{\mathbf{m}}_k \leftarrow \mathbf{m}_k / (1-\beta_1^k)$, $\hat{\mathbf{v}}_k \leftarrow \mathbf{v}_k / (1-\beta_2^k)$ ▷ *bias correction*
  $\boldsymbol{\theta}_k \leftarrow \boldsymbol{\theta}_{k-1} - \alpha_k \left(\hat{\mathbf{m}}_k / \left(\sqrt{\hat{\mathbf{v}}_k} + \epsilon \right) + \boxed{\lambda\, \boldsymbol{\theta}_{k-1}} \right)$ ▷ *boxed term: AdamW only*
end for

BIAS CORRECTION. Both EWMAs start at zero, so early on they underestimate the moments they track. The correction factor follows from unrolling the recursion.

Proposition 38 (Bias correction [26, §3]). *If the squared gradients are stationary, $\mathbb{E}[\mathbf{g}_j \odot \mathbf{g}_j] = \mathbb{E}[\mathbf{g} \odot \mathbf{g}]$ for all j, then $\mathbb{E}[\mathbf{v}_k] = (1-\beta_2^k)\,\mathbb{E}[\mathbf{g} \odot \mathbf{g}]$, so $\hat{\mathbf{v}}_k = \mathbf{v}_k/(1-\beta_2^k)$ is unbiased. The same holds for $\mathbf{m}_k$ and $\hat{\mathbf{m}}_k$ with β_1.*

Proof. Unrolling $\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1-\beta_2)\,\mathbf{g}_k \odot \mathbf{g}_k$ from $\mathbf{v}_0 = \mathbf{0}$ and taking expectations:

$$\begin{aligned}
\mathbb{E}[\mathbf{v}_k] &= (1-\beta_2) \sum_{j=1}^{k} \beta_2^{k-j}\, \mathbb{E}[\mathbf{g}_j \odot \mathbf{g}_j] && \text{(unroll, linearity)} \\
&= \mathbb{E}[\mathbf{g} \odot \mathbf{g}]\,(1-\beta_2) \sum_{i=0}^{k-1} \beta_2^{i} && \text{(stationarity, } i = k-j\text{)} \\
&= \mathbb{E}[\mathbf{g} \odot \mathbf{g}]\,(1-\beta_2^k) && \text{(geometric sum } \textstyle\sum_{i<k} \beta_2^i = \frac{1-\beta_2^k}{1-\beta_2}\text{)}
\end{aligned}$$

The argument for $\mathbf{m}_k$ is identical. □

If the moments drift slowly the stationarity error is small, because the EWMA puts little weight on old gradients [26].[18] The correction matters most when β_2 is close to one: with $\beta_2 = 0.999$ and $k = 10$, $1-\beta_2^k \approx 0.01$, so the uncorrected $\mathbf{v}_k$ is about 100 times too small and the first steps about 10 times too large, which makes early training unstable [26, §6.4].

WHY IT WORKS. With $\epsilon = 0$ and a constant α, the step in each coordinate is $\Delta_k = \alpha\, \hat{m}_{k,i} / \sqrt{\hat{v}_{k,i}}$. Except under extreme sparsity, $|\hat{m}_{k,i}/\sqrt{\hat{v}_{k,i}}| \lesssim 1$, since $|\mathbb{E}[g]| \leq \sqrt{\mathbb{E}[g^2]}$, so $|\Delta_k| \lesssim \alpha$ whatever the gradient's scale: Adam steps inside a trust region of radius about α (§14), which makes the right order of α easy to guess [26, §2.1].

- *Signal-to-noise ratio.* $\hat{m}_{k,i}/\sqrt{\hat{v}_{k,i}}$ behaves like a signal-to-noise ratio: when the gradient's sign is uncertain it is small and the step shrinks, which near an optimum is a form of automatic annealing.
- *Scale invariance.* Rescaling the gradient by $c > 0$ scales $\hat{\mathbf{m}}_k$ by c and $\hat{\mathbf{v}}_k$ by c^2, leaving the step unchanged.
- *Relation to RMSProp and Adagrad.* With $\beta_1 = 0$ and no bias correction, Adam is RMSProp (70). With $\beta_1 = 0$, $\beta_2 \to 1$ and the annealed step $\alpha_k = \alpha/\sqrt{k}$, the bias-corrected $\hat{\mathbf{v}}_k \to \frac{1}{k}\sum_{j\leq k} \mathbf{g}_j \odot \mathbf{g}_j$ and the update becomes
$$\frac{\alpha}{\sqrt{k}} \frac{\mathbf{g}_k}{\sqrt{\frac{1}{k}\sum_{j=1}^{k} \mathbf{g}_j \odot \mathbf{g}_j}} = \alpha \frac{\mathbf{g}_k}{\sqrt{\sum_{j=1}^{k} \mathbf{g}_j \odot \mathbf{g}_j}},$$
which is Adagrad (68). The limit exists only because of the bias correction: without it $\mathbf{v}_k \to \mathbf{0}$ as $\beta_2 \to 1$.
- *Diagonal empirical Fisher.* For a log-likelihood loss, $\mathbf{v}_k$ estimates the diagonal of the empirical Fisher, so Adam preconditions with its inverse square root [26, §5]. The empirical Fisher tracks gradient noise rather than curvature (§16.6), and the square root makes Adam close to sign descent (§23.2); neither makes it a curvature method.

Adam's convergence guarantees are weaker than its practice suggests: its original regret analysis contains an error, and the EWMA can make it fail to converge even on convex problems; AMSGrad and ADOPT modify the second-moment update to restore convergence [1, §8.4.6.4]. For a weight matrix Adam's second moment is as large as the weights; Adafactor [27] replaces it by a rank-one factorisation, defined in §20.4.

READINGS OF ADAM. Later sections read Adam as

- an implicit trust region of radius about α (above, and §14.5);
- preconditioning by the diagonal empirical Fisher, which tracks gradient noise rather than curvature (§16.6);
- diagonal rescaling without curvature (§21);
- without the moving averages, sign descent, steepest descent under ℓ_∞ (§23.2).

18 The Adam paper writes β_1 at this point where β_2 is meant.

12.6 AdamW: Decoupled Weight Decay

Weight decay shrinks the weights by a factor $1-\lambda$ every step, $\boldsymbol{\theta}_{k+1} = (1-\lambda)\boldsymbol{\theta}_k - \alpha\nabla f_k(\boldsymbol{\theta}_k)$, where f_k is the minibatch loss at step k [20]. It is usually implemented as an ℓ_2 penalty $\frac{\lambda'}{2}\|\boldsymbol{\theta}\|^2$ added to the loss. For SGD the two coincide with $\lambda' = \lambda/\alpha$, since SGD on the penalised loss subtracts $\alpha\lambda'\boldsymbol{\theta}_k$ [20, Prop. 1]; even then the best λ' moves whenever α does. For a preconditioned method $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha\mathbf{P}_k\nabla f_k$ the ℓ_2 penalty subtracts $\alpha\lambda'\mathbf{P}_k\boldsymbol{\theta}_k$, which equals the decay $\lambda\boldsymbol{\theta}_k$ for every $\boldsymbol{\theta}_k$ only if $\mathbf{P}_k$ is a multiple of the identity, so no λ' reproduces weight decay [20, Prop. 2].

With an ℓ_2 penalty, Adam rescales the penalty's gradient along with the loss gradient, so weights whose gradients have historically been large get a small $\mathbf{P}_k$ entry and are regularised *less* than the others. *Decoupled* weight decay applies the decay outside the adaptive rescaling, so every weight decays at the same rate; this is AdamW, the boxed term in Algorithm 4.[19] Decoupling makes the best values of α and λ roughly independent, where with ℓ_2 the good settings lie along a diagonal of the (α, λ) grid, and it closes most of Adam's generalisation gap to SGD with momentum on image classification [20].

DECAY AS A PROXIMAL STEP. For a convex regulariser h, *proximal gradient descent* minimises $f + h$ by a gradient step on f followed by the proximal step $\operatorname{prox}_{\alpha h}(\mathbf{v}) \coloneqq \arg\min_{\mathbf{u}}\{h(\mathbf{u}) + \frac{1}{2\alpha}\|\mathbf{u}-\mathbf{v}\|^2\}$ [1, §8.6]. For $h(\mathbf{u}) = \frac{\lambda}{2}\|\mathbf{u}\|^2$ the proximal step is a shrinkage:

$$\begin{aligned} \mathbf{0} &= \lambda\mathbf{u} + \tfrac{1}{\alpha}(\mathbf{u}-\mathbf{v}) && \text{(stationarity of the strongly convex objective)} \\ \mathbf{u} &= \frac{\mathbf{v}}{1+\alpha\lambda} \;=\; (1-\alpha\lambda)\,\mathbf{v} + O(\alpha^2\lambda^2)\,\mathbf{v}. \end{aligned}$$

To first order, proximal gradient descent on $f + \frac{\lambda}{2}\|\boldsymbol{\theta}\|^2$ therefore takes a gradient step on f alone and then decays the weights by $1-\alpha\lambda$. AdamW replaces the gradient step by an Adam step and keeps the decay, so the regulariser never passes through the preconditioner. Part V returns to this: under any non-Euclidean norm, steepest descent and the ℓ_2 penalty again disagree, so decoupled decay is needed there too.

19 Algorithm 4 follows the PyTorch implementation, whose per-step decay is $\alpha_k\lambda$; the paper's decay does not scale with the base step [20, Alg. 2]. Muon (§23.5) uses the PyTorch form.

Part IV.
Second-Order Methods and Curvature

This part first sets out the curvature (Sections 13–18), then how to use it at scale (Sections 19–21). A curvature step needs a $P \times P$ solve, and there are two ways around it: approximate the matrix by one that is cheap to invert, a diagonal (§21) or per-layer Kronecker factors (§20), or keep the exact curvature and minimise the quadratic model only over a low-dimensional subspace. Conjugate gradient builds that subspace from the gradient, one sequential curvature-vector product per dimension (§19); SOFO draws it at random, with forward-mode products computed in parallel (§3 and §4 of the SOFO notes).

13 Newton's Method

Set $\mathbf{B}_k = \mathbf{H}_k := \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}_k)$, the Hessian of f at the current iterate. The local model (27) now coincides with the second-order Taylor expansion of f around $\boldsymbol{\theta}_k$:

$$M_k(\mathbf{d}) \;=\; f(\boldsymbol{\theta}_k) + \nabla f(\boldsymbol{\theta}_k)^\top \mathbf{d} + \tfrac{1}{2}\,\mathbf{d}^\top \mathbf{H}_k\,\mathbf{d}.$$

Unlike first-order methods, no artificial penalty is needed — the objective's own curvature provides the quadratic term. From (28):

$$\boldsymbol{\theta}_{k+1} \;\leftarrow\; \boldsymbol{\theta}_k \;-\; \mathbf{H}_k^{-1}\,\nabla f(\boldsymbol{\theta}_k). \tag{71}$$

The second-order model determines both the direction *and* the step size ($\alpha_k = 1$). Geometrically, $\mathbf{H}_k^{-1}$ "undoes" the local curvature, transforming elongated elliptical contours into circles and eliminating the zig-zagging that hobbles gradient descent at large κ.

13.1 The Newton Decrement

The natural measure of progress for Newton's method is not the Euclidean gradient norm but the Hessian-weighted version — the quantity used by the second-order subspace methods of §3.3–§4.

Definition 19 (Newton decrement)**.** The *(squared) Newton decrement* of f at $\boldsymbol{\theta}$ is

$$\|\nabla f(\boldsymbol{\theta})\|^2_{\mathbf{H}_{\boldsymbol{\theta}}^{-1}} \;:=\; \nabla f(\boldsymbol{\theta})^\top\, \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta})^{-1}\, \nabla f(\boldsymbol{\theta}). \tag{72}$$

It admits two interpretations:

(1) *Newton-step length in the Hessian norm:* $\|\nabla f\|^2_{\mathbf{H}^{-1}} = \mathbf{d}_k^\top \mathbf{H}_k\,\mathbf{d}_k = \|\mathbf{d}_k\|^2_{\mathbf{H}_k}$, where $\mathbf{d}_k = -\mathbf{H}_k^{-1}\nabla f$ is the Newton step.

(2) *Twice the predicted decrease:* $f(\boldsymbol{\theta}_k) - M_k(\mathbf{d}_k) = \tfrac{1}{2}\|\nabla f\|^2_{\mathbf{H}_k^{-1}}$. The Newton step's effect on the local quadratic model is determined entirely by this quantity — it is the affine-invariant analogue of $\|\nabla f\|^2/(2L)$ used in gradient-descent analyses.

The sketched version $\mathbf{C}^\top(\boldsymbol{\Theta}^\top \mathbf{G}\boldsymbol{\Theta})^{+}\mathbf{C}$, formed from the sketched curvature and gradient, is the r-dimensional analogue of (72) — the quantity SOFO actually computes.

13.2 Local Quadratic Convergence

Newton's headline property is that, sufficiently close to a non-degenerate minimum, the error squares each iteration.

Theorem 39 (Local quadratic convergence)**.** *Suppose $\boldsymbol{\theta}^*$ is a minimiser with $\mathbf{H}_* := \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}^*) \succeq \mu\,\mathbf{I}$ for some $\mu > 0$, and the Hessian is M-Lipschitz on a neighbourhood $\mathcal{N}$ of $\boldsymbol{\theta}^*$:*

$$\|\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}) - \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}')\|_{\mathrm{op}} \;\le\; M\,\|\boldsymbol{\theta} - \boldsymbol{\theta}'\| \qquad \forall\, \boldsymbol{\theta}, \boldsymbol{\theta}' \in \mathcal{N}.$$

If $\boldsymbol{\theta}_0 \in \mathcal{N}$ with $\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\| < \mu/M$, then the Newton iterates (71) stay in $\mathcal{N}$ and satisfy

$$\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}^*\| \;\le\; \frac{M}{2(\mu - M\,\|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|)}\,\|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|^2. \tag{73}$$

In particular, once $\|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\| < \mu/(2M)$, $\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}^*\| \le (M/\mu)\,\|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|^2$: the error squares each step.

Proof. Write $\mathbf{e}_k := \boldsymbol{\theta}_k - \boldsymbol{\theta}^*$. By the fundamental theorem of calculus applied componentwise to ∇f along the segment from $\boldsymbol{\theta}^*$ to $\boldsymbol{\theta}_k$, and using $\nabla f(\boldsymbol{\theta}^*) = \mathbf{0}$:

$$\nabla f(\boldsymbol{\theta}_k) \;=\; \int_0^1 \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}^* + t\,\mathbf{e}_k)\,\mathbf{e}_k\,dt \;=:\; \bar{\mathbf{H}}_k\,\mathbf{e}_k, \qquad \bar{\mathbf{H}}_k := \int_0^1 \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}^* + t\,\mathbf{e}_k)\,dt.$$

Substituting into the Newton update:

$$\mathbf{e}_{k+1} \;=\; \mathbf{e}_k - \mathbf{H}_k^{-1}\nabla f(\boldsymbol{\theta}_k) \;=\; \mathbf{e}_k - \mathbf{H}_k^{-1}\bar{\mathbf{H}}_k\,\mathbf{e}_k \;=\; \mathbf{H}_k^{-1}(\mathbf{H}_k - \bar{\mathbf{H}}_k)\mathbf{e}_k.$$

For the difference $\mathbf{H}_k - \bar{\mathbf{H}}_k = \int_0^1 [\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}_k) - \mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}^* + t\mathbf{e}_k)]\,dt$, the integrand has operator norm at most $M\,\|\mathbf{e}_k - t\mathbf{e}_k\| = M(1-t)\|\mathbf{e}_k\|$, so

$$\|\mathbf{H}_k - \bar{\mathbf{H}}_k\|_{\mathrm{op}} \;\le\; \int_0^1 M(1-t)\,\|\mathbf{e}_k\|\,dt \;=\; \tfrac{M}{2}\,\|\mathbf{e}_k\|.$$

For $\|\mathbf{H}_k^{-1}\|_{\mathrm{op}}$: $\mathbf{H}_k = \mathbf{H}_* + (\mathbf{H}_k - \mathbf{H}_*)$ with $\|\mathbf{H}_k - \mathbf{H}_*\|_{\mathrm{op}} \le M\|\mathbf{e}_k\|$ by Hessian-Lipschitz, so for any unit $\mathbf{v}$:

$$\mathbf{v}^\top\mathbf{H}_k\mathbf{v} \;\ge\; \mathbf{v}^\top\mathbf{H}_*\mathbf{v} - \|\mathbf{H}_k - \mathbf{H}_*\|_{\mathrm{op}} \;\ge\; \mu - M\|\mathbf{e}_k\|.$$

Provided $\|\mathbf{e}_k\| < \mu/M$, $\mathbf{H}_k \succ 0$ and $\|\mathbf{H}_k^{-1}\|_{\mathrm{op}} \le (\mu - M\|\mathbf{e}_k\|)^{-1}$. Combining:

$$\|\mathbf{e}_{k+1}\| \;\le\; \|\mathbf{H}_k^{-1}\|_{\mathrm{op}}\,\|\mathbf{H}_k - \bar{\mathbf{H}}_k\|_{\mathrm{op}}\,\|\mathbf{e}_k\| \;\le\; \frac{M}{2(\mu - M\,\|\mathbf{e}_k\|)}\,\|\mathbf{e}_k\|^2,$$

which is (73). The right-hand side is at most $\|\mathbf{e}_k\|/2$ whenever $M\|\mathbf{e}_k\| \le \mu/2$, so the condition $\|\mathbf{e}_k\| < \mu/M$ propagates to $\|\mathbf{e}_{k+1}\| < \mu/M$ and the iterates remain in $\mathcal{N}$. □

The error squares each step, so the number of correct digits doubles per iteration: the $O(\log\log(1/\varepsilon))$ quadratic rate of the taxonomy in §6, which none of the three convergence templates captures. The constants are the local quantities M and μ at $\boldsymbol{\theta}^*$, not the global condition number $\kappa = L/\mu$ that governs gradient descent. For a quadratic the Hessian is constant, $M = 0$, and Newton converges in one step from any $\boldsymbol{\theta}_0$; SOFO inherits this in the limit where the sketch dimension reaches $\mathrm{rank}(\mathbf{G})$.

13.3 Affine Invariance

The condition-number independence of Theorem 39 is a special case of a more fundamental property: Newton's method commutes with linear reparameterisations.

Lemma 40 (Affine invariance)**.** *Let $\mathbf{T} \in \mathbb{R}^{P\times P}$ be invertible and define $\bar{f}(\boldsymbol{\gamma}) := f(\mathbf{T}\boldsymbol{\gamma})$. If $\{\boldsymbol{\theta}_k\}$ are the Newton iterates of f from $\boldsymbol{\theta}_0$ and $\{\boldsymbol{\gamma}_k\}$ are the Newton iterates of $\bar{f}$ from $\boldsymbol{\gamma}_0 = \mathbf{T}^{-1}\boldsymbol{\theta}_0$, then $\boldsymbol{\theta}_k = \mathbf{T}\boldsymbol{\gamma}_k$ for all $k \ge 0$.*

Proof. By the chain rule, $\nabla\bar{f}(\boldsymbol{\gamma}) = \mathbf{T}^\top\nabla f(\mathbf{T}\boldsymbol{\gamma})$ and $\nabla^2\bar{f}(\boldsymbol{\gamma}) = \mathbf{T}^\top\mathbf{H}_{\boldsymbol{\theta}}(\mathbf{T}\boldsymbol{\gamma})\mathbf{T}$. By induction: $\boldsymbol{\theta}_0 = \mathbf{T}\boldsymbol{\gamma}_0$ by hypothesis; if $\boldsymbol{\theta}_k = \mathbf{T}\boldsymbol{\gamma}_k$, then

$$\boldsymbol{\gamma}_{k+1} \;=\; \boldsymbol{\gamma}_k - (\mathbf{T}^\top\mathbf{H}_k\mathbf{T})^{-1}\mathbf{T}^\top\nabla f(\boldsymbol{\theta}_k) \;=\; \boldsymbol{\gamma}_k - \mathbf{T}^{-1}\mathbf{H}_k^{-1}\nabla f(\boldsymbol{\theta}_k),$$

so $\mathbf{T}\boldsymbol{\gamma}_{k+1} = \mathbf{T}\boldsymbol{\gamma}_k - \mathbf{H}_k^{-1}\nabla f(\boldsymbol{\theta}_k) = \boldsymbol{\theta}_k - \mathbf{H}_k^{-1}\nabla f(\boldsymbol{\theta}_k) = \boldsymbol{\theta}_{k+1}$. □

Corollary 41 (Invariants of Newton's method)**.** *Under the change of variables of Lemma 40:*

1. Newton step: $\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k = \mathbf{T}\,(\boldsymbol{\gamma}_{k+1} - \boldsymbol{\gamma}_k)$.
2. Newton decrement: $\|\nabla\bar{f}(\boldsymbol{\gamma})\|^2_{\nabla^2\bar{f}(\boldsymbol{\gamma})^{-1}} \;=\; \|\nabla f(\boldsymbol{\theta})\|^2_{\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta})^{-1}}$ *whenever* $\boldsymbol{\theta} = \mathbf{T}\boldsymbol{\gamma}$.

Proof. (1) is immediate from Lemma 40. (2) expands as

$$\nabla\bar{f}^\top(\nabla^2\bar{f})^{-1}\nabla\bar{f} \;=\; (\mathbf{T}^\top\nabla f)^\top(\mathbf{T}^\top\mathbf{H}\mathbf{T})^{-1}(\mathbf{T}^\top\nabla f) \;=\; \nabla f^\top\mathbf{T}\mathbf{T}^{-1}\mathbf{H}^{-1}\mathbf{T}^{-\top}\mathbf{T}^\top\nabla f \;=\; \nabla f^\top\mathbf{H}^{-1}\nabla f.$$

□

The Euclidean condition number $\kappa = L/\mu$ is *not* affine-invariant — rescaling axes non-uniformly with a non-orthogonal $\mathbf{T}$ changes L and μ unequally and shifts κ arbitrarily. Newton's affine invariance (Corollary 41) is precisely why its local rate needs no condition-number-dependent tuning, unlike gradient descent. The random-subspace methods of §3 of the SOFO notes keep it only in part. Given its subspace, a sketched Newton step is the $\mathbf{H}_k$-orthogonal projection of the Newton step onto that subspace (Lemma 6 of the SOFO notes), which does not depend on the parameterisation. The subspace itself is random, and a Gaussian sketch is invariant only under orthogonal changes of variables (§3.1 of the SOFO notes), so a non-orthogonal $\mathbf{T}$ changes which subspaces are drawn.

13.4 Failure Modes and Damped Newton

Two obstacles prevent the pure Newton step from being deployed as-is on neural networks.

INDEFINITE HESSIAN. The pure Newton step requires $\mathbf{H}_k \succ 0$. Far from the optimum, or for non-convex f in general, $\mathbf{H}_k$ may have zero or negative eigenvalues; the Newton step then has no minimisation interpretation and may point uphill. The standard fix is *Tikhonov damping* (also called *Levenberg–Marquardt*):

$$\mathbf{d}_k \;=\; -\big(\mathbf{H}_k + \gamma\,\mathbf{I}\big)^{-1}\,\nabla f(\boldsymbol{\theta}_k), \qquad \gamma \geq 0. \tag{74}$$

γ shifts every eigenvalue of $\mathbf{H}_k$ by $+\gamma$, restoring positive-definiteness for $\gamma > -\lambda_{\min}(\mathbf{H}_k)$, and interpolates between Newton ($\gamma \to 0$) and a short gradient step ($\gamma \to \infty$). Section 14 shows that γ is the Lagrange multiplier of a step-length constraint and explains how to choose it.

COMPUTATIONAL COST. Forming $\mathbf{H}_k$ costs $O(P^2)$ memory and inverting it costs $O(P^3)$. For neural networks with P in the millions, both are infeasible. Even Hessian–vector products at $\leq 5\times$ a forward evaluation (§5.5) place a Krylov-style inversion (e.g. conjugate gradient on $\mathbf{H}_k\mathbf{x} = -\nabla f$) at one HVP per CG iteration, with the number of iterations growing with the effective condition number. The subspace methods of §3 replace the $P \times P$ inversion with an $r \times r$ one ($r \ll P$); SOFO further uses the GGN's factored form $\mathbf{G} = \mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}} \mathbf{J}$ to replace each HVP with a JVP (no reverse pass), bringing the per-iteration cost into the regime of practical NN training.

14 TRUST REGIONS AND DAMPING

Every second-order method in this part minimises a local quadratic model (27),

$$M_k(\mathbf{p}) \;=\; f(\boldsymbol{\theta}_k) + \mathbf{g}^\top \mathbf{p} + \tfrac{1}{2}\,\mathbf{p}^\top \mathbf{B}\,\mathbf{p}, \qquad \mathbf{g} \coloneqq \nabla f(\boldsymbol{\theta}_k),$$

with a symmetric PSD curvature matrix $\mathbf{B}$: the Hessian where it is PSD, the GGN (§15) or the Fisher (§16). The model is accurate only near $\mathbf{p} = \mathbf{0}$, yet its minimiser $-\mathbf{B}^{-1}\mathbf{g}$ can lie far away, especially along directions of small curvature. *Damping* is the name for the techniques that keep the step inside the region where M_k can be trusted [28]. This section treats the standard one, Tikhonov damping, its equivalence with a trust region, and how its strength is chosen.

14.1 Trust Region versus Line Search

A line search (§7.3) fixes the direction first and then chooses how far to move along it. A *trust-region method* reverses the order: it first fixes a radius Δ within which the model is believed, then finds the best step inside that ball [1, §8.3.3]:

$$\mathbf{p}^* \;=\; \arg\min_{\mathbf{p}} \; M_k(\mathbf{p}) \quad \text{s.t.} \quad \|\mathbf{p}\| \leq \Delta. \tag{75}$$

The difference matters when $\mathbf{B}$ is ill-conditioned. A line search can only rescale $-\mathbf{B}^{-1}\mathbf{g}$ as a whole: if a spurious low-curvature direction inflates one component by a factor of 10^3, every component must be shrunk by that factor to make the step safe, and the useful high-curvature components become negligible [28, §8.8]. The trust-region step instead changes direction as Δ shrinks, turning from the Newton step towards $-\mathbf{g}$ (Figure 12). The next subsection shows that this is exactly what adding $\gamma\mathbf{I}$ to $\mathbf{B}$ does, with γ the Lagrange multiplier of the constraint.

14.2 Tikhonov Damping and the Moré–Sorensen Characterisation

Theorem 42 (Moré–Sorensen, PSD case [11, Thm. 4.1])**.** *Let $\mathbf{B} \succeq 0$ and $\Delta > 0$. Then $\mathbf{p}^*$ solves (75) if and only if $\|\mathbf{p}^*\| \leq \Delta$ and there is a $\gamma \geq 0$ with*

$$(\mathbf{B} + \gamma\,\mathbf{I})\,\mathbf{p}^* = -\mathbf{g}, \tag{76}$$

$$\gamma\,(\Delta - \|\mathbf{p}^*\|) = 0. \tag{77}$$

Proof. Write the constraint as $c(\mathbf{p}) = \frac{1}{2}(\|\mathbf{p}\|^2 - \Delta^2) \leq 0$, so that $\nabla c(\mathbf{p}) = \mathbf{p}$, and apply Theorem 19 with multiplier γ:

$$\begin{aligned} \nabla M_k(\mathbf{p}^*) + \gamma\,\nabla c(\mathbf{p}^*) = \mathbf{g} + \mathbf{B}\mathbf{p}^* + \gamma\,\mathbf{p}^* = \mathbf{0} \qquad & \text{(stationarity: gives (76))} \\ \gamma\,c(\mathbf{p}^*) = \tfrac{\gamma}{2}\,(\|\mathbf{p}^*\| - \Delta)(\|\mathbf{p}^*\| + \Delta) = 0 \qquad & \text{(complementary slackness: gives (77), as } \|\mathbf{p}^*\| + \Delta > 0) \end{aligned}$$

with $\gamma \geq 0$ (dual feasibility) and $\|\mathbf{p}^*\| \leq \Delta$ (primal feasibility). These conditions are necessary: if the constraint is active then $\nabla c(\mathbf{p}^*) = \mathbf{p}^* \neq \mathbf{0}$, so the constraint qualification of Theorem 19 holds. They are sufficient because $\mathbf{B} \succeq 0$ makes M_k convex and the ball is a convex set, so KKT points are global minima. □

For indefinite $\mathbf{B}$ the same characterisation holds with the extra condition $\mathbf{B}+\gamma\mathbf{I} \succeq 0$, which forces $\gamma \geq -\lambda_{\min}(\mathbf{B})$ [11, Thm. 4.1]; we do not need this case, since the curvature matrices used for neural networks are PSD by construction. Equation (76) is *Tikhonov damping*: the trust-region step is an ordinary Newton-type step with $\mathbf{B}$ replaced by $\mathbf{B}+\gamma\mathbf{I}$. Complementarity (77) leaves two cases:

- $\gamma = 0$: the undamped step lies inside the ball and is accepted as is;
- $\gamma > 0$: the step lies on the boundary, $\|\mathbf{p}^*\| = \Delta$, and damping has shortened it.

THE DAMPED STEP IN THE EIGENBASIS. Let $\mathbf{B} = \mathbf{U}\mathbf{S}\mathbf{U}^\top$ with $\mathbf{U}$ orthogonal and $\mathbf{S} = \operatorname{diag}(\sigma_1, \dots, \sigma_P)$, $\sigma_j \geq 0$. Since $\mathbf{B}+\gamma\mathbf{I} = \mathbf{U}(\mathbf{S}+\gamma\mathbf{I})\mathbf{U}^\top$, inversion is diagonal in this basis:

$$\begin{aligned} \mathbf{p}(\gamma) &= -(\mathbf{B}+\gamma\,\mathbf{I})^{-1}\mathbf{g} = -\mathbf{U}(\mathbf{S}+\gamma\,\mathbf{I})^{-1}\mathbf{U}^\top\mathbf{g} && (\mathbf{U}\mathbf{U}^\top = \mathbf{I}) \\ &= -\sum_{j=1}^{P} \frac{\mathbf{u}_j^\top \mathbf{g}}{\sigma_j + \gamma}\,\mathbf{u}_j, && \text{(expand in the eigenbasis)} \end{aligned} \tag{78}$$

so that

$$\|\mathbf{p}(\gamma)\|^2 \;=\; \sum_{j=1}^{P} \frac{(\mathbf{u}_j^\top \mathbf{g})^2}{(\sigma_j+\gamma)^2}.$$

Each term decreases strictly in γ, so $\|\mathbf{p}(\gamma)\|$ is continuous and strictly decreasing, and every radius Δ smaller than the undamped step length corresponds to exactly one $\gamma > 0$ [11]. Choosing Δ and choosing γ are therefore the same decision. The two limits are

$$\gamma \to 0: \;\; \mathbf{p}(\gamma) \to -\mathbf{B}^{+}\mathbf{g} \;\text{ (Newton-type step)}, \qquad \gamma \to \infty: \;\; \mathbf{p}(\gamma) \approx -\tfrac{1}{\gamma}\,\mathbf{g} \;\text{ (short gradient step)},$$

where the first limit assumes $\mathbf{g}$ has no component in the null space of $\mathbf{B}$, which holds for the GGN with $\mathbf{H}_{\boldsymbol{\phi}} \succ 0$ since $\mathbf{g} = \mathbf{J}^\top \nabla_{\boldsymbol{\phi}} \mathcal{L}$ lies in the range of $\mathbf{J}^\top$.

DAMPING SUPPRESSES LOW-CURVATURE DIRECTIONS. Comparing (78) with the undamped step, whose j-th component is $-\mathbf{u}_j^\top \mathbf{g}/\sigma_j$, the damped component is

$$\mathbf{u}_j^\top \mathbf{p}(\gamma) \;=\; \frac{\sigma_j}{\sigma_j+\gamma}\left(-\frac{\mathbf{u}_j^\top \mathbf{g}}{\sigma_j}\right).$$

Directions with $\sigma_j \gg \gamma$ keep almost their full Newton component, while directions with $\sigma_j \ll \gamma$ are cut by the factor σ_j/γ [28, §8.1]. These are exactly the directions along which the undamped step travels furthest, and where the quadratic model is least reliable. This selective shrinkage is what rotates the step towards $-\mathbf{g}$ in Figure 12, and what a line search cannot do.

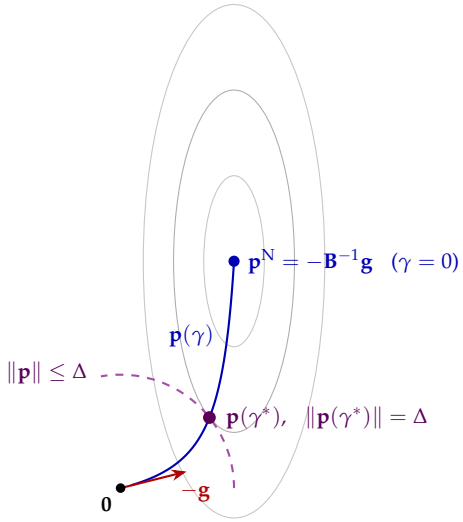


Figure 12: The damped step $\mathbf{p}(\gamma) = -(\mathbf{B}+\gamma\mathbf{I})^{-1}\mathbf{g}$ for $\mathbf{B} = \operatorname{diag}(4, 0.5)$ and $\mathbf{g} = (-4, -1)$, traced from $\gamma = 0$ (the Newton point) towards $\gamma \to \infty$, where it approaches $\mathbf{0}$ along $-\mathbf{g}$. Grey ellipses are level sets of the model M_k. For the radius $\Delta = 1$ the path crosses the trust-region boundary at $\gamma^* \approx 1.11$, where a level set is tangent to the ball: this is the solution of (75). Damping turns the step towards the high-curvature axis and away from the low-curvature one, rather than only shortening it.

14.3 Levenberg–Marquardt Adaptation

No fixed γ works throughout training: the region where M_k is accurate changes with $\boldsymbol{\theta}_k$, and γ also depends on the scale of f (Tikhonov damping with γ for f gives the same step as 2γ for $2f$). The *Levenberg–Marquardt heuristic* adapts γ by comparing the actual decrease with the decrease the model predicted [28, 29], through the *reduction ratio*

$$\rho \;=\; \frac{f(\boldsymbol{\theta}_k+\mathbf{p})-f(\boldsymbol{\theta}_k)}{M_k(\mathbf{p})-M_k(\mathbf{0})}. \tag{79}$$

A ratio near one means the model predicted the decrease well, so the trust region can grow; a small ratio means the model over-promised, so it must shrink. Martens and Sutskever use the rule

$$\rho > \tfrac{3}{4}: \;\; \gamma \leftarrow \tfrac{2}{3}\,\gamma, \qquad\qquad \rho < \tfrac{1}{4}: \;\; \gamma \leftarrow \tfrac{3}{2}\,\gamma.$$

Classical trust-region methods apply the same test to Δ and then solve for the matching γ [11]; in deep learning it is simpler to adapt γ directly, because the map from γ to Δ changes as $\mathbf{B}$ changes and finding the γ for a given Δ needs extra solves [28, §8.6]. Near a minimum the model becomes accurate on a growing neighbourhood, $\rho \to 1$, and the rule drives $\gamma \to 0$ fast enough that the local convergence rate of the undamped method is recovered [29].

14.4 Structural Damping

Tikhonov damping penalises every direction equally, but in a recurrent network a small change in the recurrent weights acts at every time step and can change the whole hidden-state trajectory, so the quadratic model fails along those directions first. *Structural damping* adds the GGN $\mathbf{D}$ of a penalty on changes in the hidden states, giving the damped curvature $\mathbf{B}+\mu\mathbf{D}+\gamma\mathbf{I}$, and was what made Hessian-free training of RNNs on long-range dependency problems robust [28, 30].

14.5 Relative Damping and Links to First-Order Methods

SOFO sets the damping relative to the curvature it has just measured, $\gamma = \lambda\, s_{\max}(\tilde{\mathbf{G}})$, where $\tilde{\mathbf{G}}$ is its sketched GGN, $s_{\max}$ its largest eigenvalue and $\lambda > 0$ a dimensionless constant. This removes the scale dependence of γ. If the loss is multiplied by $c > 0$, every second-order quantity scales by c:

$$\begin{aligned} \tilde{\mathbf{G}} \to c\,\tilde{\mathbf{G}}, \quad \mathbf{C} \to c\,\mathbf{C}, \quad s_{\max} \to c\,s_{\max}, \quad \gamma \to c\,\gamma \qquad & \text{(curvature, gradient sketch, damping)} \\ (c\tilde{\mathbf{G}} + c\gamma\mathbf{I})^{-1}\, c\,\mathbf{C} \;=\; (\tilde{\mathbf{G}}+\gamma\mathbf{I})^{-1}\mathbf{C} \qquad & \text{(the step is unchanged)} \end{aligned}$$

so λ needs no retuning when the loss is rescaled. The rule also adapts in the right direction: sharper curvature raises γ and makes the step more conservative, and γ falls as the curvature shrinks near a solution. It also handles rank deficiency: when the sketch dimension exceeds the rank of $\mathbf{G}$, the sketched GGN $\tilde{\mathbf{G}}$ is singular (§17), and any $\gamma > 0$ makes $\tilde{\mathbf{G}}+\gamma\mathbf{I}$ invertible, here with γ on the scale of the measured curvature.

The same multiplier appears in first-order methods. Adam's per-coordinate step satisfies $|\Delta_k| \lesssim \alpha$ whatever the gradient's scale, an implicit trust region of radius about α (§12.5). And gradient descent is the fully damped limit of (76): with $\mathbf{B} = \mathbf{0}$ the step is $-\mathbf{g}/\gamma$, so $\gamma = 1/\alpha$ is an inverse learning rate. Part V keeps this penalty $\frac{1}{2\alpha}\|\mathbf{p}\|^2$ but measures the step in a norm other than the Euclidean one (§22.1), which recovers sign descent and Muon.

15 GENERALIZED GAUSS–NEWTON

CLASSICAL GAUSS–NEWTON. The idea comes from nonlinear least squares [4, 11]. For residuals $\mathbf{r}(\boldsymbol{\theta}) \in \mathbb{R}^m$ with Jacobian $\mathbf{J} = \partial\mathbf{r}/\partial\boldsymbol{\theta}^\top$, the objective $f = \frac{1}{2}\|\mathbf{r}\|^2$ has

$$\begin{aligned} \nabla f = \mathbf{J}^\top\mathbf{r} = \textstyle\sum_j r_j\,\nabla r_j \qquad & \text{(chain rule, } \nabla_{\mathbf{r}}\tfrac{1}{2}\|\mathbf{r}\|^2 = \mathbf{r}\text{)} \\ \mathbf{H} = \textstyle\sum_j \nabla r_j\,\nabla r_j^\top + \sum_j r_j\,\nabla^2 r_j \qquad & \text{(product rule on each } r_j\nabla r_j\text{)} \\ = \mathbf{J}^\top\mathbf{J} + \mathbf{S}, \qquad \mathbf{S} \coloneqq \textstyle\sum_j r_j\,\nabla^2 r_j. \end{aligned}$$

The first term needs only first derivatives and is always PSD. The second is small when the residuals are small or nearly affine near the solution, and vanishes at a zero-residual fit. Dropping it gives the Gauss–Newton step $\mathbf{d}_{\mathrm{GN}} = -(\mathbf{J}^\top\mathbf{J})^{-1}\mathbf{J}^\top\mathbf{r}$, which is the solution of a linear least-squares problem: linearising the residual inside the square,

$$\mathbf{d}_{\mathrm{GN}} \;=\; \arg\min_{\mathbf{d}} \;\tfrac{1}{2}\,\|\mathbf{r}+\mathbf{J}\mathbf{d}\|^2 \;=\; -\mathbf{J}^{+}\mathbf{r}.$$

The generalised Gauss–Newton matrix below applies the same idea to any convex loss: keep the curvature of the loss, drop the curvature of the model. Which curvature is kept depends on where the model ends and the loss

begins, so the split should put as much of the computation into the convex loss as possible, for example the final softmax into a cross-entropy loss [13, §6.3][31].

We now specialise to the composite structure arising in neural network training [32, 33]. Let $\boldsymbol{\phi} : \mathbb{R}^P \to \mathbb{R}^d$ be the model output (e.g. logits, predicted torques) and $\mathcal{L} : \mathbb{R}^d \to \mathbb{R}$ a **convex loss** in $\boldsymbol{\phi}$ (Definition 12), meaning $\mathbf{H}_{\boldsymbol{\phi}} \succeq 0$. The overall objective $f : \mathbb{R}^P \to \mathbb{R}$ is the composition of $\mathcal{L}$ with the network:

$$f(\boldsymbol{\theta}) = \mathcal{L}\big(\boldsymbol{\phi}(\boldsymbol{\theta})\big)$$

Here f is our objective (the same role as c in Section 1), $\boldsymbol{\phi}$ maps parameters to model outputs, and $\mathcal{L}$ maps outputs to a scalar loss. We derive the GGN for a single sample with d-dimensional output. [20] The GGN arises from this composite structure, with $\mathbf{J} = \partial\boldsymbol{\phi}/\partial\boldsymbol{\theta}^\top \in \mathbb{R}^{d\times P}$ the Jacobian of the model output (Definition 18).

GRADIENT. By the chain rules:

$$\nabla_{\boldsymbol{\theta}} f = \mathbf{J}^\top \nabla_{\boldsymbol{\phi}} \mathcal{L} \tag{80}$$

HESSIAN. Differentiating again with the product rule:

$$\begin{aligned}
\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}) &= \frac{d}{d\boldsymbol{\theta}}\Big(\mathbf{J}^\top \nabla_{\boldsymbol{\phi}} \mathcal{L}\Big) && \text{(differentiate the gradient)} \\
&= \frac{d\mathbf{J}^\top}{d\boldsymbol{\theta}} \nabla_{\boldsymbol{\phi}} \mathcal{L} \;+\; \mathbf{J}^\top \frac{d(\nabla_{\boldsymbol{\phi}} \mathcal{L})}{d\boldsymbol{\theta}} && \text{(product rule)} \\
&= \underbrace{\sum_{j=1}^{d} \frac{\partial \mathcal{L}}{\partial \phi_j} \nabla^2_{\boldsymbol{\theta}} \phi_j}_{\mathbf{S}(\boldsymbol{\theta})} \;+\; \underbrace{\mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}} \mathbf{J}}_{\text{GGN term} \succeq 0} && \text{(chain rule on each term)}
\end{aligned} \tag{81}$$

Definition 20 (Output-Space Loss Hessian)**.** The *output-space Hessian* of the loss $\mathcal{L}$ w.r.t. model outputs $\boldsymbol{\phi}$ — distinct from $\mathbf{H}_{\boldsymbol{\theta}}$, which differentiates w.r.t. parameters $\boldsymbol{\theta}$:

$$\mathbf{H}_{\boldsymbol{\phi}}(\boldsymbol{\phi}) \;\coloneqq\; \nabla^2_{\boldsymbol{\phi}} \mathcal{L}(\boldsymbol{\phi}) \;=\; \begin{bmatrix} \dfrac{\partial^2 \mathcal{L}}{\partial \phi_1^2} & \cdots & \dfrac{\partial^2 \mathcal{L}}{\partial \phi_1 \partial \phi_d} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial^2 \mathcal{L}}{\partial \phi_d \partial \phi_1} & \cdots & \dfrac{\partial^2 \mathcal{L}}{\partial \phi_d^2} \end{bmatrix} \;\in\; \mathbb{R}^{d\times d}.$$

The second-order term $\mathbf{S}$ involves second derivatives of each model output ϕ_j w.r.t. parameters, weighted by $\partial\mathcal{L}/\partial\phi_j$. For piecewise-linear activations (ReLU, Leaky-ReLU), each ϕ_j is piecewise-linear in $\boldsymbol{\theta}$, so $\nabla^2_{\boldsymbol{\theta}}\phi_j = 0$ wherever defined and $\mathbf{S} = \mathbf{0}$. Even when $\mathbf{S} \neq 0$, two problems make it undesirable: (i) computing it requires one $P \times P$ second-derivative matrix per output dimension j, which is prohibitively expensive; (ii) $\mathbf{S}$ can be indefinite, making $\mathbf{H}_{\boldsymbol{\theta}}$ indefinite and the Newton direction potentially uphill. The GGN drops $\mathbf{S}$ entirely.

Definition 21 (Generalized Gauss-Newton)**.** The Generalized Gauss-Newton (GGN) matrix drops $\mathbf{S}$ (81), retaining only the tractable, PSD term:

$$\mathbf{G}(\boldsymbol{\theta}) = \mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}} \mathbf{J} \;\in\; \mathbb{R}^{P\times P} \tag{82}$$

With $\mathbf{G} \approx \mathbf{H}_{\boldsymbol{\theta}}$, the Newton update (71) becomes the *GGN update direction*:

$$\mathbf{G}\,\mathbf{d}_k = -\mathbf{J}^\top \nabla_{\boldsymbol{\phi}} \mathcal{L} \tag{83}$$

Lemma 43 (GGN is PSD)**.** *When $\mathcal{L}$ is convex in $\boldsymbol{\phi}$ (i.e. $\mathbf{H}_{\boldsymbol{\phi}} \succeq 0$), the GGN satisfies $\mathbf{G} \succeq 0$.*[21]

Proof. For any $\mathbf{v} \in \mathbb{R}^P$: $\mathbf{v}^\top \mathbf{G}\,\mathbf{v} = (\mathbf{J}\mathbf{v})^\top \mathbf{H}_{\boldsymbol{\phi}}\,(\mathbf{J}\mathbf{v}) \geq 0$, since $\mathbf{H}_{\boldsymbol{\phi}} \succeq 0$. □

This ensures the quadratic subproblem $\min_{\mathbf{d}}\ (\nabla_{\boldsymbol{\theta}} f)^\top \mathbf{d} + \frac{1}{2}\,\mathbf{d}^\top \mathbf{G}\,\mathbf{d}$ is convex and bounded below, giving a well-defined descent direction.

20 Over a minibatch of M samples (or sequences), each with N scalar outputs, $d = MN$; for the RNN of §1 with per-timestep output dimension D and horizon T, $N = DT$, and $\mathbf{J} \in \mathbb{R}^{MN\times P}$ stacks one Jacobian block per sequence.

21 Note that $f(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\phi}(\boldsymbol{\theta}))$ need not be convex in $\boldsymbol{\theta}$ globally (e.g. $\boldsymbol{\phi}$ is a neural network) — convexity of $\mathcal{L}$ in $\boldsymbol{\phi}$ alone suffices to guarantee $\mathbf{G} \succeq 0$.

LINEARISATION VIEW. The GGN is the *exact* Hessian of the objective on the locally linearised model. Let $\hat{\boldsymbol{\phi}}(\boldsymbol{\theta})$ be the linear approximation of the model output $\boldsymbol{\phi}$, so:

$$\begin{aligned}
\hat{\boldsymbol{\phi}}(\boldsymbol{\theta}) &= \boldsymbol{\phi}(\boldsymbol{\theta}_k) + \mathbf{J}(\boldsymbol{\theta}_k)\,(\boldsymbol{\theta} - \boldsymbol{\theta}_k) && \text{(first order Taylor around } \boldsymbol{\theta}_k\text{)} \\
\hat{f}(\boldsymbol{\theta}) &= \mathcal{L}(\hat{\boldsymbol{\phi}}(\boldsymbol{\theta})) && \text{(loss on the linearised model)} \\
\mathbf{H}_{\boldsymbol{\theta}}\hat{f} &= \underbrace{\sum_{j=1}^{d} \frac{\partial \mathcal{L}}{\partial \hat{\phi}_j}\, \nabla^2_{\boldsymbol{\theta}} \hat{\phi}_j}_{\mathbf{0}\ (\text{affine } \hat{\phi})} + \mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}}\, \mathbf{J} \\
&= \mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}}\, \mathbf{J} \;=\; \mathbf{G}. && (84)
\end{aligned}$$

Beyond the linearisation and PSD motivations, $\mathbf{H}_{\boldsymbol{\phi}}$ is the correct output curvature: at the true parameters, the GGN is exact in expectation.

Theorem 44 (Botev [33, Lemma 3.20])**.** *Suppose the loss is a conditional negative log-likelihood, $\mathcal{L} = -\log p_{\boldsymbol{\theta}}(\mathbf{y}|\boldsymbol{\phi}(\boldsymbol{\theta}))$, and there exist parameters $\boldsymbol{\theta}^*$ under which the model distribution $p_{\boldsymbol{\theta}^*}(\mathbf{y}|\mathbf{x})$ coincides with the true data-generating distribution $p_{\text{true}}(\mathbf{y}|\mathbf{x})$. Then:*

$$\mathbb{E}_{p_{\text{true}}}\big[\mathbf{G}(\boldsymbol{\theta}^*)\big] \;=\; \mathbb{E}_{p_{\text{true}}}\big[\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}^*)\big]$$

Proof. From (81), $\mathbf{H}_{\boldsymbol{\theta}} = \mathbf{G} + \mathbf{S}$ where $\mathbf{S} = \sum_{j=1}^{d} \frac{\partial \mathcal{L}}{\partial \phi_j} \nabla^2_{\boldsymbol{\theta}} \phi_j$. It suffices to show $\mathbb{E}_{p_{\text{true}}}[\mathbf{S}(\boldsymbol{\theta}^*)] = \mathbf{0}$. Since $\nabla^2_{\boldsymbol{\theta}} \phi_j$ depends only on $\mathbf{x}$ (not $\mathbf{y}$), we can push the inner expectation over $\mathbf{y}$ through:

$$\mathbb{E}_{p_{\text{true}}}[\mathbf{S}] = \sum_{j=1}^{d} \mathbb{E}_{p(\mathbf{x})}\left[\mathbb{E}_{p_{\text{true}}(\mathbf{y}|\mathbf{x})}\left[\frac{\partial \mathcal{L}}{\partial \phi_j}\right] \nabla^2_{\boldsymbol{\theta}} \phi_j\right]$$

At $\boldsymbol{\theta}^*$ we have $p_{\boldsymbol{\theta}^*}(\mathbf{y}|\mathbf{x}) = p_{\text{true}}(\mathbf{y}|\mathbf{x})$ and $\mathcal{L} = -\log p_{\boldsymbol{\theta}}(\mathbf{y}|\boldsymbol{\phi})$, so the inner expectation becomes:

$$\mathbb{E}_{p_{\boldsymbol{\theta}^*}(\mathbf{y}|\mathbf{x})}\left[\frac{\partial \mathcal{L}}{\partial \phi_j}\right] = \mathbb{E}_{p_{\boldsymbol{\theta}^*}}\left[-\frac{\partial \log p_{\boldsymbol{\theta}^*}}{\partial \phi_j}\right] = 0$$

by Lemma 45 applied in the output space. Hence every term in $\mathbb{E}[\mathbf{S}]$ vanishes. □

15.1 Levenberg–Marquardt Damping

The damped GGN update

$$\boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k \;-\; \eta\,(\mathbf{G} + \gamma\,\mathbf{I})^{-1} \nabla_{\boldsymbol{\theta}} f \tag{85}$$

is Tikhonov damping (76) with $\mathbf{B} = \mathbf{G}$, so by Theorem 42 it is the exact solution of the trust-region problem (75) for the GGN model, with γ adapted by the Levenberg–Marquardt rule (§14.3). For the GGN, damping is needed for a second reason: $\mathbf{G}$ is severely rank-deficient (§17), with $\operatorname{rank}(\mathbf{G}) \le MN \ll P$ for M samples with N outputs each, so $\mathbf{G}$ alone is singular and $\gamma > 0$ is what makes $\mathbf{G} + \gamma\mathbf{I}$ invertible. In the eigenbasis (78), the components along the zero-curvature directions are simply those of the gradient scaled by $1/\gamma$. SOFO's choice of γ relative to the sketched curvature is discussed in §14.5.

16 FISHER INFORMATION AND NATURAL GRADIENT

16.1 The Fisher Matrix

We derive some standard properties of the Fisher matrix that will be used throughout this article.

Definition 22 (Fisher information matrix)**.** For a probabilistic model $p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})$, the Fisher information matrix is the expected outer product of the score:

$$\mathbf{F}(\boldsymbol{\theta}) \;:=\; \mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}\Big[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})\; \big(\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})\big)^\top\Big] \tag{86}$$

The Fisher is PSD by construction (it is an expected outer product). Using $\mathbf{F}$ as the curvature matrix gives the *natural gradient* $\mathbf{F}^{-1}\nabla_{\boldsymbol{\theta}} f$, which is invariant to reparameterisation of the model [10].

Lemma 45 (Zero expected score)**.** $\mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})] = \mathbf{0}$

Proof. $\mathbb{E}_{p_{\boldsymbol{\theta}}}[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}] = \int p_{\boldsymbol{\theta}} \cdot \dfrac{\nabla_{\boldsymbol{\theta}} p_{\boldsymbol{\theta}}}{p_{\boldsymbol{\theta}}}\, d\mathbf{y} = \nabla_{\boldsymbol{\theta}} \int p_{\boldsymbol{\theta}}\, d\mathbf{y} = \nabla_{\boldsymbol{\theta}}\, 1 = \mathbf{0}.$ □

Lemma 46 (Log-derivative trick). *For a differentiable $p_{\boldsymbol{\theta}}$ and vector-valued $\mathbf{g}(\boldsymbol{\theta}, \mathbf{y})$:*

$$\nabla_{\boldsymbol{\theta}} \mathbb{E}_{p_{\boldsymbol{\theta}}}[\mathbf{g}] \;=\; \mathbb{E}_{p_{\boldsymbol{\theta}}}\Big[\nabla_{\boldsymbol{\theta}}\mathbf{g} \;+\; (\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}})\, \mathbf{g}^\top\Big]$$

Proof. Write $\mathbb{E}_{p_{\boldsymbol{\theta}}}[\cdot] = \int p_{\boldsymbol{\theta}}(\mathbf{y})\,(\cdot)\,d\mathbf{y}$ and use $\nabla_{\boldsymbol{\theta}} p_{\boldsymbol{\theta}} = p_{\boldsymbol{\theta}} \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}$. For scalar $h(\mathbf{y})$:

$$\nabla_{\boldsymbol{\theta}} \mathbb{E}_{p_{\boldsymbol{\theta}}}[h] = \int (\nabla_{\boldsymbol{\theta}} p_{\boldsymbol{\theta}})\, h\, d\mathbf{y} = \mathbb{E}_{p_{\boldsymbol{\theta}}}[h\, \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}]$$

For vector $\mathbf{g}(\boldsymbol{\theta}, \mathbf{y})$, the product rule on the integrand $p_{\boldsymbol{\theta}}(\mathbf{y})\, \mathbf{g}(\boldsymbol{\theta}, \mathbf{y})$:

$$\nabla_{\boldsymbol{\theta}} \mathbb{E}_{p_{\boldsymbol{\theta}}}[\mathbf{g}] = \int \nabla_{\boldsymbol{\theta}}(p_{\boldsymbol{\theta}}\, \mathbf{g})\, d\mathbf{y} = \int \left[(\nabla_{\boldsymbol{\theta}} p_{\boldsymbol{\theta}})\, \mathbf{g}^\top + p_{\boldsymbol{\theta}}\, \nabla_{\boldsymbol{\theta}} \mathbf{g}\right] d\mathbf{y} = \mathbb{E}_{p_{\boldsymbol{\theta}}}\Big[(\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}})\, \mathbf{g}^\top + \nabla_{\boldsymbol{\theta}} \mathbf{g}\Big] \quad \square$$

Lemma 47 ([33, Eq. 3.32–3.34]). *The Fisher equals the negative expected Hessian of the log-likelihood:*

$$\mathbf{F}(\boldsymbol{\theta}) \;=\; -\mathbb{E}_{p_{\boldsymbol{\theta}}}\Big[\nabla^2_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})\Big] \tag{87}$$

Proof. By Lemma 45, the expected score is zero: $\mathbb{E}_{p_{\boldsymbol{\theta}}}[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}] = \mathbf{0}$. Differentiating both sides with respect to $\boldsymbol{\theta}$ and applying Lemma 46 with $\mathbf{g} = \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}$:

$$\begin{aligned}
\mathbf{0} &= \nabla_{\boldsymbol{\theta}}\, \mathbb{E}_{p_{\boldsymbol{\theta}}}[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}] \\
&= \mathbb{E}_{p_{\boldsymbol{\theta}}}\Big[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}\, (\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}})^\top \;+\; \nabla^2_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}\Big] && \text{(Lemma 46)} \\
&= \underbrace{\mathbb{E}_{p_{\boldsymbol{\theta}}}\Big[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}\, (\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}})^\top\Big]}_{\mathbf{F}} + \mathbb{E}_{p_{\boldsymbol{\theta}}}\Big[\nabla^2_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}\Big] && \text{(linearity of } \mathbb{E}\text{)} \\
\mathbf{F} &= -\mathbb{E}_{p_{\boldsymbol{\theta}}}[\nabla^2_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}] \quad \square
\end{aligned}$$

The following proposition establishes the fundamental link between the Fisher and the KL divergence: $\mathbf{F}$ is precisely the local quadratic approximation of the KL divergence.

Proposition 48 (Fisher as the Hessian of the KL divergence). *Let $p_{\boldsymbol{\theta}}$ be a parametric family of distributions. The KL divergence from $p_{\boldsymbol{\theta}}$ to a perturbed distribution $p_{\boldsymbol{\theta}+\mathbf{d}}$ admits the Taylor expansion:*

$$D_{\mathrm{KL}}(p_{\boldsymbol{\theta}} \,\|\, p_{\boldsymbol{\theta}+\mathbf{d}}) \;=\; \frac{1}{2}\, \mathbf{d}^\top \mathbf{F}(\boldsymbol{\theta})\, \mathbf{d} \;+\; O(\|\mathbf{d}\|^3). \tag{88}$$

In particular, the zeroth-order term is zero, the first-order term vanishes, and the second-order term is governed by the Fisher.

Proof. Write the KL divergence as

$$D_{\mathrm{KL}}(p_{\boldsymbol{\theta}} \,\|\, p_{\boldsymbol{\theta}+\mathbf{d}}) \;=\; \mathbb{E}_{p_{\boldsymbol{\theta}}}[\log p_{\boldsymbol{\theta}} - \log p_{\boldsymbol{\theta}+\mathbf{d}}]\,.$$

Taylor-expand $\log p_{\boldsymbol{\theta}+\mathbf{d}}$ around $\mathbf{d} = \mathbf{0}$:

$$\log p_{\boldsymbol{\theta}+\mathbf{d}} \;=\; \log p_{\boldsymbol{\theta}} \;+\; \mathbf{d}^\top \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}} \;+\; \tfrac{1}{2}\, \mathbf{d}^\top \nabla^2_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}\, \mathbf{d} \;+\; O(\|\mathbf{d}\|^3).$$

Subtracting and taking $\mathbb{E}_{p_{\boldsymbol{\theta}}}[\cdot]$ term by term:

- **Zeroth order**: $\mathbb{E}_{p_{\boldsymbol{\theta}}}[\log p_{\boldsymbol{\theta}} - \log p_{\boldsymbol{\theta}}] = 0$.
- **First order**: $-\mathbf{d}^\top \mathbb{E}_{p_{\boldsymbol{\theta}}}[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}] = -\mathbf{d}^\top \cdot \mathbf{0} = 0$, by Lemma 45.
- **Second order**: $-\frac{1}{2}\, \mathbf{d}^\top \mathbb{E}_{p_{\boldsymbol{\theta}}}[\nabla^2_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}]\, \mathbf{d} = \frac{1}{2}\, \mathbf{d}^\top \mathbf{F}(\boldsymbol{\theta})\, \mathbf{d}$, by Lemma 47.

Combining gives (88). $\square$

16.2 Fisher for Neural Networks

For a model with outputs $\boldsymbol{\phi}(\boldsymbol{\theta})$ and Jacobian $\mathbf{J} = \partial \boldsymbol{\phi} / \partial \boldsymbol{\theta}^\top$:

$$\begin{aligned}
\mathbf{F} &= \mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}\Big[\nabla_{\boldsymbol{\theta}} \log p\; (\nabla_{\boldsymbol{\theta}} \log p)^\top\Big] \\
&= \mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}\Big[(\mathbf{J}^\top \nabla_{\boldsymbol{\phi}} \log p)\, (\mathbf{J}^\top \nabla_{\boldsymbol{\phi}} \log p)^\top\Big] && \text{(chain rule: } \nabla_{\boldsymbol{\theta}} \log p = \mathbf{J}^\top \nabla_{\boldsymbol{\phi}} \log p\text{)} \\
&= \mathbf{J}^\top \underbrace{\mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}\Big[\nabla_{\boldsymbol{\phi}} \log p\; (\nabla_{\boldsymbol{\phi}} \log p)^\top\Big]}_{=:\, \mathbf{F}_L \text{ (pre-activation Fisher)}} \mathbf{J} && \text{(}\mathbf{J}\text{ indep. of }\mathbf{y}\text{)}
\end{aligned}$$

This gives the decomposition:

$$\mathbf{F} \;=\; \mathbf{J}^\top\, \mathbf{F}_L\, \mathbf{J} \tag{89}$$

Theorem 49 (GGN–Fisher equivalence [33, Section 3.5.1], [32, Appendix C.2])**.** *The GGN and Fisher matrices are equal whenever* $\mathbf{H}_{\boldsymbol{\phi}}$ *is independent of* $\mathbf{y}$*.*

Proof. The GGN is $\mathbf{G} = \mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}} \mathbf{J}$. From (89) and applying Lemma 47 in the output space:

$$\begin{aligned} \mathbf{F}_L &= \mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}\left[\nabla_{\boldsymbol{\phi}} \log p \, (\nabla_{\boldsymbol{\phi}} \log p)^\top\right] \\ &= \mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}\left[\mathbf{H}_{\boldsymbol{\phi}}\right] \qquad \text{(Lemma 47 in output space, } \mathcal{L} = -\log p_{\boldsymbol{\theta}}\text{)} \end{aligned}$$

Comparing $\mathbf{F} = \mathbf{J}^\top \mathbb{E}_{p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})}[\mathbf{H}_{\boldsymbol{\phi}}]\,\mathbf{J}$ with $\mathbf{G} = \mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}} \mathbf{J}$, the two coincide precisely when $\mathbf{H}_{\boldsymbol{\phi}}$ does not depend on $\mathbf{y}$, since then the expectation is trivial. □

16.3 Exponential Family Losses

The independence condition of Theorem 49 holds whenever $\boldsymbol{\phi}$ parametrises the natural parameters of an *exponential family* distribution [33]. An exponential family is any distribution whose log-probability can be written as:

$$\log p(\mathbf{y}|\boldsymbol{\eta}) = \log h(\mathbf{y}) + T(\mathbf{y})^\top \boldsymbol{\eta} - \log Z(\boldsymbol{\eta})$$

where $\boldsymbol{\eta}$ are the *natural parameters*, $T(\mathbf{y})$ is the *sufficient statistic* (a function of the data only), $h(\mathbf{y})$ is the *base measure* (a normalisation term depending only on $\mathbf{y}$), and $Z(\boldsymbol{\eta}) = \int h(\mathbf{y}) \exp(T(\mathbf{y})^\top \boldsymbol{\eta})\, d\mathbf{y}$ is the *partition function* ensuring the distribution integrates to 1. The key structural property is that the log-probability is *linear* in $\boldsymbol{\eta}$ (the $T(\mathbf{y})^\top \boldsymbol{\eta}$ term), with all nonlinearity in $\boldsymbol{\eta}$ confined to the log-partition function $\log Z$.

Most common supervised learning losses arise from exponential families: the Gaussian gives MSE loss, the Bernoulli gives binary cross-entropy, and the Categorical gives softmax cross-entropy. In all these cases the network output $\boldsymbol{\phi}(\boldsymbol{\theta})$ plays the role of the natural parameters $\boldsymbol{\eta}$. The negative log-likelihood is then $\mathcal{L} = -\log p$:

$$\mathcal{L} = \underbrace{-\log h(\mathbf{y})}_{\text{const in } \boldsymbol{\phi}} - \underbrace{T(\mathbf{y})^\top \boldsymbol{\phi}}_{\text{linear in } \boldsymbol{\phi}} + \underbrace{\log Z(\boldsymbol{\phi})}_{\text{nonlinear in } \boldsymbol{\phi}}$$

Differentiating with respect to the model output $\boldsymbol{\phi}$:

$$\begin{aligned} \nabla_{\boldsymbol{\phi}} \mathcal{L} &= -T(\mathbf{y}) + \nabla_{\boldsymbol{\phi}} \log Z(\boldsymbol{\phi}) \qquad \text{(first term vanishes, linear term gives } -T(\mathbf{y})\text{)} \\ \mathbf{H}_{\boldsymbol{\phi}} &= \nabla^2_{\boldsymbol{\phi}} \log Z(\boldsymbol{\phi}) \qquad\qquad (-T(\mathbf{y}) \text{ is constant in } \boldsymbol{\phi}\text{, drops out}) \end{aligned}$$

The second derivative depends only on $\boldsymbol{\phi}$ through the partition function Z — the data $\mathbf{y}$ has disappeared entirely. Hence $\mathbf{H}_{\boldsymbol{\phi}}$ is independent of $\mathbf{y}$, and $\mathbf{G} = \mathbf{F}$ exactly by Theorem 49. This covers the losses most common in supervised learning:

- **MSE** (Gaussian): $\mathbf{H}_{\boldsymbol{\phi}} = \frac{1}{\sigma^2}\mathbf{I}$, constant in $\mathbf{y}$.
- **Cross-entropy / Bernoulli**: $\mathbf{H}_{\boldsymbol{\phi}} = \operatorname{diag}(\sigma(\boldsymbol{\phi}) \odot (1 - \sigma(\boldsymbol{\phi})))$.
- **Categorical / Softmax**: $\mathbf{H}_{\boldsymbol{\phi}} = \operatorname{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$.

The equivalence needs a likelihood whose natural parameter is the network output. For other objectives, such as mixture likelihoods, contrastive losses (InfoNCE) or the surrogate objectives of reinforcement learning, $\mathbf{G} \neq \mathbf{F}$, and which of the two is the right curvature depends on the problem. For a policy-gradient surrogate the output-space Hessian is indefinite, so its GGN is not even PSD, while the Fisher of the policy remains a valid metric; this is the case SOFO-NPG is built on (§5 of the SOFO notes).

16.4 Natural Gradient Descent

Set $\mathbf{B}_k = \frac{1}{\alpha}\mathbf{F}(\boldsymbol{\theta}_k)$, where $\mathbf{F}$ is the Fisher information matrix (Definition 22). When $f(\boldsymbol{\theta})$ depends on a probability distribution $p_{\boldsymbol{\theta}}$ — as in maximum-likelihood training, where $f(\boldsymbol{\theta}) = -\sum_n \log p_{\boldsymbol{\theta}}(\mathbf{y}_n|\mathbf{x}_n)$ — the isotropic penalty $\frac{1}{2\alpha}\|\mathbf{d}\|^2$ used by gradient descent is inappropriate. Some parameter directions change $p_{\boldsymbol{\theta}}$ drastically while others barely affect it, but the Euclidean penalty treats them identically. This is the same ill-conditioning that plagues gradient descent (Section I), except now manifested in the *distributional* geometry of the model rather than the curvature of f itself.

The Fisher matrix measures exactly this distributional sensitivity: by Proposition 48,

$$\frac{1}{2}\,\mathbf{d}^\top \mathbf{F}(\boldsymbol{\theta})\,\mathbf{d} \;\approx\; D_{\mathrm{KL}}(p_{\boldsymbol{\theta}} \,\|\, p_{\boldsymbol{\theta}+\mathbf{d}}).$$

Replacing the Euclidean penalty with a Fisher penalty gives the local model:

$$M_k(\mathbf{d}) \;=\; f(\boldsymbol{\theta}_k) + \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)^\top \mathbf{d} + \frac{1}{2\alpha}\, \mathbf{d}^\top \mathbf{F}(\boldsymbol{\theta}_k)\, \mathbf{d}$$

The penalty now measures KL divergence rather than Euclidean distance: a step that causes large distributional change is heavily penalised, while a step that barely affects $p_{\boldsymbol{\theta}}$ is left free — regardless of its Euclidean length. From (28):

$$\boldsymbol{\theta}_{k+1} \;\leftarrow\; \boldsymbol{\theta}_k \;-\; \alpha\, \mathbf{F}(\boldsymbol{\theta}_k)^{-1}\, \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k) \tag{90}$$

The quantity $\tilde{\nabla} f \coloneqq \mathbf{F}^{-1} \nabla_{\boldsymbol{\theta}} f$ is the **natural gradient** [34]. Geometrically, it is the direction of steepest descent on the Riemannian manifold of distributions (where distance is measured by the KL divergence), rather than in the Euclidean parameter space.

The three methods are unified through the local model (27):

Method	$\mathbf{B}_k$	Penalty measures	Update direction
Gradient descent	$\frac{1}{\alpha}\mathbf{I}$	Euclidean distance	$-\alpha\, \nabla_{\boldsymbol{\theta}} f$
Newton	$\mathbf{H}_{\boldsymbol{\theta}}$	Objective curvature	$-\mathbf{H}_{\boldsymbol{\theta}}^{-1} \nabla_{\boldsymbol{\theta}} f$
Natural gradient	$\frac{1}{\alpha}\mathbf{F}$	KL divergence	$-\alpha\, \mathbf{F}^{-1} \nabla_{\boldsymbol{\theta}} f$

REPARAMETERISATION INVARIANCE. The natural gradient is invariant to smooth invertible reparameterisations $\boldsymbol{\theta} = \zeta(\gamma)$: the direction in distribution space is unchanged, because $\mathbf{F}$ transforms as a covariant 2-tensor ($\mathbf{F}_\gamma = \mathbf{J}_\zeta^\top \mathbf{F}_{\boldsymbol{\theta}} \mathbf{J}_\zeta$) and the Jacobian factors cancel in $\mathbf{F}_\gamma^{-1} \nabla_\gamma f$ [10]. Neither gradient descent (tied to the Euclidean metric) nor Newton's method (the Hessian does *not* transform as $\mathbf{J}^\top \mathbf{H} \mathbf{J}$ in general) possess this property.

16.5 Interpretations of Natural Gradient

THE GREEDY STEP IN DISTRIBUTION SPACE. Natural gradient can be derived as the best step whose effect on the model's distribution is small [13, §6.4]. Linearise f and measure the step by the KL divergence, which by Proposition 48 is locally $\frac{1}{2}\mathbf{d}^\top \mathbf{F} \mathbf{d}$:

$$\min_{\mathbf{d}}\; \mathbf{g}^\top \mathbf{d} \quad \text{s.t.} \quad \tfrac{1}{2}\, \mathbf{d}^\top \mathbf{F}\, \mathbf{d} \le \eta^2, \qquad \mathbf{g} \coloneqq \nabla f(\boldsymbol{\theta}_k).$$

This is a convex problem, so the KKT conditions (Theorem 19) characterise its solution. With multiplier $\nu \ge 0$:

$$\begin{aligned}
&\mathbf{g} + \nu\, \mathbf{F}\mathbf{d} = \mathbf{0} \;\Longrightarrow\; \mathbf{d} = -\tfrac{1}{\nu}\, \mathbf{F}^{-1}\mathbf{g} && \text{(stationarity; } \nu > 0 \text{ since } \mathbf{g} \neq \mathbf{0}\text{)} \\
&\tfrac{1}{2}\, \mathbf{d}^\top \mathbf{F}\mathbf{d} = \eta^2 \;\Longrightarrow\; \frac{\mathbf{g}^\top \mathbf{F}^{-1} \mathbf{g}}{2\nu^2} = \eta^2 && \text{(complementary slackness: the constraint is active)} \\
&\qquad\Longrightarrow\; \mathbf{d}^* = -\eta \sqrt{\frac{2}{\mathbf{g}^\top \mathbf{F}^{-1} \mathbf{g}}}\; \mathbf{F}^{-1}\mathbf{g}.
\end{aligned}$$

The direction is the natural gradient, and the multiplier only sets its length; writing $\alpha = 1/\nu$ recovers the update (90). This is also the step TRPO takes (§9 of the RL notes). As $\eta \to 0$ the KL ball shrinks to a neighbourhood that depends only on the distributions, not on how they are parameterised, so the natural gradient is the steepest-descent direction in the space of distributions. With the KL divergence acting as a local squared length, the Fisher plays the role of a Riemannian metric, and following the natural gradient with infinitesimal steps traces a path through distribution space that does not depend on the parameterisation [10, 34].

GRADIENT DESCENT IN WHITENED COORDINATES. For a fixed $\mathbf{F} \succ 0$, the change of variables $\boldsymbol{\varphi} = \mathbf{F}^{1/2} \boldsymbol{\theta}$ turns the Fisher norm into the Euclidean one, $\|\mathbf{F}^{1/2}\mathbf{d}\|^2 = \mathbf{d}^\top \mathbf{F} \mathbf{d}$ [35, §6.4.2]. Gradient descent in $\boldsymbol{\varphi}$ is natural gradient descent in $\boldsymbol{\theta}$:

$$\begin{aligned}
\boldsymbol{\varphi}_{k+1} &= \boldsymbol{\varphi}_k - \alpha\, \nabla_{\boldsymbol{\varphi}} f = \boldsymbol{\varphi}_k - \alpha\, \mathbf{F}^{-1/2}\, \nabla_{\boldsymbol{\theta}} f && \text{(chain rule, } \boldsymbol{\theta} = \mathbf{F}^{-1/2}\boldsymbol{\varphi}\text{)} \\
\boldsymbol{\theta}_{k+1} &= \mathbf{F}^{-1/2} \boldsymbol{\varphi}_{k+1} = \boldsymbol{\theta}_k - \alpha\, \mathbf{F}^{-1}\, \nabla_{\boldsymbol{\theta}} f && \text{(map back)}
\end{aligned}$$

A small example shows the effect [35, §6.4.3]. Let $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}(\boldsymbol{\theta}), \mathbf{I})$ in $\mathbb{R}^2$ with the coupled mean $\boldsymbol{\mu}(\boldsymbol{\theta}) = (3\theta_1 + \frac{1}{3}\theta_2,\ \frac{1}{3}\theta_1)$, fitted to data from $\boldsymbol{\theta} = \mathbf{0}$ by minimising the expected negative log-likelihood. The mean is linear in $\boldsymbol{\theta}$ with Jacobian $\mathbf{J}$, so

$$f(\boldsymbol{\theta}) = \tfrac{1}{2}\, \boldsymbol{\theta}^\top \mathbf{J}^\top \mathbf{J}\, \boldsymbol{\theta} + \text{const}, \qquad \mathbf{F} = \mathbf{J}^\top \mathbf{J} = \begin{bmatrix} 9 + \frac{1}{9} & 1 \\ 1 & \frac{1}{9} \end{bmatrix}.$$

The eigenvalues of $\mathbf{F}$ are about 9.22 and 0.0013, a condition number near 7000, so gradient descent crawls along the low-curvature direction. Here $\mathbf{F}$ is also the Hessian of f, so natural gradient with $\alpha = 1$ is Newton's method and reaches the optimum in one step.

FISHER VERSUS HESSIAN. For the negative log-likelihood the two matrices are expectations of the same per-example loss Hessian under different distributions [10, §5][13, §6.4]:

$$\mathbf{F} \;=\; \mathbb{E}_{\mathbf{x}}\, \mathbb{E}_{\mathbf{y}\sim p_{\boldsymbol{\theta}}(\cdot|\mathbf{x})}\big[\nabla^2_{\boldsymbol{\theta}}\big(-\log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})\big)\big], \qquad \mathbf{H} \;=\; \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\text{data}}\big[\nabla^2_{\boldsymbol{\theta}}\big(-\log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})\big)\big].$$

The first equality is Lemma 47. When the model distribution matches the data, the two coincide, so near a well-specified optimum natural gradient behaves like Newton's method (compare Theorem 44). Combining Theorem 49 with the exponential-family computation of §16 gives the corresponding statement for the GGN.

Corollary 50 (NGD is a GGN method for exponential-family outputs)**.** *If the network output $\boldsymbol{\phi}$ is the natural parameter of an exponential-family likelihood and $\mathcal{L} = -\log p$, then $\mathbf{F} = \mathbf{G}$, and the natural-gradient step (90) with $\alpha = 1$ is the Gauss–Newton step (83): it minimises the GGN quadratic model of f.*

DAMPING IS ESSENTIAL. The steepest-descent view describes infinitesimal steps. A practical method takes finite steps along straight lines in $\boldsymbol{\theta}$, and these curve in distribution space, so the natural gradient direction can stop being a good direction long before the step ends [10, §10]. Corollary 50 gives the fix: natural gradient is a second-order method in disguise, minimising a quadratic model, and it should be damped like one, with Tikhonov damping and the Levenberg–Marquardt rule of §14. Hessian-free training and KFAC both apply natural-gradient-like updates with step size one only because of this damping [28, 36].

16.6 The Empirical Fisher and Why It Fails

Write the training loss as $f(\boldsymbol{\theta}) = \sum_{n=1}^{N} \ell_n(\boldsymbol{\theta})$ with $\ell_n = -\log p_{\boldsymbol{\theta}}(\mathbf{y}_n|\mathbf{x}_n)$. The *empirical Fisher* replaces the model's samples in (86) by the training labels [37]:

$$\tilde{\mathbf{F}}(\boldsymbol{\theta}) \;=\; \sum_{n=1}^{N} \nabla\ell_n\, \nabla\ell_n^{\top}, \qquad \text{whereas} \qquad \mathbf{F}(\boldsymbol{\theta}) \;=\; \sum_{n=1}^{N} \mathbb{E}_{\mathbf{y}\sim p_{\boldsymbol{\theta}}(\cdot|\mathbf{x}_n)}\big[\nabla \log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x}_n)\, \nabla \log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x}_n)^{\top}\big]. \tag{91}$$

Despite the name, $\tilde{\mathbf{F}}$ is *not a Monte Carlo estimate* of $\mathbf{F}$: the labels $\mathbf{y}_n$ come from the data, not from the model. The terminology also differs between communities. Statisticians call $\mathbf{F}$ itself the empirical Fisher, because its inputs $\mathbf{x}_n$ are empirical; we follow the machine-learning usage, where "empirical" means that both inputs and outputs come from the data. The empirical Fisher is popular because it needs only per-example gradients. Two arguments are used to justify it, and both fail.

"THE EMPIRICAL FISHER IS A GGN." For a likelihood loss, split $\ell = a(b(\boldsymbol{\theta}))$ with the model output $b = p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x})$ and the convex loss $a(b) = -\log b$. Since $a''(b) = 1/b^2 = a'(b)^2$, the GGN of this split is $\sum_n \nabla b_n\, a''(b_n)\, \nabla b_n^{\top} = \sum_n \nabla\ell_n \nabla\ell_n^{\top} = \tilde{\mathbf{F}}$ [13, (6.17)]. The identity is correct, but this split does not give a useful GGN [37, §4.1]. The GGN approximates the Hessian well when the loss gradient, the generalised residual, is small (§15); here $a'(b) = -1/b$ never vanishes. For least squares with scalar model output $\phi_n = \phi(\mathbf{x}_n, \boldsymbol{\theta})$, $\ell_n = \frac{1}{2}(\phi_n - y_n)^2$ and residual $r_n = \phi_n - y_n$, the contrast is explicit:

$$\begin{aligned} &\tilde{\mathbf{F}} = \textstyle\sum_n r_n^2\, \nabla\phi_n \nabla\phi_n^{\top} \quad && \text{(since } \nabla\ell_n = r_n \nabla\phi_n\text{)} \\ &\mathbf{F} = \textstyle\sum_n \nabla\phi_n \nabla\phi_n^{\top} && \text{(classical Gauss–Newton, §15)} \end{aligned}$$

As the fit improves, $\tilde{\mathbf{F}} \to \mathbf{0}$ while $\mathbf{F}$ approaches the Hessian. The second argument, that $\tilde{\mathbf{F}} \to \mathbf{F}$ near a minimum, needs a well-specified model and many more data than parameters [37, §4.2]. Overparameterised networks fit their training data, the per-example gradients vanish, and $\tilde{\mathbf{F}}$ vanishes with them while the curvature does not.

A SCALAR COUNTEREXAMPLE. Martens gives the simplest failure [10, §11.1]. Take one parameter, model output $\phi(x, \theta) = \theta$, a unit-variance Gaussian likelihood and a single training pair $(x, y) = (0, 0)$, so that $f(\theta) = \frac{1}{2}\theta^2$, $\nabla f = \theta$, $\tilde{F} = \theta^2$ and $F = 1$. Preconditioning with $\tilde{F}^{\xi}$ for some $\frac{1}{2} \le \xi \le 1$ gives

$$\theta_{k+1} = \theta_k - \alpha_k\, (\theta_k^2)^{-\xi}\, \theta_k = \big(1 - \alpha_k |\theta_k|^{-2\xi}\big)\, \theta_k,$$

whose effective step blows up as $\theta_k \to 0$: it fails to converge unless $\xi < 1$ and $\alpha_k \to 0$ quickly, and even then converges only as fast as α_k decays. The true Fisher gives $\theta_{k+1} = (1 - \alpha)\, \theta_k$, linear convergence for any $\alpha \in (0, 2)$.

WHAT THE EMPIRICAL FISHER MEASURES. If $\mathbf{g} = N\nabla\ell_n$ is the stochastic gradient for a uniformly sampled example, then $\mathbb{E}[\mathbf{g}] = \nabla f$ and $\mathbb{E}[\mathbf{g}\mathbf{g}^{\top}] = N \sum_n \nabla\ell_n \nabla\ell_n^{\top}$, so

$$N\, \tilde{\mathbf{F}} \;=\; \operatorname{Cov}[\mathbf{g}] + \nabla f\, \nabla f^{\top},$$

the second moment of the gradient noise [37, §5]. Preconditioning with it adapts to the noise in the gradient, not to the curvature. Adam and RMSProp use the square root of its diagonal (§12.5), which further cancels the

gradient's magnitude and brings them close to sign descent (§23.2); their success is evidence for noise adaptation, not for the empirical Fisher as a curvature matrix [10]. The true Fisher is no more expensive: sampling one label $\tilde{\mathbf{y}}_n \sim p_{\boldsymbol{\theta}}(\cdot|\mathbf{x}_n)$ and using $\nabla \log p_{\boldsymbol{\theta}}(\tilde{\mathbf{y}}_n|\mathbf{x}_n)$ in place of $\nabla \ell_n$ gives an unbiased estimate at the same cost [36]. In on-policy reinforcement learning the actions are sampled from the model, so the outer product of score functions is exactly this unbiased estimate (§5 of the SOFO notes).

16.7 Convergence: The Pre-Asymptotic Advantage

Does natural gradient converge faster than gradient descent in the stochastic setting? Martens answers this for a convex quadratic $h(\boldsymbol{\theta}) = \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^*)^\top \mathbf{H}(\boldsymbol{\theta} - \boldsymbol{\theta}^*)$ and the preconditioned stochastic iteration $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha\, \mathbf{B}^{-1}\mathbf{g}_k$ with unbiased gradients of covariance $\boldsymbol{\Sigma}_g$, where $\mathbf{B} = \mathbf{I}$ is SGD and $\mathbf{B} = \mathbf{H}$ an ideal second-order method [10, §14]. Because the gradient of a quadratic is affine, the mean iterate follows the deterministic iteration, $\mathbb{E}[\boldsymbol{\theta}_k] - \boldsymbol{\theta}^* = (\mathbf{I} - \alpha \mathbf{B}^{-1}\mathbf{H})^k(\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*)$, and the noise only spreads the iterates around it. With iterate averaging (§10.6) the expected gap of the averaged iterate splits into two terms [10, Thm. 6]. The noise term, $\operatorname{tr}(\mathbf{H}^{-1}\boldsymbol{\Sigma}_g)/(2(k+1))$, does not depend on $\mathbf{B}$: asymptotically, averaged SGD is as good as any preconditioner, which is the sense in which natural gradient is "Fisher efficient" [18, 34]. The whole advantage of curvature sits in the starting-point term, which decays like $1/(k+1)^2$ and, at the largest stable step, is larger for $\mathbf{B} = \mathbf{I}$ than for $\mathbf{B} = \mathbf{H}$ by a factor up to $\kappa(\mathbf{H})$ when $\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*$ lies along low-curvature directions. With a limited iteration budget, large batches (small $\boldsymbol{\Sigma}_g$) or early stopping, this pre-asymptotic term is the one that matters. The analysis assumes a fixed quadratic, so it says nothing about why the GGN beats the Hessian in practice or why damping matters [10, §15].

16.8 Approximating the Fisher

The exact Fisher is a $P \times P$ matrix and cannot be stored for a network. Practical methods take one of four routes [35, §6.4.4][13, §6.4].

- *Structured approximations:* diagonal [38], per-unit blocks [39], or per-layer Kronecker factors as in KFAC and Shampoo (§20).
- *Matrix-free solves:* keep the exact Fisher and solve $\mathbf{F}\mathbf{d} = -\mathbf{g}$ by CG with Fisher-vector products $\mathbf{J}^\top \mathbf{F}_L \mathbf{J}\mathbf{v}$, computed like GGN-vector products (Algorithm 2); this is Hessian-free optimisation (§19).
- *Unbiased estimators:* labels sampled from the model give an unbiased estimate of $\mathbf{F}$ (§16.6), and random vectors $\mathbf{v}$ with $\mathbb{E}[\mathbf{v}\mathbf{v}^\top] = \mathbf{I}$ give $\operatorname{diag}(\mathbf{G}) = \mathbb{E}\big[(\mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}}^{1/2}\mathbf{v})^{\odot 2}\big]$ [40].
- *Sketching:* project onto a random subspace, $\boldsymbol{\Theta}^\top \mathbf{F} \boldsymbol{\Theta}$ with $\boldsymbol{\Theta} \in \mathbb{R}^{P \times r}$, from r forward-mode products; this is what SOFO does with the GGN (§4 of the SOFO notes).

17 LOW-RANK STRUCTURE

The GGN of a network is far from full rank, and once damped its curvature matters along even fewer directions; this is why a few CG iterations, or a small random sketch as in SOFO, can capture it.

Lemma 51 (GGN rank bound [33])**.** $\operatorname{rank}(\mathbf{G}) \leq \min(P, MN)$*, where MN is the total number of scalar model outputs: M samples (or sequences) with N outputs each.*

Proof. From (82), $\mathbf{G} = \mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}} \mathbf{J}$ with $\mathbf{J} \in \mathbb{R}^{MN \times P}$, so $\operatorname{rank}(\mathbf{G}) \leq \operatorname{rank}(\mathbf{J}) \leq \min(P, MN)$. □

When $MN < P$ the curvature of $\mathbf{G}$ is zero along at least $P - MN$ directions. The bound is not always small, however. For the RNN of §1 of the SOFO notes each sequence has $N = DT$ outputs, so $MN = MDT$ grows with the horizon and can exceed P. For a damped method a smaller count is what matters.

Definition 23 (Effective rank)**.** For a damping $\gamma > 0$, the *effective rank* of $\mathbf{G}$ is the number of its eigenvalues above the damping,

$$s_\gamma \;:=\; \#\{j : \sigma_j(\mathbf{G}) > \gamma\} \;\leq\; \operatorname{rank}(\mathbf{G}).$$

In the eigenbasis $\mathbf{G} = \mathbf{U}\mathbf{S}\mathbf{U}^\top$, the damped step (78) has components $-\mathbf{u}_j^\top \mathbf{g}/(\sigma_j + \gamma)$. Along the directions with $\sigma_j \leq \gamma$,

$$\gamma \;\leq\; \sigma_j + \gamma \;\leq\; 2\gamma \quad\Longrightarrow\quad \frac{1}{2\gamma} \;\leq\; \frac{1}{\sigma_j + \gamma} \;\leq\; \frac{1}{\gamma},$$

so there the damped step is a plain gradient step of length between $1/(2\gamma)$ and $1/\gamma$, the same in every such direction. Curvature changes the step only along the s_γ directions with $\sigma_j > \gamma$, so an approximation of the damped GGN needs to resolve only those, and can treat the rest as $\gamma\mathbf{I}$. The measured spectra of §18.1 have a few large outliers, roughly one per class, that come from $\mathbf{G}$, and most of the remaining eigenvalues near zero, so s_γ

can be far smaller than both $\operatorname{rank}(\mathbf{G})$ and P. This is why CG on the damped GGN needs about $s_\gamma + 1$ iterations (§19.1); what a random sketch of $\mathbf{G}$ captures, and how SOFO's guarantees depend on r and the spectrum, is in §4.5 of the SOFO notes.

17.1 Conditioning and Convergence

The convergence rate of the preconditioned update (85) depends on the pseudo-condition number $\kappa(\mathbf{G}) = \lambda_{\max}/\lambda^{+}_{\min}$. Zhao et al. bound it for deep linear networks $\mathbf{W}_{n_\ell} \cdots \mathbf{W}_1 \mathbf{x}$ with input covariance $\mathbf{\Sigma}$ [41], and the bound has three architectural consequences.

- *Data normalisation.* The bound is proportional to $\kappa(\mathbf{\Sigma})$, so whitening the inputs improves conditioning.
- *Depth.* At fixed width the condition number at initialisation grows roughly quadratically with the number of layers n_ℓ in their experiments, and widening the layers in proportion to depth compensates.
- *Residual connections.* Replace each $\mathbf{W}_i$ by $\mathbf{W}_i + \beta\mathbf{I}$, and assume both that this shifts every singular value, $\sigma(\mathbf{W}_i + \beta\mathbf{I}) = \sigma(\mathbf{W}_i) + \beta$ (true when the left and right singular vectors of $\mathbf{W}_i$ coincide), and that consecutive layers' singular vectors align, which holds approximately at small initialisation. Each term of the bound then becomes a product of ratios, and for $\beta > 0$

$$\prod_{i \neq \ell} \Big(\frac{\sigma_{\max}(\mathbf{W}_i) + \beta}{\sigma_{\min}(\mathbf{W}_i) + \beta} \Big)^2 \; < \; \prod_{i \neq \ell} \Big(\frac{\sigma_{\max}(\mathbf{W}_i)}{\sigma_{\min}(\mathbf{W}_i)} \Big)^2, \qquad \text{since } \frac{a+\beta}{b+\beta} < \frac{a}{b} \text{ for } a > b > 0,$$

  so the bound decreases with β. This is a statement about an upper bound under these assumptions; Zhao et al. observe the same improvement in the actual condition number numerically. Ghorbani et al. report a different measurement for real residual networks: removing the skip connections shrinks the whole Hessian spectrum (§18.1), which is a statement about the scale of the eigenvalues rather than their ratio.

18 LOSS LANDSCAPE OF NEURAL NETWORKS

The convergence results of Parts II and III assume one smoothness constant and say little about the shape of the curvature. This section collects what is measured about the Hessian of real networks, which is the curvature SOFO sketches, and how gradient descent interacts with it.

18.1 Bulk and Outliers

The full Hessian spectrum of networks with tens of millions of parameters can be estimated from Hessian-vector products alone, with the stochastic Lanczos method [42, 43]. Across architectures and datasets it has the same three features (Figure 13).

- *Most eigenvalues are close to zero*, as the rank bound of Lemma 51 predicts when there are far more parameters than outputs.
- *A few large outliers* stand apart from the rest, roughly one per class in C-class classification.
- *A bulk of small eigenvalues of both signs.* Negative eigenvalues are large at initialisation, shrink within the first few hundred steps, but persist to the end of training [43].

Papyan attributes these features using the decomposition $\mathbf{H}_{\boldsymbol{\theta}} = \mathbf{G} + \mathbf{S}$ of (81), by computing the spectrum of each term separately [42]. The outliers appear at the same locations in the spectrum of $\mathbf{G}$ and not in that of $\mathbf{S}$, while the bulk comes mostly from $\mathbf{S}$, whose spectrum is nearly symmetric about zero and heavy-tailed rather than of a classical random-matrix form. The large curvature directions are therefore those of the PSD, low-rank GGN, and the indefinite part is the term $\mathbf{S}$ that the GGN drops (§15).

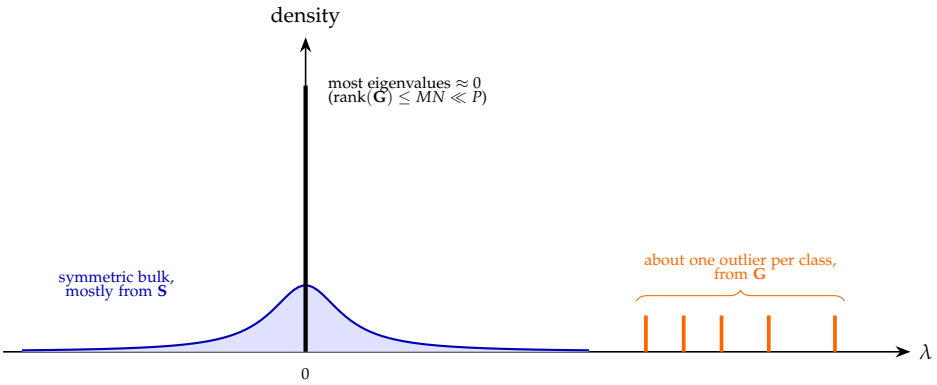


Figure 13: Schematic Hessian spectrum of a trained classifier [42, 43]. Most eigenvalues sit at zero; a few outliers, roughly one per class, come from the GGN term $\mathbf{G}$; a symmetric heavy-tailed bulk of small positive and negative eigenvalues comes mostly from the term $\mathbf{S}$ of (81).

WHY OUTLIERS SLOW TRAINING. Ghorbani et al. give two reasons [43, §4.1, App. E]. First, on a quadratic with eigenpairs $(\lambda_i, \mathbf{q}_i)$, $\lambda_1 \geq \lambda_2 \geq \cdots > 0$, gradient descent shrinks the error along $\mathbf{q}_i$ by $1 - \alpha\lambda_i$ per step, and the step must satisfy $\alpha < 2/\lambda_1$ (§18.2). Along directions with $\lambda_i \ll \lambda_1$ progress is then slow, and in a network without batch normalisation these flat directions carry about half of the Hessian's total eigenvalue mass at convergence. Second, stochastic gradients concentrate in the outlier directions. Model the minibatch loss as the quadratic with a random shift, $\hat{h}(\boldsymbol{\theta}) = \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^* + \mathbf{z})^\top \mathbf{H}(\boldsymbol{\theta} - \boldsymbol{\theta}^* + \mathbf{z})$ with $\mathbb{E}[\mathbf{z}] = \mathbf{0}$, $\mathbb{E}[\mathbf{z}\mathbf{z}^\top] = \boldsymbol{\Sigma}_z$ and independent $\mathbf{z}_k$ at each step. With $\mathbf{e}_k := \boldsymbol{\theta}_k - \boldsymbol{\theta}^*$, SGD and its unrolled error are

$$\begin{aligned} \boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k &= -\alpha\mathbf{H}(\mathbf{e}_k + \mathbf{z}_k), && \text{(gradient of } \hat{h}) \\ \mathbf{e}_k &= (\mathbf{I} - \alpha\mathbf{H})^k \mathbf{e}_0 - \alpha\mathbf{H} \sum_{j=0}^{k-1} (\mathbf{I} - \alpha\mathbf{H})^j \mathbf{z}_{k-1-j}. && \text{(unroll } \mathbf{e}_{k+1} = (\mathbf{I} - \alpha\mathbf{H})\mathbf{e}_k - \alpha\mathbf{H}\mathbf{z}_k) \end{aligned}$$

Project the step onto $\mathbf{q}_i$, write $s_i := \mathbf{q}_i^\top \boldsymbol{\Sigma}_z \mathbf{q}_i$, and let $k \to \infty$, so the term in $\mathbf{e}_0$ vanishes and only the independent noise terms survive in the second moment:

$$\begin{aligned} \mathbb{E}\big[(\mathbf{q}_i^\top(\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k))^2\big] &\to \alpha^2\lambda_i^2 \Big(\alpha^2\lambda_i^2 \sum_{j\geq 0} (1 - \alpha\lambda_i)^{2j} + 1\Big) s_i && (\mathbf{q}_i^\top \mathbf{H} = \lambda_i \mathbf{q}_i^\top, \text{ independent } \mathbf{z}_j) \\ &= \alpha^2\lambda_i^2 \Big(\frac{\alpha\lambda_i}{2 - \alpha\lambda_i} + 1\Big) s_i \;=\; \frac{2\alpha^2\lambda_i^2}{2 - \alpha\lambda_i} s_i. && (\text{geometric sum } \textstyle\sum_j (1 - \alpha\lambda_i)^{2j} = \frac{1}{\alpha\lambda_i(2 - \alpha\lambda_i)}) \end{aligned}$$

With $\alpha = c/\lambda_1$ for some $c < 1$, the energy of the steps along $\mathbf{q}_i$ scales like $(\lambda_i/\lambda_1)^2 s_i$: the updates concentrate quadratically on the outlier directions and hardly move along the flat ones, where much of the remaining loss lies. In networks without batch normalisation almost all of the gradient's energy is measured to lie in the top few Hessian eigendirections; batch normalisation suppresses the outliers and this concentration, and computing its statistics on the full dataset brings the outliers, and slow training, back while leaving λ_1 almost unchanged. So λ_1 alone does not measure how hard a problem is to optimise [43]. A curvature-aware step divides each eigendirection by its own λ_i and removes both effects. Ghorbani et al. also find that removing the residual connections of a ResNet contracts all its Hessian eigenvalues towards zero; this concerns the scale of the eigenvalues, whereas the bound of §17 concerns their ratio.

18.2 Sharpness and the Edge of Stability

The *sharpness* is the largest Hessian eigenvalue, $\lambda_{\max}(\mathbf{H})$. On a quadratic $f(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{A}\boldsymbol{\theta} - \mathbf{b}^\top\boldsymbol{\theta}$ with eigenpairs $(a_i, \mathbf{q}_i)$, $a_i > 0$, gradient descent is stable only if the sharpness is below $2/\alpha$ [44, §2]:

$$\begin{aligned} \mathbf{e}_{k+1} &= \mathbf{e}_k - \alpha\mathbf{A}\mathbf{e}_k = (\mathbf{I} - \alpha\mathbf{A})\,\mathbf{e}_k && (\nabla f = \mathbf{A}\mathbf{e} \text{ with } \mathbf{e} = \boldsymbol{\theta} - \boldsymbol{\theta}^*) \\ \mathbf{q}_i^\top \mathbf{e}_k &= (1 - \alpha a_i)^k\, \mathbf{q}_i^\top \mathbf{e}_0 && \text{(eigenbasis: each coordinate evolves alone)} \\ |1 - \alpha a_i| < 1 &\iff a_i < 2/\alpha. \end{aligned}$$

If $a_i > 2/\alpha$ the factor $1 - \alpha a_i < -1$, and the iterates oscillate along $\mathbf{q}_i$ with growing amplitude. The threshold is twice the step $1/L$ of Lemma 23, which also guarantees a decrease for every $\alpha < 2/L$. The same argument applied to heavy ball (40) gives the threshold $(2 + 2\beta)/\alpha$ [44, App. B].

For a neural network the quadratic is the local Taylor model, so gradient descent with step α is unstable wherever the sharpness exceeds $2/\alpha$. Cohen et al. find that full-batch gradient descent on real networks behaves in two phases (Figure 14) [44].

- *Progressive sharpening.* While it is below $2/\alpha$, the sharpness keeps rising. It rises more for deeper and narrower networks and larger datasets, and very little for very wide networks, whose Hessian barely moves during training.
- *Edge of stability.* Once the sharpness reaches $2/\alpha$, the iterates start oscillating along the top eigenvector, as on the quadratic, but gradient descent does not diverge. The sharpness then hovers at or just above $2/\alpha$, and the loss keeps decreasing over long timescales while going up and down from step to step. Lowering α restarts the sharpening until the sharpness meets the new $2/\alpha$, and with momentum the plateau is at the momentum threshold.

Step sizes small enough to avoid this regime exist, but on standard architectures they are impractically small: a VGG trained on CIFAR-10 with a stable step would need about fifty times as many iterations as at a step that enters the edge of stability [44, §4].

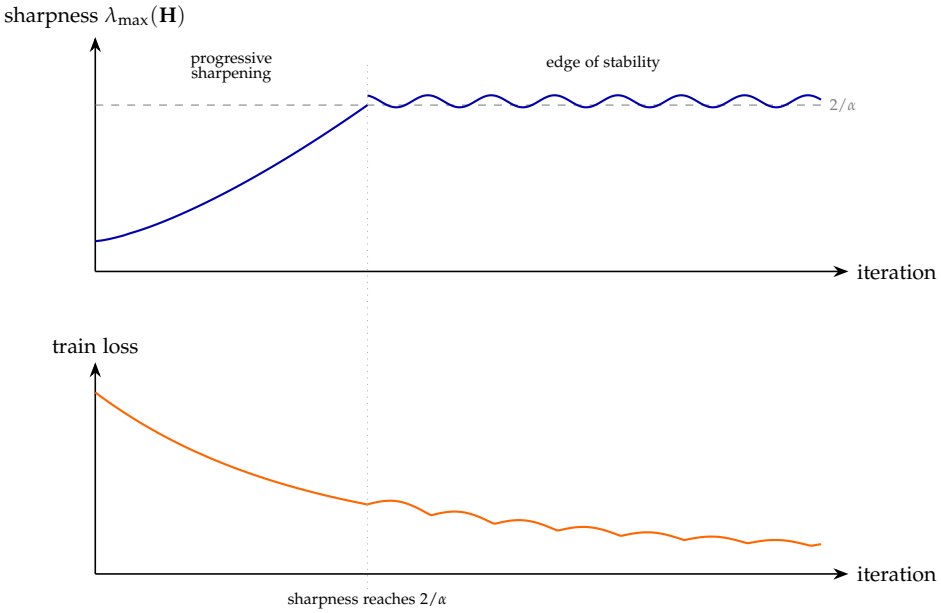


Figure 14: Schematic of full-batch gradient descent at step α on a neural network [44]. The sharpness rises until it reaches the stability threshold $2/\alpha$ of a quadratic, then hovers just above it. Before that point the loss decreases monotonically; afterwards it still decreases over long timescales, but not from step to step.

This regime is outside the theory of Parts II and III. Any analysis that assumes L-smoothness along the trajectory with a step below $2/L$, including the templates of §6 and the monotone-decrease result for clipping (§12.1), cannot describe gradient descent at practical step sizes. The local quadratic model also fails there: gradient descent run on the quadratic Taylor model at an edge-of-stability iterate diverges within a few steps, while on the real objective it keeps making progress, so the dynamics are inherently non-quadratic [44, §5, App. E]. For the same reason the classical rule $\alpha_k = 1/\lambda_{\max}(\mathbf{H}_k)$ does worse than a fixed step: progressive sharpening makes it shrink the step continually. For SGD the sharpness does not settle at a predictable value, but larger steps and smaller batches lead to flatter regions [44, §6]. For second-order methods the lesson is the one of §14: the quadratic model is trustworthy only locally, and damping is what keeps the step inside the region where it is.

CROSS-ENTROPY AT THE END OF TRAINING. With cross-entropy the sharpness typically falls near the end of training [44, App. C]. The top eigenvalue of the Hessian is close to that of the GGN $\mathbf{J}^\top \mathbf{H}_{\boldsymbol{\phi}} \mathbf{J}$, with $\mathbf{H}_{\boldsymbol{\phi}} = \operatorname{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$ per example (§16). Once examples are classified with large margins, $\mathbf{p}$ approaches a one-hot vector, so the entries $p_c(1-p_c)$ and $-p_c p_{c'}$ of $\mathbf{H}_{\boldsymbol{\phi}}$ go to zero, and the sharpness falls even though $\mathbf{J}^\top \mathbf{J}$ keeps growing.

18.3 PL in Over-Parameterised Networks

Section 3.4 argued that PL, rather than convexity, is the right regularity assumption for neural networks. Liu, Zhu and Belkin make this precise for the square loss [45]. Stack the n scalar outputs of the model on the training set into $\mathbf{F}(\boldsymbol{\theta}) \in \mathbb{R}^n$, with Jacobian $\mathbf{J} \in \mathbb{R}^{n \times P}$, residual $\mathbf{r} = \mathbf{F}(\boldsymbol{\theta}) - \mathbf{y}$ and loss $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\|\mathbf{r}\|^2$, and define the *tangent kernel* $\mathbf{K} := \mathbf{J}\mathbf{J}^\top \in \mathbb{R}^{n \times n}$. Then

$$\nabla \mathcal{L} = \mathbf{J}^\top \mathbf{r} \qquad \text{(chain rule)}$$

$$\|\nabla \mathcal{L}\|^2 = \mathbf{r}^\top \mathbf{J}\mathbf{J}^\top \mathbf{r} = \mathbf{r}^\top \mathbf{K}\, \mathbf{r} \;\geq\; \lambda_{\min}(\mathbf{K})\, \|\mathbf{r}\|^2 = 2\lambda_{\min}(\mathbf{K})\, \mathcal{L}, \qquad \text{(Rayleigh bound (8))}$$

which is the PL inequality (20) with $f^* = 0$ and $\mu = \lambda_{\min}(\mathbf{K})$ wherever $\lambda_{\min}(\mathbf{K}) \geq \mu > 0$. By Lemma 5, the nonzero eigenvalues of $\mathbf{K} = \mathbf{J}\mathbf{J}^\top$ are those of $\mathbf{J}^\top\mathbf{J}$, the GGN of the square loss. $\mathbf{K}$ can only be positive definite if $\mathbf{J}$ has full row rank n, which needs $P \geq n$, so only over-parameterised models can satisfy PL in this way, and the condition says that the nonzero eigenvalues of the GGN stay away from zero while its $P - n$ zero eigenvalues are unconstrained, as in the picture of §3.4.

Over-parameterisation also explains why convexity is the wrong framework. The global minimisers $\{\boldsymbol{\theta} : \mathbf{F}(\boldsymbol{\theta}) = \mathbf{y}\}$ generically form a curved manifold of dimension $P - n$, and a function that is convex on a neighbourhood has a convex set of minimisers there, so $\mathcal{L}$ is not convex in any neighbourhood of a global minimiser [45, §3]. Liu et al. then show that sufficiently wide networks satisfy PL in a ball around a random initialisation, because the Hessian of the network output is small there, so $\mathbf{K}$ hardly changes from its well-conditioned initial value; the ball contains a global minimiser, and gradient descent with a small enough fixed step converges to it linearly, as in Theorem 27 [45, §4, §5].

19 HESSIAN-FREE METHODS

A Newton-type step solves the linear system $\mathbf{B}\mathbf{p} = -\mathbf{g}$, which costs $O(P^2)$ memory and $O(P^3)$ time when $\mathbf{B}$ is formed explicitly. *Hessian-free* methods, also called truncated Newton or Newton-CG, never form $\mathbf{B}$. They solve the system approximately with the conjugate gradient method, which touches $\mathbf{B}$ only through matrix-vector products $\mathbf{B}\mathbf{v}$, and these cost a few gradient evaluations (§5.5) [13, 28, 46].

19.1 Conjugate Gradient

Conjugate gradient (CG) minimises a convex quadratic

$$q(\mathbf{x}) \;=\; \tfrac{1}{2}\,\mathbf{x}^\top\mathbf{A}\mathbf{x} - \mathbf{b}^\top\mathbf{x}, \qquad \mathbf{A} \succ 0,$$

or equivalently solves $\mathbf{A}\mathbf{x} = \mathbf{b}$, since $\nabla q(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}$. Gradient descent on q zig-zags when $\mathbf{A}$ is ill-conditioned (§7.3); the eigenvectors of $\mathbf{A}$ would be ideal search directions, but finding them costs as much as solving the system. CG uses directions that are orthogonal in the metric of $\mathbf{A}$ instead.

Definition 24 (Conjugate directions)**.** Nonzero vectors $\mathbf{d}_0, \dots, \mathbf{d}_{P-1}$ are $\mathbf{A}$-*conjugate* if $\mathbf{d}_i^\top\mathbf{A}\,\mathbf{d}_j = 0$ for all $i \neq j$.

Conjugate directions are linearly independent (if $\sum_j c_j\mathbf{d}_j = \mathbf{0}$, multiplying by $\mathbf{d}_i^\top\mathbf{A}$ leaves $c_i\,\mathbf{d}_i^\top\mathbf{A}\mathbf{d}_i = 0$, so $c_i = 0$), so P of them form a basis. In that basis the quadratic separates.

Proposition 52 (Conjugacy decouples the quadratic)**.** *Let* $\mathbf{d}_0, \dots, \mathbf{d}_{P-1}$ *be* $\mathbf{A}$*-conjugate,* $\mathbf{S} = [\mathbf{d}_0 \,|\, \cdots \,|\, \mathbf{d}_{P-1}]$*, and* $\mathbf{r}_0 = \mathbf{A}\mathbf{x}_0 - \mathbf{b}$*. Then for* $\mathbf{x} = \mathbf{x}_0 + \mathbf{S}\mathbf{y}$*,*

$$q(\mathbf{x}_0 + \mathbf{S}\mathbf{y}) \;=\; q(\mathbf{x}_0) + \sum_{i=0}^{P-1}\Big((\mathbf{d}_i^\top\mathbf{r}_0)\,y_i + \tfrac{1}{2}\,(\mathbf{d}_i^\top\mathbf{A}\mathbf{d}_i)\,y_i^2\Big), \tag{92}$$

a sum of independent one-dimensional quadratics. Consequently, exact line searches along $\mathbf{d}_0, \mathbf{d}_1, \dots$ *in turn reach the minimiser* $\mathbf{A}^{-1}\mathbf{b}$ *in at most* P *steps.*

Proof. Expand q around $\mathbf{x}_0$; the expansion is exact because q is quadratic:

$$\begin{aligned} q(\mathbf{x}_0 + \mathbf{S}\mathbf{y}) &= q(\mathbf{x}_0) + \mathbf{r}_0^\top\mathbf{S}\mathbf{y} + \tfrac{1}{2}\,\mathbf{y}^\top\mathbf{S}^\top\mathbf{A}\mathbf{S}\,\mathbf{y} && \text{(Taylor, } \nabla q(\mathbf{x}_0) = \mathbf{r}_0\text{)} \\ &= q(\mathbf{x}_0) + \sum_i (\mathbf{d}_i^\top\mathbf{r}_0)\,y_i + \tfrac{1}{2}\sum_i (\mathbf{d}_i^\top\mathbf{A}\mathbf{d}_i)\,y_i^2 && (\mathbf{S}^\top\mathbf{A}\mathbf{S} \text{ is diagonal by conjugacy}) \end{aligned}$$

Each y_i is minimised independently at $y_i^* = -\mathbf{d}_i^\top\mathbf{r}_0 / \mathbf{d}_i^\top\mathbf{A}\mathbf{d}_i$. For the sequential claim, let $\mathbf{x}_k = \mathbf{x}_0 + \sum_{i<k} y_i^*\mathbf{d}_i$ and $\mathbf{r}_k = \mathbf{A}\mathbf{x}_k - \mathbf{b}$. The exact line search along $\mathbf{d}_k$ from $\mathbf{x}_k$ picks $y_k = -\mathbf{d}_k^\top\mathbf{r}_k / \mathbf{d}_k^\top\mathbf{A}\mathbf{d}_k$ (§7.3), and

$$\mathbf{d}_k^\top\mathbf{r}_k \;=\; \mathbf{d}_k^\top\mathbf{r}_0 + \sum_{i<k} y_i^*\,\mathbf{d}_k^\top\mathbf{A}\mathbf{d}_i \;=\; \mathbf{d}_k^\top\mathbf{r}_0$$

by conjugacy, so $y_k = y_k^*$. After P steps every coordinate is at its minimiser, so $\mathbf{x}_P = \mathbf{A}^{-1}\mathbf{b}$. □

CG builds the conjugate directions on the fly, each from the current gradient and the previous direction, $\mathbf{d}_k = -\nabla q(\mathbf{x}_k) + \beta_k\mathbf{d}_{k-1}$, with β_k chosen to make $\mathbf{d}_k$ conjugate to $\mathbf{d}_{k-1}$. With exact line searches this choice makes $\mathbf{d}_k$ conjugate to *all* earlier directions, and β_k simplifies to a ratio of gradient norms (Fletcher–Reeves); the derivation is in the 3M1 notes [4]. Algorithm 5 uses one product $\mathbf{A}\mathbf{d}_k$ per iteration and stores only a few vectors.

Algorithm 5 Conjugate gradient for $\min_{\mathbf{x}} \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x}$

input: $\mathbf{x}_0$, a routine for $\mathbf{v} \mapsto \mathbf{A}\mathbf{v}$
$\mathbf{g}_0 \leftarrow \mathbf{A}\mathbf{x}_0 - \mathbf{b}$, $\mathbf{d}_0 \leftarrow -\mathbf{g}_0$
for $k = 0, 1, 2, \dots$ until $\|\mathbf{g}_k\|$ is small or a step limit is reached **do**
  $\alpha_k \leftarrow \|\mathbf{g}_k\|^2 / \mathbf{d}_k^\top \mathbf{A}\mathbf{d}_k$ ▷ *exact line search,* $= -\mathbf{g}_k^\top \mathbf{d}_k / \mathbf{d}_k^\top \mathbf{A}\mathbf{d}_k$
  $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{d}_k$
  $\mathbf{g}_{k+1} \leftarrow \mathbf{g}_k + \alpha_k \mathbf{A}\mathbf{d}_k$ ▷ *gradient update, reuses* $\mathbf{A}\mathbf{d}_k$
  $\beta_{k+1} \leftarrow \|\mathbf{g}_{k+1}\|^2 / \|\mathbf{g}_k\|^2$ ▷ *Fletcher–Reeves*
  $\mathbf{d}_{k+1} \leftarrow -\mathbf{g}_{k+1} + \beta_{k+1} \mathbf{d}_k$
end for
output: $\mathbf{x}_k$

By Proposition 52, CG terminates in at most P iterations in exact arithmetic. Long before that it converges linearly with the same $\sqrt{\kappa}$ dependence as heavy ball, $\|\mathbf{x}_k - \mathbf{x}^*\|_{\mathbf{A}} \le 2\big(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\big)^k \|\mathbf{x}_0 - \mathbf{x}^*\|_{\mathbf{A}}$ with $\kappa = \kappa(\mathbf{A})$ [11], against $\frac{\kappa-1}{\kappa+1}$ per step for gradient descent. This is no accident: CG is heavy ball with (α_k, β_k) re-optimised at every step (§8.1).

CG AS A SUBSPACE METHOD. Each direction $\mathbf{d}_j$ of Algorithm 5 is built from the current gradient and the previous direction, so by induction $\mathbf{d}_0, \dots, \mathbf{d}_{k-1}$ span the *Krylov subspace*

$$\mathcal{K}_k \;:=\; \operatorname{span}\{\mathbf{g}_0,\, \mathbf{A}\mathbf{g}_0,\, \dots,\, \mathbf{A}^{k-1}\mathbf{g}_0\}, \tag{93}$$

and the separation of Proposition 52 holds over every prefix of the directions, so the k-th iterate is the exact minimiser of q over all of it [11, Thms. 5.2, 5.3]:

$$\mathbf{x}_k \;=\; \arg \min_{\mathbf{x} \in \mathbf{x}_0 + \mathcal{K}_k} q(\mathbf{x}).$$

After k iterations CG has solved the quadratic model exactly within a k-dimensional subspace that it builds from the gradient and the curvature, one matrix-vector product per dimension. Because that subspace adapts to $\mathbf{A}$, the number of iterations depends on the spectrum rather than on P. If $\mathbf{A}$ has only m distinct eigenvalues, CG terminates in m iterations; if it has m large eigenvalues and the rest lie in an interval $[a, b]$, then after $m+1$ iterations the error $\|\mathbf{x} - \mathbf{x}^*\|_{\mathbf{A}}$ has shrunk by at least $(b-a)/(b+a)$: each large eigenvalue costs one iteration and then plays no further role [11, Thms. 5.4, 5.5]. For the damped GGN $\mathbf{A} = \mathbf{G} + \gamma\mathbf{I}$, every eigenvalue of $\mathbf{G}$ at most γ becomes one in $[\gamma, 2\gamma]$, so after $s_\gamma + 1$ iterations, with s_γ the effective rank of Definition 23, the error has shrunk by at least $(2\gamma - \gamma)/(2\gamma + \gamma) = 1/3$, whatever P is. The random-subspace methods of §3 of the SOFO notes also minimise the model exactly over a subspace, but draw it at random, independently of $\mathbf{g}_0$ and $\mathbf{A}$: it can be built in parallel, but it does not contain the gradient and does not adapt to the spectrum.

19.2 Inexact Newton with Conjugate Gradient

Applying CG to the damped model, with $\mathbf{A} = \mathbf{B} + \gamma\mathbf{I}$, $\mathbf{b} = -\mathbf{g}$ and $\mathbf{x}_0 = \mathbf{0}$, gives an approximate Newton step that needs only curvature-vector products [13, §6.1]. The first CG direction is $-\mathbf{g}$, so one CG iteration is a gradient step with the step length chosen by the model; each further iteration lowers the model value, and after enough iterations CG returns the full damped Newton step. The number of CG iterations therefore interpolates between gradient descent and Newton's method. In practice CG is truncated at a fixed maximum number of iterations or when its relative progress on M_k stalls [28, §4].

TRUNCATION AS DAMPING. CG started from $\mathbf{x}_0 = \mathbf{0}$ produces iterates of increasing norm, so stopping it when the iterate leaves the ball $\|\mathbf{x}\| \le \Delta$ (and, for an indefinite Hessian, at the first direction with $\mathbf{d}_k^\top \mathbf{B}\mathbf{d}_k \le 0$) solves the trust-region problem (75) approximately; this is Steihaug's method [47]. Early stopping damps even without a radius: CG reduces the error first along directions of large curvature, so a truncated solve under-steps exactly the low-curvature directions that Tikhonov damping suppresses (§14.2) [28, §8.6, §9].

PRECONDITIONING. Preconditioned CG runs CG in coordinates where the system is better conditioned, using an easily inverted $\mathbf{P} \approx \mathbf{A}$, typically a (damped, partly flattened) diagonal; $\mathbf{P} = \mathbf{A}$ would converge in one step [28, §11]. The curvature mini-batch must stay fixed during a CG run, since conjugacy is defined only for a fixed $\mathbf{A}$.

19.3 Hessian-Vector Products

The product $\mathbf{Hv}$ is the directional derivative of the gradient, so the finite difference

$$\mathbf{Hv} \approx \frac{\nabla f(\boldsymbol{\theta} + \varepsilon \mathbf{v}) - \nabla f(\boldsymbol{\theta})}{\varepsilon}$$

costs one extra gradient evaluation, but its truncation and rounding errors make it fragile for highly nonlinear networks [28, §5]. Automatic differentiation computes the same product exactly, as forward mode applied to the gradient computation (Algorithm 1, Pearlmutter's $\mathcal{R}$-operator [48]), at a cost of a few gradient evaluations. For Hessian-free training the GGN product of Algorithm 2 is preferred: it is cheaper, never nested, and always PSD.

19.4 Subsampled Hessians and Nonconvexity

Because the inexact Newton step tolerates noise in the curvature better than noise in the gradient, the curvature products can be computed on a smaller subsample than the gradient, which makes each CG iteration cheap [13, §6.1.1]. This only pays off when the gradient batch is already moderate to large; with very noisy gradients the curvature sample would have to be larger still, and plain SG is the better choice. Martens and Sutskever add a caution: if the gradient and curvature come from different samples, the gradient can have a component outside the range of the estimated curvature, and the undamped model is then unbounded below [28, §12.1]. For nonconvex f the subsampled Hessian may be indefinite; the classical remedies are a trust region with negative-curvature termination (above), or replacing the Hessian with the PSD Gauss-Newton matrix (§15), which is the choice that works for neural networks [13, §6.1.2].

QUASI-NEWTON METHODS. A different way to avoid the Hessian is to build an inverse-Hessian approximation from successive gradient differences, as BFGS and its limited-memory version L-BFGS do; in the stochastic setting differences of noisy gradients give poor curvature estimates, so these methods are rarely used to train large networks [13, §6.2].

20 KRONECKER-FACTORED METHODS

Despite the low-rank structure of §17, storing and inverting $\mathbf{G} \in \mathbb{R}^{P \times P}$ remains infeasible for any practical network. Traditional second-order methods (KFAC, KFLR, KFRA [32, 33, 36]) approximate $\mathbf{G}$ as block-diagonal — one block per layer — and factorise each block as a Kronecker product, enabling efficient inversion via $(\mathbf{A} \otimes \mathbf{B})^{-1} = \mathbf{A}^{-1} \otimes \mathbf{B}^{-1}$. This discards all inter-layer curvature information and introduces approximation error from the factorised expectation. SOFO takes a fundamentally different approach: rather than approximating $\mathbf{G}$ layer by layer, it projects the *full* $\mathbf{G}$ into a single global random r-dimensional subspace (Section 4). Shampoo and SOAP use the same Kronecker structure for a different curvature proxy, the gradient's second moment, and arrive at it from adaptive methods rather than from the Fisher.

20.1 Kronecker Product Toolkit

For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B}$, the Kronecker product $\mathbf{A} \otimes \mathbf{B}$ is the block matrix with blocks $a_{ij}\mathbf{B}$. We use the column-major vec, which stacks the columns of a matrix, so that $\operatorname{vec}(\mathbf{u}\mathbf{v}^\top) = \mathbf{v} \otimes \mathbf{u}$. The Shampoo paper stacks rows instead; its formulas are translated below.[22]

Lemma 53 (Kronecker identities)**.** *For matrices of compatible sizes:*

(i) $(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = \mathbf{AC} \otimes \mathbf{BD}$ *and* $(\mathbf{A} \otimes \mathbf{B})^\top = \mathbf{A}^\top \otimes \mathbf{B}^\top$;
(ii) for $\mathbf{A}, \mathbf{B} \succeq 0$ *and* $s \in \mathbb{R}$ *(with* $\mathbf{A}, \mathbf{B} \succ 0$ *if* $s < 0$*),* $(\mathbf{A} \otimes \mathbf{B})^s = \mathbf{A}^s \otimes \mathbf{B}^s$*; in particular* $(\mathbf{A} \otimes \mathbf{B})^{-1} = \mathbf{A}^{-1} \otimes \mathbf{B}^{-1}$;
(iii) $\operatorname{tr}(\mathbf{A} \otimes \mathbf{B}) = \operatorname{tr}(\mathbf{A}) \operatorname{tr}(\mathbf{B})$*, and if* $\mathbf{A} = \mathbf{U}_A \boldsymbol{\Lambda}_A \mathbf{U}_A^\top$ *and* $\mathbf{B} = \mathbf{U}_B \boldsymbol{\Lambda}_B \mathbf{U}_B^\top$ *are eigendecompositions then* $\mathbf{A} \otimes \mathbf{B} = (\mathbf{U}_A \otimes \mathbf{U}_B)(\boldsymbol{\Lambda}_A \otimes \boldsymbol{\Lambda}_B)(\mathbf{U}_A \otimes \mathbf{U}_B)^\top$ *is an eigendecomposition, with eigenvalues* $\lambda_i(\mathbf{A})\lambda_j(\mathbf{B})$;
(iv) $(\mathbf{B}^\top \otimes \mathbf{A}) \operatorname{vec}(\mathbf{X}) = \operatorname{vec}(\mathbf{AXB})$.

Proof. Everything follows from the mixed product (i) [33, 49]. Applied twice it gives the eigendecomposition in (iii), with $(\mathbf{U}_A \otimes \mathbf{U}_B)^\top (\mathbf{U}_A \otimes \mathbf{U}_B) = \mathbf{U}_A^\top \mathbf{U}_A \otimes \mathbf{U}_B^\top \mathbf{U}_B = \mathbf{I}$, and applying $x \mapsto x^s$ to its eigenvalues $\lambda_i(\mathbf{A})\lambda_j(\mathbf{B})$ gives (ii). For (iv), take first a rank-one $\mathbf{X} = \mathbf{u}\mathbf{v}^\top$:

$$\begin{aligned} \operatorname{vec}(\mathbf{A}\mathbf{u}\mathbf{v}^\top\mathbf{B}) &= \operatorname{vec}\big((\mathbf{A}\mathbf{u})(\mathbf{B}^\top\mathbf{v})^\top\big) = (\mathbf{B}^\top\mathbf{v}) \otimes (\mathbf{A}\mathbf{u}) && (\operatorname{vec}(\mathbf{a}\mathbf{b}^\top) = \mathbf{b} \otimes \mathbf{a}) \\ &= (\mathbf{B}^\top \otimes \mathbf{A})(\mathbf{v} \otimes \mathbf{u}) = (\mathbf{B}^\top \otimes \mathbf{A}) \operatorname{vec}(\mathbf{u}\mathbf{v}^\top) && (\text{mixed product (i)}) \end{aligned}$$

22 Under row-major vectorisation the identity of Lemma 53(iv) reads $(\mathbf{A} \otimes \mathbf{B}^\top) \operatorname{vec}(\mathbf{X}) = \operatorname{vec}(\mathbf{AXB})$, as in [49, Lemma 4], and every Kronecker product below has its factors swapped.

Every $\mathbf{X}$ is a sum of rank-one matrices and both sides are linear in $\mathbf{X}$. □

Property (ii) is what makes Kronecker-factored curvature practical: a $d_1 d_2 \times d_1 d_2$ matrix is inverted, or raised to any power, by working on its two factors of sizes d_1 and d_2, and by (iv) the result is applied to a vectorised matrix by two small matrix products.

20.2 KFAC

Consider a layer $\mathbf{s}_l = \mathbf{W}_l \bar{\mathbf{a}}_{l-1}$, where $\bar{\mathbf{a}}_{l-1}$ is the previous layer's activation with a constant 1 appended for the bias, and write $\mathbf{g}_l = \partial(-\log p_{\boldsymbol{\theta}}(\mathbf{y}|\mathbf{x}))/\partial \mathbf{s}_l$ for the backpropagated signal at the pre-activations [36]. Backpropagation gives the weight gradient as an outer product, $\nabla_{\mathbf{W}_l} = \mathbf{g}_l \bar{\mathbf{a}}_{l-1}^\top$.

Proposition 54 (Kronecker-factored Fisher block [36, §3])**.** *The Fisher block of layer l is*

$$\mathbf{F}_{ll} \;=\; \mathbb{E}\big[\bar{\mathbf{a}}_{l-1}\bar{\mathbf{a}}_{l-1}^\top \otimes \mathbf{g}_l \mathbf{g}_l^\top\big] \;\approx\; \underbrace{\mathbb{E}\big[\bar{\mathbf{a}}_{l-1}\bar{\mathbf{a}}_{l-1}^\top\big]}_{\bar{\mathbf{A}}_{l-1}} \otimes \underbrace{\mathbb{E}\big[\mathbf{g}_l \mathbf{g}_l^\top\big]}_{\boldsymbol{\Gamma}_l}, \tag{94}$$

where the expectation is over inputs from the training set and labels $\mathbf{y}$ sampled from the model, as in the true Fisher.

Proof. With $\operatorname{vec}(\mathbf{u}\mathbf{v}^\top) = \mathbf{v} \otimes \mathbf{u}$, $\operatorname{vec}(\nabla_{\mathbf{W}_l}) = \bar{\mathbf{a}}_{l-1} \otimes \mathbf{g}_l$, so by definition (86) and Lemma 53(i),

$$\mathbf{F}_{ll} = \mathbb{E}\big[(\bar{\mathbf{a}}_{l-1} \otimes \mathbf{g}_l)(\bar{\mathbf{a}}_{l-1} \otimes \mathbf{g}_l)^\top\big] = \mathbb{E}\big[(\bar{\mathbf{a}}_{l-1} \otimes \mathbf{g}_l)(\bar{\mathbf{a}}_{l-1}^\top \otimes \mathbf{g}_l^\top)\big] = \mathbb{E}\big[\bar{\mathbf{a}}_{l-1}\bar{\mathbf{a}}_{l-1}^\top \otimes \mathbf{g}_l \mathbf{g}_l^\top\big].$$

The approximation replaces the expectation of a Kronecker product by the Kronecker product of expectations. □

Entry by entry, the approximation treats products of two activations as independent of products of two backpropagated signals. It is not exact under any realistic assumption, but its error is governed by higher-order cumulants and it captures the coarse structure of the Fisher well [36, §3.1]. $\bar{\mathbf{A}}_{l-1}$ needs only the forward pass; $\boldsymbol{\Gamma}_l$ needs labels drawn from the model, one extra backward pass per mini-batch (with the training labels it would be the empirical Fisher, §16.6).

Proposition 55 (Block-diagonal KFAC update)**.** *Approximate $\mathbf{F}$ by the block-diagonal matrix with blocks $\bar{\mathbf{A}}_{l-1} \otimes \boldsymbol{\Gamma}_l$. For gradient blocks $\nabla_{\mathbf{W}_l} f$, the natural-gradient direction is, layer by layer,*

$$\boldsymbol{\Delta}_l \;=\; -\boldsymbol{\Gamma}_l^{-1}\,(\nabla_{\mathbf{W}_l} f)\,\bar{\mathbf{A}}_{l-1}^{-1}. \tag{95}$$

Proof. A block-diagonal matrix is inverted block by block. For one block, Lemma 53(ii) and then (iv), with the symmetric factors $\bar{\mathbf{A}}_{l-1}^{-1}$ and $\boldsymbol{\Gamma}_l^{-1}$:

$$(\bar{\mathbf{A}}_{l-1} \otimes \boldsymbol{\Gamma}_l)^{-1} \operatorname{vec}(\nabla_{\mathbf{W}_l} f) = (\bar{\mathbf{A}}_{l-1}^{-1} \otimes \boldsymbol{\Gamma}_l^{-1}) \operatorname{vec}(\nabla_{\mathbf{W}_l} f) = \operatorname{vec}\big(\boldsymbol{\Gamma}_l^{-1}(\nabla_{\mathbf{W}_l} f)\,\bar{\mathbf{A}}_{l-1}^{-1}\big). \quad \square$$

For a layer with d inputs and outputs, the Fisher block has d^4 entries, while KFAC stores and inverts two $d \times d$ matrices.

In practice [36, §6–§8]:

- *Whitening.* Without damping, the update is plain gradient descent on a reparameterised network whose activations and backpropagated signals are whitened by $\bar{\mathbf{A}}_{l-1}^{-1/2}$ and $\boldsymbol{\Gamma}_l^{-1/2}$; batch normalisation is a cheap diagonal version of the activation half (§21).
- *Damping.* $\bar{\mathbf{A}} \otimes \boldsymbol{\Gamma} + \gamma \mathbf{I}$ has no simple inverse, so KFAC damps each factor, $(\bar{\mathbf{A}} + \pi\sqrt{\gamma}\,\mathbf{I}) \otimes (\boldsymbol{\Gamma} + \frac{\sqrt{\gamma}}{\pi}\mathbf{I})$, with π balancing the two factors' average eigenvalues, then rescales the proposed step by minimising the quadratic model built with the exact Fisher on the current mini-batch.
- *Statistics.* $\bar{\mathbf{A}}_{l-1}$ and $\boldsymbol{\Gamma}_l$ are exponential moving averages over mini-batches and are inverted only occasionally, so each iteration costs a small multiple of an SGD step.
- *GGN versions.* KFLR and KFRA factor the GGN block instead, computing its backpropagated factor exactly when the output dimension is small or passing the expectation through the backward recursion [32].

E-KFAC rescales the update in the eigenbasis of the Kronecker factors [50], and ACKTR applies KFAC to the natural policy gradient in reinforcement learning [51].

20.3 Shampoo

Shampoo [49] uses the same two-sided Kronecker structure, but builds its factors from the gradient itself. For a weight matrix $\mathbf{W} \in \mathbb{R}^{m\times n}$ with stochastic gradient $\mathbf{G}_k := \nabla_{\mathbf{W}} f_k$, it accumulates left and right second moments and preconditions from both sides:

$$\mathbf{L}_k = \mathbf{L}_{k-1} + \mathbf{G}_k\mathbf{G}_k^\top, \qquad \mathbf{R}_k = \mathbf{R}_{k-1} + \mathbf{G}_k^\top\mathbf{G}_k, \qquad \mathbf{W}_k = \mathbf{W}_{k-1} - \eta\, \mathbf{L}_k^{-1/4}\,\mathbf{G}_k\,\mathbf{R}_k^{-1/4}, \tag{96}$$

with $\mathbf{L}_0 = \epsilon\mathbf{I}_m$ and $\mathbf{R}_0 = \epsilon\mathbf{I}_n$. (Here $\mathbf{G}_k$ is a gradient matrix, not the GGN.)

Proposition 56 (Shampoo is Kronecker-preconditioned gradient descent)**.** *In vectorised form the Shampoo update is* $\operatorname{vec}(\mathbf{W}_k) = \operatorname{vec}(\mathbf{W}_{k-1}) - \eta\,(\mathbf{R}_k^{1/4}\otimes\mathbf{L}_k^{1/4})^{-1}\operatorname{vec}(\mathbf{G}_k)$.

Proof. $\mathbf{L}_k$ and $\mathbf{R}_k$ are symmetric positive definite, so by Lemma 53(iv) and then (ii),

$$\operatorname{vec}\big(\mathbf{L}_k^{-1/4}\,\mathbf{G}_k\,\mathbf{R}_k^{-1/4}\big) = (\mathbf{R}_k^{-1/4}\otimes\mathbf{L}_k^{-1/4})\operatorname{vec}(\mathbf{G}_k) = (\mathbf{R}_k^{1/4}\otimes\mathbf{L}_k^{1/4})^{-1}\operatorname{vec}(\mathbf{G}_k). \qquad \square$$

Shampoo therefore preconditions with an $mn\times mn$ matrix while storing only m^2+n^2 numbers. The matrix it stands in for is full-matrix Adagrad (§12.3): if every gradient has rank at most r, the full-matrix Adagrad preconditioner is bounded by $\sqrt{r}\,(\mathbf{R}_k^{1/4}\otimes\mathbf{L}_k^{1/4})$, and the exponent $1/4$ on each side is what makes the two factors together supply Adagrad's square root [49, Lemma 8].

Shampoo and KFAC share the Kronecker form but not the statistics. KFAC factors the Fisher into activation and backpropagated-signal covariances, which needs the layer structure and labels sampled from the model; Shampoo factors the raw gradient's second moment, needs no sampling, and applies to any tensor-valued parameter [49, §1.2]. Without the accumulation, a single Shampoo step is $\mathbf{U}\mathbf{V}^\top$ for the SVD $\mathbf{G}_k = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, steepest descent under the spectral norm (§23.3), which Muon takes with momentum (§23.5); SOAP below reads practical Shampoo as Adafactor in a rotated basis.

20.4 SOAP

Practical Shampoo replaces the exponent $1/4$ by $1/2$ and rescales by a trace [52], and with these changes it turns out to be Adafactor in a rotated basis. *Adafactor* [27] replaces Adam's second-moment matrix $\mathbf{V}\in\mathbb{R}^{m\times n}$ by the rank-one matrix built from its row sums $\mathbf{a} = \mathbf{V}\mathbf{1}_n$ and column sums $\mathbf{c}^\top = \mathbf{1}_m^\top\mathbf{V}$,

$$\mathbf{V}' \;=\; \frac{\mathbf{a}\,\mathbf{c}^\top}{\mathbf{1}_m^\top\mathbf{a}}, \tag{97}$$

which stores $m+n$ numbers instead of mn and is exact when $\mathbf{V}$ is rank one. Vyas et al. show that Shampoo with exponent $1/2$, preconditioner $(\mathbf{R}\otimes\mathbf{L})/\operatorname{tr}(\mathbf{L})$ and dataset-averaged statistics $\mathbf{L} = \mathbb{E}[\mathbf{G}\mathbf{G}^\top]$, $\mathbf{R} = \mathbb{E}[\mathbf{G}^\top\mathbf{G}]$ equals Adafactor run on the rotated gradient $\mathbf{Q}_L^\top\mathbf{G}\,\mathbf{Q}_R$, where $\mathbf{Q}_L, \mathbf{Q}_R$ are the eigenvectors of $\mathbf{L}, \mathbf{R}$: in that basis both divide entry (i,j) by $\sqrt{\lambda_i(\mathbf{L})\lambda_j(\mathbf{R})/\operatorname{tr}(\mathbf{L})}$ [52, Claim 1].

SOAP (Shampoo with Adam in the Preconditioner's eigenbasis) keeps Shampoo's rotation but runs full Adam in the rotated coordinates, so the per-direction step sizes adapt at every step while the expensive eigenbasis is refreshed only every f steps; E-KFAC applies the same idea to KFAC [50, 52]. SOAP needs one hyperparameter beyond AdamW, the frequency f, trains in fewer iterations than both AdamW and Shampoo on language-model pre-training, and follows linear scaling to larger batches than AdamW, a higher critical batch size (§11.4) [52].

21 DIAGONAL SCALING AS CURVATURE

The cheapest curvature matrix is a diagonal one: it rescales each coordinate of the gradient and costs $O(P)$ to apply. This section relates the adaptive methods of §12 to the curvature story of this part [13, §6.5].

DIAGONAL GAUSS–NEWTON. Becker and LeCun estimate the diagonal of the Gauss–Newton matrix $\mathbf{J}^\top\mathbf{J}$ by a backpropagation-like pass, keep a running average $\mathbf{d}_k$ of it, and update $\theta_{k+1,i} = \theta_{k,i} - \alpha\, g_{k,i}/(d_{k,i}+\mu)$, where $\mu > 0$ guards against near-zero curvature [38]. Each step costs little more than an SG step but the speed-up is modest, so the method is best seen as a way to retune SG's step sizes periodically rather than as a full second-order method [13, §6.5]. The weakness is the diagonal itself: neural-network curvature is strongly non-diagonal, and an exact diagonal can even point in a worse direction than the gradient [13].

RESCALING WITHOUT CURVATURE. RMSProp and Adagrad (§12.4, §12.3) divide each coordinate by the root of a running average or sum of squared gradients. This equalises the second moment of each rescaled coordinate, so that every coordinate makes progress at a similar rate; it does not estimate curvature [13, §6.5]. In the language of §16.6, the running average of $\mathbf{g} \odot \mathbf{g}$ is the diagonal of the empirical Fisher, the second moment of the gradient noise, and the square root moves the update further from natural gradient, towards sign descent. Diagonal adaptive methods therefore adapt to gradient scale and noise, and the curvature-aware methods of this part are complementary to them rather than refinements.

BATCH NORMALISATION AS WHITENING. For a fully connected layer with augmented inputs $\hat{\mathbf{x}} = (\mathbf{x}, 1)$ and the weight row $\hat{\mathbf{w}}_r$ feeding unit r, the corresponding diagonal block of the GGN is [13, (6.25)]

$$\mathbf{G}_{[r]} = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left[\frac{\partial h}{\partial s_r}^{\top} \frac{\partial^2 \ell}{\partial h^2} \frac{\partial h}{\partial s_r} \right] \hat{\mathbf{x}}_i \hat{\mathbf{x}}_i^{\top},$$

a weighted second-moment matrix of the layer's inputs, where h denotes the rest of the network above unit r. It is perfectly conditioned when the weighted inputs are white, with zero mean and identity covariance. Dropping the bracketed weight and the cross-correlations between inputs leaves the much cheaper goal of giving each input zero mean and unit variance, which is what batch normalisation does with mini-batch statistics [53]. Batch normalisation thus improves the conditioning of the Gauss–Newton blocks by a structural change to the network rather than by a better optimiser, a diagonal cousin of KFAC's whitening of activations (§20.2).

Part V.
Steepest Descent Under Norms

22 STEEPEST DESCENT AS NORM CHOICE

Every method of Parts II and IV minimises the local model (27), whose penalty $\frac{1}{2}\mathbf{d}^\top \mathbf{B}_k \mathbf{d}$ measures the length of the step in a metric built from curvature. This part keeps the linear term and the penalty, but measures the step with an arbitrary norm chosen in advance, without estimating any Hessian [54]. Gradient descent, sign descent and Adam, Shampoo and Muon, and natural gradient then differ only in the choice of norm.

22.1 The Framework

Write $\mathbf{g} := \nabla f(\boldsymbol{\theta}_k)$. The *steepest-descent problem* under a norm $\|\cdot\|$ is the gradient-descent model of §7 with the Euclidean norm replaced:

$$\mathbf{d}^* \;=\; \arg\min_{\mathbf{d}} \; \mathbf{g}^\top \mathbf{d} + \frac{1}{2\alpha}\,\|\mathbf{d}\|^2. \tag{98}$$

Its solution is expressed through the *dual norm*, the largest first-order change in f along a unit step:

$$\|\mathbf{g}\|_\dagger \;:=\; \max_{\|\mathbf{t}\|=1} \; \mathbf{g}^\top \mathbf{t}. \tag{99}$$

The Euclidean norm is its own dual, and the dual of ℓ_∞ is ℓ_1 (§23.2).

Proposition 57 (Steepest descent under a norm [54, Prop. 1]). *For any norm $\|\cdot\|$ and $\alpha > 0$, a solution of (98) is*

$$\mathbf{d}^* \;=\; -\alpha\,\|\mathbf{g}\|_\dagger\,\mathbf{t}^*, \qquad \mathbf{t}^* \in \arg\max_{\|\mathbf{t}\|=1} \; \mathbf{g}^\top \mathbf{t}. \tag{100}$$

Proof. Split the step into a length $c \geq 0$ and a unit direction $\mathbf{t}$, so $\mathbf{d} = c\,\mathbf{t}$ and $\|\mathbf{d}\| = c$:

$$\begin{aligned}
\min_{\mathbf{d}} \Big[\mathbf{g}^\top \mathbf{d} + \tfrac{1}{2\alpha}\|\mathbf{d}\|^2\Big] &= \min_{c\geq 0}\Big[c \min_{\|\mathbf{t}\|=1} \mathbf{g}^\top \mathbf{t} + \tfrac{c^2}{2\alpha}\Big] \\
&= \min_{c\geq 0}\Big[-c\,\|\mathbf{g}\|_\dagger + \tfrac{c^2}{2\alpha}\Big]. \qquad \text{(minimum at } \mathbf{t} = -\mathbf{t}^*\text{; the unit sphere is symmetric)}
\end{aligned}$$

The last bracket is a scalar quadratic in c, minimised at $c = \alpha\,\|\mathbf{g}\|_\dagger$. Multiplying this length by the direction $-\mathbf{t}^*$ gives (100). □

The proposition splits the design of an optimizer into two choices. The norm alone fixes the direction $\mathbf{t}^*$, while the length $\alpha\,\|\mathbf{g}\|_\dagger$ grows with α and with the gradient measured in the dual norm (Figure 15).

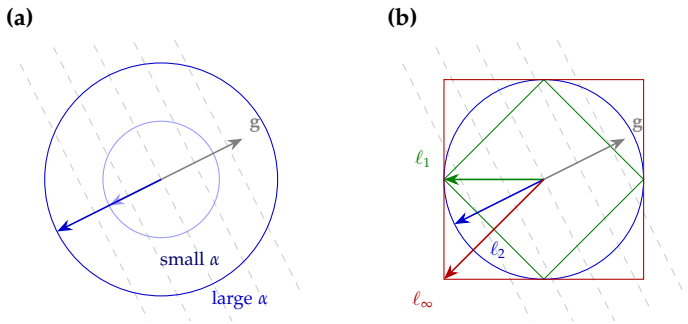


Figure 15: Steepest descent (98) for a gradient $\mathbf{g}$ (grey arrow); dashed lines are level sets of $\mathbf{g}^\top \mathbf{d}$. **(a)** Euclidean norm: the solution is the point minimising $\mathbf{g}^\top \mathbf{d}$ on the ball of radius $\alpha\,\|\mathbf{g}\|_\dagger$, so increasing α lengthens the step without turning it. **(b)** Unit balls of ℓ_1, ℓ_2 and ℓ_∞ for the same $\mathbf{g}$: the minimiser $-\mathbf{t}^*$ of $\mathbf{g}^\top \mathbf{t}$ on each ball points in a different direction, because $\mathbf{g}$ is not axis-aligned. Adapted from [54, Fig. 1].

For a positive definite $\mathbf{B}$ and the norm $\|\mathbf{d}\|_\mathbf{B} = (\mathbf{d}^\top \mathbf{B}\,\mathbf{d})^{1/2}$ of (9), the proposition recovers the local model. With $\|\mathbf{t}\|_\mathbf{B} = 1$,

$$\begin{aligned}
&\mathbf{g}^\top \mathbf{t} = (\mathbf{B}^{-1/2}\mathbf{g})^\top (\mathbf{B}^{1/2}\mathbf{t}) \;\leq\; \|\mathbf{B}^{-1/2}\mathbf{g}\|\,\|\mathbf{B}^{1/2}\mathbf{t}\| \;=\; \|\mathbf{g}\|_{\mathbf{B}^{-1}} && \text{(Cauchy–Schwarz)} \\
&\mathbf{t}^* = \mathbf{B}^{-1}\mathbf{g} \,/\, \|\mathbf{g}\|_{\mathbf{B}^{-1}} && \text{(equality when } \mathbf{B}^{1/2}\mathbf{t} \parallel \mathbf{B}^{-1/2}\mathbf{g}\text{)} \\
&\mathbf{d}^* = -\alpha\,\|\mathbf{g}\|_{\mathbf{B}^{-1}} \cdot \mathbf{B}^{-1}\mathbf{g} \,/\, \|\mathbf{g}\|_{\mathbf{B}^{-1}} \;=\; -\alpha\,\mathbf{B}^{-1}\mathbf{g}. && \text{(by (100))}
\end{aligned}$$

This is the preconditioned step (28) with $\mathbf{B}_k = \mathbf{B}/\alpha$. Every fixed positive definite curvature matrix of Parts II and IV is therefore steepest descent under $\|\cdot\|_{\mathbf{B}}$: gradient descent is $\mathbf{B} = \mathbf{I}$, and for $\mathbf{B} = \mathbf{H}_k \succ 0$ the squared dual norm $\|\mathbf{g}\|^2_{\mathbf{H}_k^{-1}}$ is the Newton decrement (Definition 19).

For a norm that does not come from an inner product, the direction $\mathbf{t}^*$ is no longer a linear function of $\mathbf{g}$: ℓ_∞ gives $\operatorname{sign}(\mathbf{g})$ (§23.2) and the spectral norm gives $\mathbf{U}\mathbf{V}^\top$ (§23.3). No matrix $\mathbf{B}$ produces these updates, and this is what the norm view adds beyond preconditioning.

Adam and Muon use the direction $\mathbf{t}^*$ of their norm but not the length $\alpha\,\|\mathbf{g}\|_\dagger$: they step a fixed learning rate along $-\mathbf{t}^*$, set by a schedule. Proposition 57 permits this, since once the norm is chosen the direction is settled and the step size can be found by any other means [54]. The correspondences in this part hold with the exponential moving averages of momentum and second moments switched off; the averages smooth out minibatch noise, and their precise role in this picture is open [54].

22.2 Induced Operator Norms

A norm on the flattened parameter vector ignores that each weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is an operator mapping layer inputs to layer outputs. Norms that respect this role are induced from norms on the input and output spaces.

Definition 25 (Induced operator norm [54, Def. 1])**.** Given norms $\|\cdot\|_a$ on $\mathbb{R}^{d_{\text{in}}}$ and $\|\cdot\|_b$ on $\mathbb{R}^{d_{\text{out}}}$, the *a to b operator norm* of $\mathbf{M} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is

$$\|\mathbf{M}\|_{a\to b} \coloneqq \max_{\mathbf{x}\neq\mathbf{0}} \frac{\|\mathbf{M}\mathbf{x}\|_b}{\|\mathbf{x}\|_a}.$$

The $\ell_2 \to \ell_2$ case is the spectral norm $\|\mathbf{M}\|_{\text{op}} = \sigma_1(\mathbf{M})$ of Definition 8. Two further families have closed forms [54, Prop. 8]: for $1 \le p \le \infty$,

$$\|\mathbf{M}\|_{\ell_1\to\ell_p} = \max_j \|\operatorname{col}_j(\mathbf{M})\|_p, \qquad \|\mathbf{M}\|_{\ell_p\to\ell_\infty} = \max_i \|\operatorname{row}_i(\mathbf{M})\|_{p/(p-1)}. \tag{101}$$

The first holds because the extreme points of the ℓ_1 unit ball are the one-hot vectors $\pm\mathbf{e}_j$, so the worst input selects a single column; the second because each output entry is a dot product with a row, bounded by Hölder's inequality with the dual exponent. In particular $\|\mathbf{M}\|_{\ell_1\to\ell_\infty} = \max_{i,j}|M_{ij}|$, the largest entry.

The choice of input and output norms should follow what the layer does [54]. Writing $\|\mathbf{x}\|_{\text{RMS}} \coloneqq \|\mathbf{x}\|_2/\sqrt{d}$ for $\mathbf{x} \in \mathbb{R}^d$, the two common cases are:

- *Linear layers* map activation vectors with entries of order one to vectors with entries of order one, so they get the RMS to RMS norm. It is a rescaled spectral norm:

$$\|\mathbf{W}\|_{\text{RMS}\to\text{RMS}} = \max_{\mathbf{x}\neq\mathbf{0}} \frac{\|\mathbf{W}\mathbf{x}\|_2/\sqrt{d_{\text{out}}}}{\|\mathbf{x}\|_2/\sqrt{d_{\text{in}}}} = \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}}\,\|\mathbf{W}\|_{\text{op}}.$$

- *Embedding layers* map one-hot token vectors (unit ℓ_1 norm) to vectors with entries of order one, so they get the ℓ_1 to RMS norm, which by (101) is $\max_j \|\operatorname{col}_j(\mathbf{W})\|_2/\sqrt{d_{\text{out}}}$.

The two layers share the weight space $\mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ but receive different norms, and hence different steepest-descent updates. Combining a norm per layer into one norm on the whole network, as a weighted maximum over layers, gives the *modular norm*, which Bernstein and Newhouse propose as a principled way to make these assignments [54].

22.3 Majorization–Minimization as the Design Pattern

Proposition 57 says how to step once a norm and α are chosen. Majorization–minimization (MM) says where they come from: bound the loss from above, then minimise the bound.

Gradient descent is already of this form. For L-smooth f, the descent lemma (17) gives an upper bound on f around $\boldsymbol{\theta}_k$ that is tight at $\mathbf{d} = \mathbf{0}$:

$$f(\boldsymbol{\theta}_k + \mathbf{d}) \le f(\boldsymbol{\theta}_k) + \mathbf{g}^\top\mathbf{d} + \frac{L}{2}\|\mathbf{d}\|^2 \qquad \forall\,\mathbf{d}.$$

The right-hand side is (98) with the Euclidean norm and $\alpha = 1/L$, so its minimiser $\mathbf{d} = -\mathbf{g}/L$ is the gradient-descent step of Lemma 23: with $\alpha = 1/L$, gradient descent exactly minimises the descent-lemma bound [15, §3.6].

In general, a surrogate Q_k *majorizes* f at $\boldsymbol{\theta}_k$ if $Q_k(\boldsymbol{\theta}) \ge f(\boldsymbol{\theta})$ for all $\boldsymbol{\theta}$ and $Q_k(\boldsymbol{\theta}_k) = f(\boldsymbol{\theta}_k)$. The MM step $\boldsymbol{\theta}_{k+1} = \arg\min_{\boldsymbol{\theta}} Q_k(\boldsymbol{\theta})$ never increases f [1, §8.7.1]:

$$f(\boldsymbol{\theta}_{k+1}) \le Q_k(\boldsymbol{\theta}_{k+1}) \le Q_k(\boldsymbol{\theta}_k) = f(\boldsymbol{\theta}_k),$$

using in turn that Q_k majorizes f, that $\boldsymbol{\theta}_{k+1}$ minimises Q_k, and that the bound is tight at $\boldsymbol{\theta}_k$. No convexity is needed. Steepest descent under a norm is therefore an MM step whenever f obeys a descent-lemma bound in that norm, $f(\boldsymbol{\theta}+\mathbf{d}) \le f(\boldsymbol{\theta}) + \mathbf{g}^\top \mathbf{d} + \frac{1}{2\alpha}\|\mathbf{d}\|^2$. The simplest non-Euclidean example is the square loss of a linear layer, which is bounded in the spectral norm. Here $\langle \mathbf{A}, \mathbf{B} \rangle := \operatorname{tr}(\mathbf{A}^\top \mathbf{B})$ is the Frobenius inner product.

Proposition 58 (Square loss of a linear predictor [54, Prop. 6])**.** *Let* $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ *map inputs* $\mathbf{x}_i$ *with* $\|\mathbf{x}_i\|_2 = \sqrt{d_{\text{in}}}$ *to predictions of targets* $\mathbf{y}_i$, $i = 1, \dots, n$, *with loss*

$$f(\mathbf{W}) \;=\; \frac{1}{2n} \sum_{i=1}^{n} \frac{1}{d_{\text{out}}} \|\mathbf{y}_i - \mathbf{W}\mathbf{x}_i\|_2^2.$$

Then for every update $\Delta\mathbf{W}$,

$$f(\mathbf{W}+\Delta\mathbf{W}) \;\le\; f(\mathbf{W}) + \langle \nabla_{\mathbf{W}} f, \Delta\mathbf{W} \rangle + \frac{1}{2}\,\frac{d_{\text{in}}}{d_{\text{out}}}\,\|\Delta\mathbf{W}\|_{\text{op}}^2. \tag{102}$$

Proof. Write $\mathbf{r}_i := \mathbf{y}_i - \mathbf{W}\mathbf{x}_i$. The loss is quadratic in $\mathbf{W}$, so expanding the square is exact:

$$\begin{aligned}
f(\mathbf{W}+\Delta\mathbf{W}) &= \frac{1}{2n\,d_{\text{out}}} \sum_i \|\mathbf{r}_i - \Delta\mathbf{W}\mathbf{x}_i\|_2^2 \\
&= f(\mathbf{W}) - \frac{1}{n\,d_{\text{out}}} \sum_i \mathbf{r}_i^\top \Delta\mathbf{W}\,\mathbf{x}_i + \frac{1}{2n\,d_{\text{out}}} \sum_i \|\Delta\mathbf{W}\mathbf{x}_i\|_2^2 \\
&= f(\mathbf{W}) + \Big\langle -\frac{1}{n\,d_{\text{out}}} \sum_i \mathbf{r}_i \mathbf{x}_i^\top,\; \Delta\mathbf{W} \Big\rangle + \frac{1}{2n\,d_{\text{out}}} \sum_i \|\Delta\mathbf{W}\mathbf{x}_i\|_2^2 \quad (\mathbf{r}^\top \Delta\mathbf{W}\mathbf{x} = \operatorname{tr}(\mathbf{x}\mathbf{r}^\top \Delta\mathbf{W})) \\
&\le f(\mathbf{W}) + \langle \nabla_{\mathbf{W}} f, \Delta\mathbf{W} \rangle + \frac{1}{2n\,d_{\text{out}}} \sum_i \|\Delta\mathbf{W}\|_{\text{op}}^2 \,\|\mathbf{x}_i\|_2^2. \qquad \text{(Definition 8)}
\end{aligned}$$

The matrix in the inner product is $\nabla_{\mathbf{W}} f$, and $\|\mathbf{x}_i\|_2^2 = d_{\text{in}}$ turns the last term into $\frac{1}{2}\frac{d_{\text{in}}}{d_{\text{out}}}\|\Delta\mathbf{W}\|_{\text{op}}^2$. □

Minimising the right-hand side of (102) over $\Delta\mathbf{W}$ is steepest descent under the spectral norm (§23.3), so for this loss spectral descent is an MM method with a guaranteed decrease. By §22.2, $\frac{d_{\text{in}}}{d_{\text{out}}}\|\Delta\mathbf{W}\|_{\text{op}}^2 = \|\Delta\mathbf{W}\|_{\text{RMS}\to\text{RMS}}^2$, so the bound is exactly $\frac{1}{2}\|\Delta\mathbf{W}\|_{\text{RMS}\to\text{RMS}}^2$: the natural norm of a linear layer is the one in which its loss has a clean descent lemma, and Muon (§23.5) steps, approximately, along the steepest-descent direction of this norm.

Newton's method also minimises a quadratic model, but the second-order Taylor model is an approximation, not a bound. Minimising it can overshoot or even increase f far from a minimum [1, §8.7.1], which is why it needs damping or a trust region (§14). An MM step cannot increase f; the price is that a global bound may be loose, and then the steps are short.

23 INSTANCES: FROM GRADIENT DESCENT TO MUON

For a weight matrix the steepest-descent problem (98) reads $\min_{\Delta\mathbf{W}} \langle \nabla_{\mathbf{W}} f, \Delta\mathbf{W} \rangle + \frac{1}{2\alpha}\|\Delta\mathbf{W}\|^2$, with the Frobenius inner product of §22.3. Throughout, $\nabla_{\mathbf{W}} f = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$ is the reduced SVD of the gradient matrix (§2.3).

23.1 Table of Norms and Optimizers

Table 4 collects the instances of Proposition 57 derived in this section and in §22.1.

Domain	Norm	Dual norm	Solution $\mathbf{d}^*$	Optimizer
$\mathbb{R}^P$	Euclidean ℓ_2	ℓ_2	$-\alpha\,\mathbf{g}$	gradient descent
$\mathbb{R}^P$	$\|\cdot\|_{\mathbf{B}}$, $\mathbf{B} \succ 0$	$\|\cdot\|_{\mathbf{B}^{-1}}$	$-\alpha\,\mathbf{B}^{-1}\mathbf{g}$	preconditioned GD (Newton, GGN)
$\mathbb{R}^P$	Fisher $\|\cdot\|_{\mathbf{F}(\boldsymbol{\theta})}$	$\|\cdot\|_{\mathbf{F}(\boldsymbol{\theta})^{-1}}$	$-\alpha\,\mathbf{F}^{-1}\mathbf{g}$	natural gradient
$\mathbb{R}^P$	ℓ_∞	ℓ_1	$-\alpha\,\|\mathbf{g}\|_1 \operatorname{sign}(\mathbf{g})$	sign descent (Adam without EMA)
$\mathbb{R}^{m\times n}$	Frobenius	Frobenius	$-\alpha\,\nabla_{\mathbf{W}} f$	gradient descent
$\mathbb{R}^{m\times n}$	spectral $\|\cdot\|_{\text{op}}$	nuclear $\operatorname{tr}\boldsymbol{\Sigma}$	$-\alpha\,(\operatorname{tr}\boldsymbol{\Sigma})\,\mathbf{U}\mathbf{V}^\top$	spectral descent (Shampoo, Muon)

Table 4: Optimizers as steepest descent (98) under different norms. The ℓ_∞ and spectral rows are nonlinear in the gradient; every other row is a preconditioned gradient. Extended from [54, Table 1].

23.2 Sign Descent and Adam

Ignoring bias correction and the stabilising ϵ, Adam (§12.5) keeps exponential averages of the gradient and its entrywise square and steps along their ratio:

$$\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1-\beta_1)\,\mathbf{g}_k, \qquad \mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1-\beta_2)\,\mathbf{g}_k^2, \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta\,\mathbf{m}_k/\sqrt{\mathbf{v}_k},$$

with all operations entrywise. Switching the averages off, $\beta_1 = \beta_2 = 0$, leaves $\mathbf{m}_k/\sqrt{\mathbf{v}_k} = \mathbf{g}_k/|\mathbf{g}_k| = \operatorname{sign}(\mathbf{g}_k)$, so Adam without EMA is *sign descent* [54]. Sign descent is in turn steepest descent under the ℓ_∞ norm.

Corollary 59 (Sign descent [54, Prop. 2])**.** *Under* $\|\mathbf{d}\|_\infty = \max_i |d_i|$*, the steepest-descent step* (98) *is* $\mathbf{d}^* = -\alpha\,\|\mathbf{g}\|_1 \operatorname{sign}(\mathbf{g})$.

Proof. For $\|\mathbf{t}\|_\infty = 1$, $\mathbf{g}^\top \mathbf{t} = \sum_i g_i t_i \le \sum_i |g_i| = \|\mathbf{g}\|_1$, with equality at $\mathbf{t} = \operatorname{sign}(\mathbf{g})$. So $\|\mathbf{g}\|_\dagger = \|\mathbf{g}\|_1$ and $\mathbf{t}^* = \operatorname{sign}(\mathbf{g})$; apply Proposition 57. □

The ℓ_∞ norm of the flattened parameter vector looks unrelated to the layered structure of a network, but a max of a max is a max: by (101), $\|\boldsymbol{\theta}\|_\infty = \max_l \max_{i,j} |(\mathbf{W}_l)_{ij}| = \max_l \|\mathbf{W}_l\|_{\ell_1 \to \ell_\infty}$, so sign descent is also steepest descent under this *max-of-max norm*, with each layer moving along $-\operatorname{sign}(\nabla_{\mathbf{W}_l} f)$, an update normalised per matrix [54, Prop. 3]. The $\ell_1 \to \ell_\infty$ norm is not, however, the RMS to RMS norm that §22.2 assigns to linear layers, which leads to spectral descent.

23.3 Spectral Descent: Shampoo and Muon

Shampoo [49] (§20) preconditions each layer's gradient from both sides, accumulating $\mathbf{L}_k = \mathbf{L}_{k-1} + \nabla_{\mathbf{W}} f\,(\nabla_{\mathbf{W}} f)^\top$ and $\mathbf{R}_k = \mathbf{R}_{k-1} + (\nabla_{\mathbf{W}} f)^\top \nabla_{\mathbf{W}} f$ and stepping $\mathbf{W}_{k+1} = \mathbf{W}_k - \eta\,\mathbf{L}_k^{-1/4} \nabla_{\mathbf{W}} f\, \mathbf{R}_k^{-1/4}$. With the accumulation switched off, the preconditioners are formed from the current gradient alone, and the reduced SVD collapses the update:[23]

$$\begin{aligned}
\left(\nabla_{\mathbf{W}} f\,(\nabla_{\mathbf{W}} f)^\top\right)^{-1/4} \nabla_{\mathbf{W}} f\,\left((\nabla_{\mathbf{W}} f)^\top \nabla_{\mathbf{W}} f\right)^{-1/4} &= (\mathbf{U}\boldsymbol{\Sigma}^2\mathbf{U}^\top)^{-1/4}\,\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top\,(\mathbf{V}\boldsymbol{\Sigma}^2\mathbf{V}^\top)^{-1/4} \\
&= \mathbf{U}\boldsymbol{\Sigma}^{-1/2}\mathbf{U}^\top\,\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top\,\mathbf{V}\boldsymbol{\Sigma}^{-1/2}\mathbf{V}^\top && \text{(Definition 7)} \\
&= \mathbf{U}\,\boldsymbol{\Sigma}^{-1/2}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^{-1/2}\,\mathbf{V}^\top \;=\; \mathbf{U}\mathbf{V}^\top. && (\mathbf{U}^\top\mathbf{U} = \mathbf{V}^\top\mathbf{V} = \mathbf{I})
\end{aligned}$$

Accumulation-free Shampoo replaces the gradient by $\mathbf{U}\mathbf{V}^\top$, which keeps its singular vectors and sets every nonzero singular value to one [54]. This is the matrix analogue of $\operatorname{sign}(\mathbf{g})$, and like sign it has two readings: a projection, and a steepest-descent direction.

Proposition 60 (Closest semi-orthogonal matrix [54, Prop. 4])**.** *Let* $\mathcal{O}_{m\times n} := \{\mathbf{A} \in \mathbb{R}^{m\times n} : \mathbf{A}\mathbf{A}^\top = \mathbf{I}_m \text{ or } \mathbf{A}^\top\mathbf{A} = \mathbf{I}_n\}$ *be the semi-orthogonal matrices, and let* $\mathbf{M} = \sum_{i=1}^{\min(m,n)} \sigma_i\,\mathbf{u}_i\mathbf{v}_i^\top$ *be an SVD with* $\min(m,n)$ *singular pairs, some* σ_i *possibly zero. Then*

$$\arg\min_{\mathbf{A}\in\mathcal{O}_{m\times n}} \|\mathbf{A}-\mathbf{M}\|_F \;=\; \mathbf{U}\mathbf{V}^\top \;=\; \sum_i \mathbf{u}_i\mathbf{v}_i^\top,$$

and the minimiser is unique if and only if $\mathbf{M}$ *has full rank.*

Proof. Expanding the square, $\|\mathbf{A}-\mathbf{M}\|_F^2 = \|\mathbf{A}\|_F^2 - 2\langle \mathbf{A}, \mathbf{M}\rangle + \|\mathbf{M}\|_F^2$, and $\|\mathbf{A}\|_F^2 = \min(m,n)$ is the same for every semi-orthogonal $\mathbf{A}$. Minimising the distance is therefore maximising the alignment $\langle \mathbf{A}, \mathbf{M}\rangle$:

$$\begin{aligned}
\langle \mathbf{A}, \mathbf{M}\rangle = \operatorname{tr}\Big(\mathbf{A}^\top \sum_i \sigma_i\,\mathbf{u}_i\mathbf{v}_i^\top\Big) &= \sum_i \sigma_i\,\mathbf{u}_i^\top \mathbf{A}\,\mathbf{v}_i && \text{(cyclic trace)} \\
&\le \sum_i \sigma_i, && (\mathbf{u}^\top\mathbf{A}\mathbf{v} \le \|\mathbf{A}\|_{\mathrm{op}} = 1 \text{ for unit } \mathbf{u},\mathbf{v})
\end{aligned}$$

and $\mathbf{A} = \sum_j \mathbf{u}_j\mathbf{v}_j^\top$ attains the bound, since orthonormality gives $\mathbf{u}_i^\top\mathbf{u}_j\mathbf{v}_j^\top\mathbf{v}_i = \delta_{ij}$. If every $\sigma_i > 0$, attaining the bound forces $\mathbf{u}_i^\top\mathbf{A}\mathbf{v}_i = 1$ for all i, hence $\mathbf{A}\mathbf{v}_i = \mathbf{u}_i$ and $\mathbf{A}^\top\mathbf{u}_i = \mathbf{v}_i$, which determines $\mathbf{A} = \mathbf{U}\mathbf{V}^\top$. If some $\sigma_i = 0$, flipping the sign of that $\mathbf{v}_i$ gives a second minimiser. □

Proposition 61 (Spectral descent [54, Prop. 5])**.** *Under the spectral norm* $\|\Delta\mathbf{W}\|_{\mathrm{op}}$*, the steepest-descent step is*

$$\Delta\mathbf{W}^* \;=\; -\alpha\,(\operatorname{tr}\boldsymbol{\Sigma})\,\mathbf{U}\mathbf{V}^\top, \tag{103}$$

unique if and only if $\nabla_{\mathbf{W}} f$ *has full rank. The dual norm is the nuclear norm* $\|\nabla_{\mathbf{W}} f\|_* := \operatorname{tr}\boldsymbol{\Sigma} = \sum_i \sigma_i$.

23 When $\nabla_{\mathbf{W}} f$ is rank-deficient, the inverse roots are taken on its range, as for the pseudoinverse of Lemma 7.

Proof. By Proposition 57 we need the dual norm and the maximising direction. For $\|\mathbf{T}\|_{\mathrm{op}} = 1$, the same alignment bound as in Proposition 60 gives

$$\langle \nabla_{\mathbf{W}} f, \mathbf{T} \rangle \;=\; \sum_i \sigma_i \, \mathbf{u}_i^\top \mathbf{T} \, \mathbf{v}_i \;\le\; \sum_i \sigma_i \;=\; \operatorname{tr} \mathbf{\Sigma},$$

attained by $\mathbf{T} = \mathbf{U}\mathbf{V}^\top$, which has $\|\mathbf{U}\mathbf{V}^\top\|_{\mathrm{op}} = 1$. So $\|\nabla_{\mathbf{W}} f\|_\dagger = \operatorname{tr} \mathbf{\Sigma}$ and $\mathbf{t}^* = \mathbf{U}\mathbf{V}^\top$, giving (103). Uniqueness follows as in Proposition 60. □

For a network with layers $\mathbf{W}_1, \dots, \mathbf{W}_L$ under the norm $\max_l \|\Delta \mathbf{W}_l\|_{\mathrm{op}}$, the same argument gives $\Delta \mathbf{W}_l^* = -\alpha \left(\sum_k \operatorname{tr} \mathbf{\Sigma}_k\right) \mathbf{U}_l \mathbf{V}_l^\top$: every layer moves along its own $\mathbf{U}_l \mathbf{V}_l^\top$ with one global step [54, Prop. 5]. Accumulation-free Shampoo takes exactly these directions with a fixed step η in place of $\alpha \sum_k \operatorname{tr} \mathbf{\Sigma}_k$, so it is spectral steepest descent, and Proposition 58 makes it an MM method for a linear layer under square loss. Setting every nonzero singular value to one gives all directions in the range of the gradient equal weight: gradients of transformer weight matrices are typically close to low rank, and orthogonalisation raises the small directions that plain gradient descent would barely move [55].

23.4 Natural Gradient as the Fisher Norm

Natural gradient descent (§16.4) is the $\|\cdot\|_{\mathbf{B}}$ case of §22.1 with $\mathbf{B} = \mathbf{F}(\boldsymbol{\theta}_k) \succ 0$: steepest descent under the Fisher norm $\|\mathbf{d}\|_{\mathbf{F}} := (\mathbf{d}^\top \mathbf{F}\, \mathbf{d})^{1/2}$ gives $-\alpha \mathbf{F}^{-1}\mathbf{g}$, the update (90), with dual norm $\|\mathbf{g}\|_{\mathbf{F}^{-1}}$. By Proposition 48, $\frac{1}{2}\|\mathbf{d}\|_{\mathbf{F}}^2$ is the KL divergence between the model before and after the step, so this norm measures a step by how much it changes the predicted distribution. Since $\mathbf{F}_\gamma = \mathbf{J}_\zeta^\top \mathbf{F}_{\boldsymbol{\theta}} \mathbf{J}_\zeta$ under a reparameterisation $\boldsymbol{\theta} = \zeta(\gamma)$, a step and its image have the same Fisher length, which is the reparameterisation invariance of §16.4. The norms of gradient, sign and spectral descent are fixed in the chosen coordinates instead, which is why §22.2 chooses them to match what each layer computes.

23.5 Muon in Practice

Muon [55] applies spectral descent to the hidden-layer weight matrices, with momentum accumulated before orthogonalising:

$$\mathbf{M}_k = \beta\, \mathbf{M}_{k-1} + \nabla_{\mathbf{W}} f(\mathbf{W}_{k-1}), \qquad \mathbf{W}_k = \mathbf{W}_{k-1} - \eta \operatorname{Ortho}(\mathbf{M}_k), \tag{104}$$

where $\operatorname{Ortho}(\mathbf{M}) := \mathbf{U}\mathbf{V}^\top$ for $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ is the closest semi-orthogonal matrix (Proposition 60). Since $\operatorname{Ortho}(c\,\mathbf{M}) = \operatorname{Ortho}(\mathbf{M})$ for $c > 0$, the $(1-\beta)$ normalisation of Adam's first moment would change nothing. The public implementation uses Nesterov momentum, $\operatorname{Ortho}(\beta \mathbf{M}_k + \nabla_{\mathbf{W}} f)$, which was slightly better in every case tested. With $\beta = 0$, Muon is accumulation-free Shampoo with a fixed step (§23.3). Only two-dimensional hidden-layer matrices use Muon. Scalars, vectors, the embedding and the output head are trained with AdamW: the embedding exception matches its ℓ_1 to RMS norm (§22.2), while the head exception is empirical [55].

NEWTON–SCHULZ ITERATION. An SVD per step is too slow on GPUs, and the coupled Newton iteration Shampoo uses for inverse roots needs float32 to stay stable. Muon instead applies an odd matrix polynomial built only from matrix products, which is stable in bfloat16 [54, 55]. One step with coefficients (a, b, c) acts on the singular values alone. With $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$,

$$\begin{aligned} a\mathbf{X} + b(\mathbf{X}\mathbf{X}^\top)\mathbf{X} + c(\mathbf{X}\mathbf{X}^\top)^2\mathbf{X} &= a\,\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top + b\,\mathbf{U}\mathbf{\Sigma}^2\mathbf{U}^\top\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top + c\,\mathbf{U}\mathbf{\Sigma}^4\mathbf{U}^\top\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \qquad (\mathbf{X}\mathbf{X}^\top = \mathbf{U}\mathbf{\Sigma}^2\mathbf{U}^\top) \\ &= \mathbf{U}\left(a\mathbf{\Sigma} + b\mathbf{\Sigma}^3 + c\mathbf{\Sigma}^5\right)\mathbf{V}^\top. \qquad (\mathbf{U}^\top\mathbf{U} = \mathbf{I}) \end{aligned}$$

After N steps the output is $\mathbf{U}\varphi^N(\mathbf{\Sigma})\mathbf{V}^\top$, with $\varphi(x) = ax + bx^3 + cx^5$ applied to each singular value and the singular vectors untouched. The iteration therefore reaches $\operatorname{Ortho}(\mathbf{M}_k)$ exactly when $\varphi^N(\sigma_i) \to 1$ for every σ_i. It starts from $\mathbf{X}_0 = \mathbf{M}_k / \|\mathbf{M}_k\|_F$, which puts every σ_i in $(0, 1]$, since $\sigma_1^2 \le \sum_i \sigma_i^2 = \|\mathbf{M}_k\|_F^2$, and leaves Ortho unchanged.

The coefficients are chosen so that φ^N pushes every value in $(0, 1]$ towards 1. The simplest choice, the cubic $\varphi(x) = \frac{3}{2}x - \frac{1}{2}x^3$, converges to 1 from any start in $(0, \sqrt{3})$ [54, App. A], but $\varphi'(0) = \frac{3}{2}$, so a small singular value grows only by a factor $\frac{3}{2}$ per step; and since momentum matrices are close to low rank, most of their normalised singular values are small. Jordan et al. therefore tune a quintic that grows small singular values much faster, at the price of bringing them only into a band around 1 rather than to 1 exactly, which empirically does not hurt the loss [55]. Muon is therefore approximately, not exactly, spectral descent.

UPDATE SCALE AND WEIGHT DECAY. Since $\|\mathbf{U}\mathbf{V}^\top\|_F^2 = \operatorname{tr}(\mathbf{V}\mathbf{U}^\top\mathbf{U}\mathbf{V}^\top) = \operatorname{rank}$, a full-rank $\mathbf{U}\mathbf{V}^\top \in \mathbb{R}^{m\times n}$ has root-mean-square entry $1/\sqrt{\max(m,n)}$, which shrinks with the layer's size; Liu et al. rescale the update by $0.2\sqrt{\max(m,n)}$ to match AdamW's typical update RMS, so that AdamW's learning rate and weight decay can be reused [56, Lemma 1]. They also find that over long runs the weights grow unless decoupled weight decay is

added, $\mathbf{W}_k = \mathbf{W}_{k-1} - \eta(\mathrm{Ortho}(\mathbf{M}_k) + \lambda \mathbf{W}_{k-1})$. It must be decoupled for the same reason as in AdamW (§12.6): Ortho is nonlinear, so an ℓ_2 penalty added to the gradient before orthogonalisation does not act as a shrinkage of $\mathbf{W}$ [56].

REFERENCES

[1] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.

[2] Yurii E. Nesterov. *Lectures on convex optimization*. Springer optimization and its applications ; Volume 137. Springer, Cham, Switzerland, second edition. edition, 2018.

[3] Hamed Karimi, Julie Nutini, and Mark Schmidt. Linear convergence of gradient and proximal-gradient methods under the polyak-łojasiewicz condition, 2020.

[4] Zach Liu. 3M1 optimization notes. Personal notes for Part IIA 3M1 Mathematical Methods, University of Cambridge, 2025.

[5] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey, 2018.

[6] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. Society for Industrial and Applied Mathematics, USA, second edition, 2008.

[7] Yurii Nesterov and Vladimir Spokoiny. Random gradient-free minimization of convex functions. *Foundations of Computational Mathematics*, 17(2):527–566, 2017.

[8] Atılım Güneş Baydin, Barak A. Pearlmutter, Don Syme, Frank Wood, and Philip Torr. Gradients without backpropagation, 2022.

[9] Matilde Gargiani, Andrea Zanelli, Moritz Diehl, and Frank Hutter. On the promise of the stochastic generalized Gauss-Newton method for training DNNs. *arXiv preprint arXiv:2006.02409*, 2020.

[10] James Martens. New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76, 2020.

[11] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, New York, NY, 2 edition, 2006.

[12] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

[13] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.

[14] Yurii Nesterov. A method for solving the convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady*, 27(2):372–376, 1983.

[15] Bernd Gärtner and Martin Jaggi. Optimization for machine learning. Lecture Notes CS-439, EPFL, 2025.

[16] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.

[17] Aaron Defazio, Francis Bach, and Simon Lacoste-Julien. SAGA: A fast incremental gradient method with support for non-strongly convex composite objectives. In *Advances in Neural Information Processing Systems*, 2014.

[18] Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.

[19] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.

[20] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[21] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour, 2017.

[22] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training, 2018.

[23] Jingzhao Zhang, Tianxing He, Suvrit Sra, and Ali Jadbabaie. Why gradient clipping accelerates training: A theoretical justification for adaptivity. In *International Conference on Learning Representations*, 2020.

[24] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.

[25] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5 – RMSProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.

[26] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[27] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pages 4596–4604, 2018.

[28] James Martens and Ilya Sutskever. Training deep and recurrent networks with hessian-free optimization. In Grégoire Montavon, Geneviève B. Orr, and Klaus-Robert Müller, editors, *Neural Networks: Tricks of the Trade*, volume 7700 of *Lecture Notes in Computer Science*. Springer, 2012.

[29] Jorge J. Moré. The Levenberg–Marquardt algorithm: Implementation and theory. In *Numerical Analysis*, volume 630 of *Lecture Notes in Mathematics*, pages 105–116. Springer, 1978.

[30] James Martens and Ilya Sutskever. Learning recurrent neural networks with hessian-free optimization. In *Proceedings of the 28th International Conference on Machine Learning*, pages 1033–1040, 2011.

[31] Nicol N. Schraudolph. Fast curvature matrix-vector products for second-order gradient descent. *Neural Computation*, 14(7):1723–1738, 2002.

[32] Aleksandar Botev, Hippolyt Ritter, and David Barber. Practical Gauss-Newton optimisation for deep learning. In *Proceedings of the 34th International Conference on Machine Learning*, pages 557–565, 2017.

[33] Aleksandar Botev. *The Gauss-Newton matrix for Deep Learning models and its applications*. PhD thesis, University College London, 2020.

[34] Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 02 1998.

[35] Kevin P. Murphy. Probabilistic machine learning: Advanced topics. http://probml.github.io/book2, 2023.

[36] James Martens and Roger Grosse. Optimizing neural networks with Kronecker-factored approximate curvature. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 2408–2417, 2015.

[37] Frederik Kunstner, Lukas Balles, and Philipp Hennig. Limitations of the empirical fisher approximation for natural gradient descent, 2020.

[38] Sue Becker and Yann LeCun. Improving the convergence of back-propagation learning with second order methods. In *Proceedings of the 1988 Connectionist Models Summer School*, pages 29–37. Morgan Kaufmann, 1989.

[39] Nicolas Le Roux, Pierre-Antoine Manzagol, and Yoshua Bengio. Topmoumoute online natural gradient algorithm. In *Advances in Neural Information Processing Systems 20*, pages 849–856, 2008.

[40] Olivier Chapelle and Dumitru Erhan. Improved preconditioner for Hessian free optimization. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011.

[41] Jim Zhao, Sidak Pal Singh, and Aurelien Lucchi. Theoretical characterisation of the Gauss-Newton conditioning in neural networks. In *Advances in Neural Information Processing Systems*, 2024.

[42] Vardan Papyan. The full spectrum of deep net hessians at scale: Dynamics with sample size, 2018.

[43] Behrooz Ghorbani, Shankar Krishnan, and Ying Xiao. An investigation into neural net optimization via hessian eigenvalue density. In *International Conference on Machine Learning*, 2019.

[44] Jeremy Cohen, Simran Kaur, Yuanzhi Li, J. Zico Kolter, and Ameet Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. In *International Conference on Learning Representations*, 2021.

[45] Chaoyue Liu, Libin Zhu, and Mikhail Belkin. Loss landscapes and optimization in over-parameterized nonlinear systems and neural networks. *Applied and Computational Harmonic Analysis*, 59:85–116, 2022.

[46] James Martens. Deep learning via Hessian-free optimization. In *Proceedings of the 27th International Conference on Machine Learning*, pages 735–742, 2010.

[47] Trond Steihaug. The conjugate gradient method and trust regions in large scale optimization. *SIAM Journal on Numerical Analysis*, 20(3):626–637, 1983.

[48] Barak A. Pearlmutter. Fast exact multiplication by the Hessian. *Neural Computation*, 6(1):147–160, 1994.

[49] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning*, pages 1842–1850, 2018.

[50] Thomas George, César Laurent, Xavier Bouthillier, Nicolas Ballas, and Pascal Vincent. Fast approximate natural gradient descent in a Kronecker-factored eigenbasis. In *Advances in Neural Information Processing Systems 31*, 2018.

[51] Yuhuai Wu, Elman Mansimov, Shun Liao, Roger Grosse, and Jimmy Ba. Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation. In *Advances in Neural Information Processing Systems 30*, 2017.

[52] Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing Shampoo using Adam. In *International Conference on Learning Representations*, 2025.

[53] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, 2015.

[54] Jeremy Bernstein and Laker Newhouse. Old optimizer, new norm: An anthology, 2024.

[55] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024.

[56] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, et al. Muon is scalable for LLM training, 2025.